



**Ministère de l'Enseignement Supérieur
Et de la Recherche Scientifique**



Rapport de Projet de fin d'études

Filière : Licence en Génie Logiciel et Systèmes d'Information

Réalisé par :

Hosni Iyed
&
Elomri Tayssir

Encadré par :

Encadrant académique : Gassara Elyes

Encadrant professionnel : Ghezail Aymen

**Sujet : Mise en place d'une plateforme pour l'automatisation des application web
multi-service via l'approche CI/CD**



Année Universitaire : 2024/2025

Note au lecteur

Nous sommes conscients que la lecture de ce rapport peut sembler longue au premier abord, et nous tenons à vous remercier pour le temps et l'attention que vous lui consacrez.

Chaque section présentée ici a été soigneusement rédigée, non pas par excès de détails, mais parce que **chaque partie du projet apporte une brique essentielle à la compréhension globale de notre travail**. Ce projet, par son ampleur, **dépasse largement le cadre habituel d'un PFE de licence en Génie Logiciel et Systèmes d'Information**, et c'est justement cette ambition qui justifie sa richesse.

Nous avons fait tout notre possible pour rester clairs, structurés et concis tout en couvrant **l'ensemble des éléments innovants et techniques** que nous avons dû maîtriser en seulement quatre mois : DevOps, CI/CD, infrastructure distribuée, sécurité, modularité du code, interactions avec des outils industriels comme Jenkins, SonarQube, Nexus, Kubernetes, GitHub, etc.

Si ce rapport est long, c'est aussi parce que **le lecteur pourra y trouver une vraie valeur ajoutée** : il est pensé comme un **véritable guide d'entrée dans l'univers DevOps**, accessible et détaillé, que ce soit pour un lecteur novice ou curieux, ou pour un professionnel souhaitant comprendre la mise en œuvre concrète de ces pratiques dans un environnement réel.

Merci encore pour votre lecture et votre compréhension.

Table des matières

Introduction Générale	1
Chapitre 1 : Contexte et cadrage du projet	3
Introduction	4
I. Cadre du projet	4
II. Présentation de l'organisme d'accueil	4
1) Les services fournis par l'organisme d'accueil	5
2) Les clients de 3S	6
3) Présentation du domaine métier.....	7
III. Analyse de l'existant	8
1) Présentation de l'existant	8
2) Critique de l'existant.....	10
3) Solution proposée	11
IV. Méthodologie de travail :	12
1) Adoption de Scrum dans ce projetf.....	14
2) Rôles dans Scrum.....	15
Conclusion.....	15
Chapitre 2 : Revue de l'état de l'art	16
Introduction	17
I. La culture DevOps	17
1) Définition et Objectifs.....	17
2) Principes de DevOps.....	18
3) Amélioration de la qualité du code	19
4) Conclusion sur les Avantages de DevOps	21
II. Choix des outils pour ce projet.....	21
1) Jenkins pour la gestion des pipelines CI/CD	22
2) SonarQube pour l'analyse de la qualité du code	23
3) Nexus pour la gestion des artefacts.....	24
4) Ubuntu pour les machines virtuelles	24

5) MobaXterm pour la connexion SSH.....	25
6) Kubernetes pour l'orchestration des conteneurs	25
7) Docker Hub pour la gestion des images conteneurisées	26
8) Grafana pour la visualisation des métriques.....	27
9) Prometheus pour la collecte des métriques	28
10) Node.js pour le développement backend.....	28
11) L'architecture de notre projet.....	29
12) Ressources Matérielles	29
Conclusion.....	31
Chapitre 3 : Sprint 0 - Spécification des besoins	32
Introduction	33
I. Spécification des besoins	33
1) Besoins fonctionnels	33
2) Besoins non fonctionnels	34
II. Backlog du produit	35
III. Diagramme de cas d'utilisation général	36
1) Description scénariste du cas d'utilisation : Créer un pipeline	37
2) Description scénariste du cas d'utilisation : Afficher le contenu d'un projet	38
IV. Architecture du système	39
V. Planification de la release.....	41
Conclusion.....	41
Chapitre 4 : Sprint 1 - Installation des outils DevOps	42
Introduction	43
I. Installation de Docker	43
1) Installation des paquets requis :.....	43
2) Ajout de la clé GPG de Docker	44
3) Ajout du dépôt Docker.....	44
4) Installation du moteur Docker	44

5) Vérification du statut de Docker.....	45
6) Utilisation de Docker sans sudo :	46
II. Mise en place de Jenkins.....	46
1) Création du volume persistant Jenkins.....	46
2) Création du Dockerfile personnalisé.....	46
3) Construction de l'image personnalisée	47
4) Lancement du conteneur Jenkins	48
5) Accès à Jenkins via le navigateur	48
6) Récupération du mot de passe administrateur Jenkins.....	49
III. Installation de Prometheus.....	Error! Bookmark not defined.
1) Téléchargement de l'image Prometheus	Error! Bookmark not defined.
2) Création de la configuration Prometheus	Error! Bookmark not defined.
3) Lancement du conteneur Prometheus	Error! Bookmark not defined.
4) Accès à Prometheus via le navigateur.....	Error! Bookmark not defined.
IV. Installation de Grafana.....	Error! Bookmark not defined.
1) Téléchargement et exécution	Error! Bookmark not defined.
2) Accès à Grafana via le navigateur.....	Error! Bookmark not defined.
V. Déploiement de SonarQube.....	49
1) Téléchargement de l'image.....	49
2) Exécution du conteneur	50
3) Accès à SonarQube via le navigateur.....	50
VI. Mise en place de Nexus Repository.....	51
1) Téléchargement de l'image Nexus.....	51
2) Lancement du conteneur	51
3) Accès à SonaType nexus via le navigateur	52
VII. Installation de Kubernetes avec kubeadm.....	53
1) Configuration des noms d'hôtes et du fichier /etc/hosts	53
2) Désactivation du swap.....	54

3) Chargement des modules noyaux et configuration sysctl	54
4) Installation d'exécution de conteneur	55
5) Ajout du dépôt Kubernetes et installation de kubeadm, kubelet et kubectl	57
6) Initialisation du nœud master	58
7) Configuration de kubectl sur le master	59
8) Installation du plugin réseau Flannel	59
9) Ajout des nœuds workers au cluster	59
10) Vérification du cluster	60
VIII. Résumé des outils	63
Conclusion	64
Chapitre 5: Sprint 2 – Mise en place d'une pipeline CI/CD complète avec Jenkins	65
Introduction	66
I. Architecture cible de la pipeline	66
1) Schéma logique de l'architecture	66
2) Fonctionnement global du pipeline	66
II. Installation des plugins Jenkins	67
1) Contexte	67
2) Liste des plugins indispensables	67
3) Procédure d'installation	67
4) Illustration de l'installation	68
III. Ajout des identifiants d'accès (Credentials)	68
1) Petit rappel sécurité :	69
2) Token SonarQube	69
3) Token DockerHub	69
4) Token GitHub	70
5) Ajout Credentials dans Jenkins	71
IV. Préparation du projet Flask/MongoDB	71
1) Fork du repository GitHub	72

2) Ajout du Dockerfile.....	72
3) Ajout de deployment.yaml.....	73
4) Modification de la connexion à la base de données.....	75
V. Configuration des notifications par email	75
1) Procédure.....	75
VI. Création de la pipeline Jenkins	76
1) Création du projet Pipeline	76
2) Ajout du script Pipeline	77
VII. Configuration du webhook GitHub	81
1) Hébergement de Jenkins avec ngrok.....	81
2) Ajout du webhook GitHub.....	82
VIII. Phase de test du pipeline CI/CD.....	83
1) pousse de test avec GitHub Desktop.....	83
2) Validation du webhook GitHub	83
3) Détection par Jenkins	84
4) Suivi de l'exécution du build.....	84
5) Résultat final du build	85
6) Vérifier SonarQube et Nexus.....	86
7) Teste pipeline avec autre application (PHP).....	87
8) Vérifier les pods de kubernetes.....	87
9) Test de l'alerte email	88
Conclusion.....	89
Bibliographie	1

Liste des Tableaux

Tableau 1 : comparaison des méthodologies Cascade et Scrum	13
Tableau 2 : Backlog des fonctionnalités CI/CD	36
Tableau 3 : Planification des sprints du projet.....	41
Tableau 4 : Résumé des outils	63
Tableau 5 : Liste des plugins indispensables.....	67

Liste des Figures

Figure 1 : Logo de 3s	4
Figure 2 : Historique du groupe 3s [2].....	5
Figure 3 : List des clients de 3S.....	7
Figure 4: Image de CI/CD pipeline [6]	18
Figure 5: logo de Jenkins	22
Figure 6: logo de SonarQube.....	23
Figure 7: logo de SonaType Nexus Repository	24
Figure 8: logo de Ubuntu	24
Figure 9: logo de MobaXterm	25
Figure 10 : logo de Kubernetes.....	25
Figure 11 : logo de Docker Hub	26
Figure 12 : logo de Grafana.....	27
Figure 13 : logo de Prometheus	28
Figure 14 : logo de Node.JS	28
Figure 15 : Architecture physique.....	29
Figure 16: Cas d'utilisation et systèmes impliqués	37
Figure 17 : Installation des paquets requis	43
Figure 18 : Ajout de la clé GPG de Docker.....	44
Figure 19 : Ajout du dépôt Docker	44
Figure 20 : Installation du moteur Docker	45
Figure 21 : Vérification du statut de Docker	45
Figure 22 : Utilisation de Docker sans sudo.....	46
Figure 23 : Création du volume persistant Jenkins.....	46
Figure 24 : Création du Dockerfile personnalisé	47
Figure 25 : Construction de l'image personnalisée	48
Figure 26 : Lancement du conteneur Jenkins	48

Figure 27 : Accès à Jenkins via le navigateur.....	49
Figure 28 : Récupération du mot de passe administrateur Jenkins.....	49
Figure 29 : Téléchargement de l'image Prometheus	Error! Bookmark not defined.
Figure 30 : Création de la configuration Prometheus	Error! Bookmark not defined.
Figure 31 : Lancement du conteneur Prometheus	Error! Bookmark not defined.
Figure 32 : Accès à Prometheus via le navigateur	Error! Bookmark not defined.
Figure 33 : Téléchargement et exécution Grafana	Error! Bookmark not defined.
Figure 34 : Accès à Grafana via le navigateur.....	Error! Bookmark not defined.
Figure 35 : Téléchargement de SonarQube	50
Figure 36 : Exécution du conteneur SonarQube.....	50
Figure 37 : Accès à SonarQube via le navigateur.....	51
Figure 38 : Téléchargement de l'image Nexus.....	51
Figure 39 : Lancement du conteneur nexus.....	52
Figure 40 : Accès à SonaType nexus via le navigateur	52
Figure 41 : Configuration des noms d'hôtes et du fichier /etc/hosts	53
Figure 42 : Désactivation du swap.....	54
Figure 43 : Chargement des modules noyaux et configuration sysctl	55
Figure 44 : Installation d'exécution de conteneur Kubernetes	56
Figure 45 : Ajout du dépôt Kubernetes et installation de kubeadm, kubelet et kubectl	58
Figure 46 : Initialisation du nœud master (Kubernetes)	59
Figure 47 : Installation du plugin réseau Flannel (Kubernetes)	59
Figure 48 : Ajout des nœuds workers au cluster Kubernetes	60
Figure 49 : Vérification du cluster Kubernetes	60
Figure 50 : L'architecture logique	66
Figure 51 : Illustration de l'installation d'un plugin Jenkins	68
Figure 52 : Token SonarQube.....	69
Figure 53 : Token DockerHub	70
Figure 54 : Token GitHub	70
Figure 55 : Credentials dans Jenkins	71
Figure 56 : Fork du repository GitHub	72
Figure 57 : Mise à jour de l'URI de connexion dans le code Flask.....	75
Figure 58 : Configuration des notifications par email	76
Figure 59 : Création du projet Pipeline.....	77
Figure 60 : Ajout du script Pipeline	78
Figure 61 : Hébergement de Jenkins avec ngrok.....	81

Figure 62 : Ajout du webhook GitHub.....	82
Figure 63 : pousse de test avec GitHub Desktop.....	83
Figure 64 : Validation du webhook GitHub	83
Figure 65 : Détection par Jenkins	84
Figure 66 : Suivi de l'exécution du build.....	84
Figure 67 : Build de l'application Flask-MongoDB réussi.....	85
Figure 68 : Application web Flask-MongoDB hébergée	85
Figure 69 : Analyse réussie du projet Flask-MongoDB dans SonarQube	86
Figure 70 : Déploiement du projet Flask-MongoDB dans le repository Nexus.....	86
Figure 71 : Build de l'application PHP-MySQL réussi	87
Figure 72 : Application web PHP-MySQL hébergée.....	87
Figure 73 : Les pods de kubernetes	88
Figure 74 : Notification d'échec du pipeline Jenkins	88
Figure 75 : Notification de reprise normale du pipeline Jenkins.....	88

Tableaux des abréviations

ABREVIATION	SIGNIFICATION
CI	Continuous Integration (Intégration Continue)
CD	Continuous Deployment (Déploiement Continu)
DEVOPS	Development & Operations
VM	Virtual Machine (Machine Virtuelle)
API	Application Programming Interface
SSH	Secure Shell
CPU	Central Processing Unit (Processeur)
IAC	Infrastructure as Code
ERP	Enterprise Resource Planning
CRM	Customer Relationship Management
PHP	Hypertext Preprocessor
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
K8S	Kubernetes
PO	Product Owner
US	User Story
GPU	Graphics Processing Unit
SSD	Solid State Drive
HDD	Hard Disk Drive
RAM	Random Access Memory

Introduction Générale

Dans un contexte où les cycles de développement logiciel s'accroissent et où les exigences en matière de qualité, de réactivité et d'automatisation deviennent cruciales, les pratiques DevOps et CI/CD s'imposent comme des piliers incontournables. Elles permettent d'orchestrer de manière fluide l'intégration, le déploiement, la supervision et la collaboration entre développeurs et administrateurs système.[1]

C'est dans cette dynamique que s'inscrit notre projet de fin d'études, réalisé dans le cadre de la **licence en Génie Logiciel et Systèmes d'Information**. Il consiste à concevoir une **plateforme d'automatisation centralisée à destination d'une équipe technique complète**, regroupant plusieurs rôles (administrateur, développeur, DevOps, superviseur), chacun ayant accès à une interface personnalisée. La plateforme permet d'agrégier et de superviser les données issues de divers outils DevOps (Jenkins, SonarQube, Nexus, DockerHub, GitHub, Grafana, Prometheus) tout en assurant la sécurité, la fluidité et la lisibilité de l'ensemble.

Le rapport est structuré pour suivre de manière logique et progressive le déroulement du projet.

Nous commençons par présenter les **fondamentaux du DevOps** et les enjeux de l'automatisation des livraisons logicielles, en introduisant les principaux outils qui seront utilisés dans la suite du travail.

Nous détaillons ensuite la **méthodologie adoptée**, basée sur une approche agile découpée en sprints successifs. Nous y expliquons les technologies choisies, ainsi que l'organisation du développement entre les différents rôles de l'équipe.

La partie technique débute par l'**installation manuelle des outils DevOps** (Jenkins, SonarQube, Nexus, Kubernetes, etc.) sur **trois machines virtuelles Ubuntu**, en simulant une architecture réelle. Nous avons ensuite mis en place un **pipeline Jenkins de test**, déclenché via GitHub, afin de valider la communication entre les services et de mieux comprendre les flux attendus dans un environnement DevOps.

À partir de là, nous avons développé le **backend en Node.js**, organisé selon une architecture modulaire (services, contrôleurs, routes, modèles...), sécurisé via JWT, et connecté à des API internes (MariaDB via Sequelize) et externes (GitHub, Jenkins, etc.).

Le **frontend**, bien que léger technologiquement (HTML, CSS, JavaScript Vanilla), a été pensé pour offrir une **interface moderne, responsive, avec dark mode**, et intégrant plusieurs tableaux de bord interactifs adaptés à chaque rôle.

Le cœur du rapport est dédié aux **sprints de développement**, chacun présentant les interfaces construites (login, dashboard, paramètres, etc.), les interactions mises en place avec

les services DevOps, ainsi que les choix techniques spécifiques à chaque fonctionnalité. Des captures d'écran viennent illustrer chaque évolution de l'application.

Ce projet, mené en **seulement quatre mois**, combine plusieurs chantiers techniques en un seul : **infrastructure DevOps, développement backend, frontend, sécurité, authentification, rôle-based access**, tout en restant aligné avec les enjeux du monde professionnel. Un défi ambitieux pour une licence, qui nous a permis d'approfondir des domaines à la fois variés et complémentaires.

Chapitre 1 : Contexte et cadrage du projet

Introduction

Ce chapitre introduit le cadre général du projet en présentant l'entreprise, son environnement et les motivations qui justifient l'initiative. L'objectif est de comprendre le contexte dans lequel le projet s'inscrit, ainsi que les enjeux qu'il vise à adresser.

I. Cadre du projet

Le projet, intitulé "**Mise en place d'une plateforme d'automatisation des applications web multi-services via l'approche CI/CD**", s'inscrit dans le cadre de notre projet de fin d'études en vue de l'obtention d'une **Licence en Génie Logiciel et Systèmes d'Information**. Ce stage est réalisé au sein de l'entreprise **Standard Sharing Software (3S)**.

II. Présentation de l'organisme d'accueil

Depuis sa création en 1988, 3S s'est imposée comme un acteur incontournable des solutions technologiques en Afrique du Nord. Forte de plus de 36 ans d'expertise, l'entreprise s'est bâtie une réputation solide en tant qu'intégrateur de premier plan, accompagnant plus de 5 000 clients, dont 93 % des 500 plus grandes entreprises tunisiennes.

Portée par un engagement constant envers l'innovation, 3S propose un large éventail de services couvrant les réseaux, la cybersécurité, le cloud, l'IoT, la vidéosurveillance et les solutions collaboratives. Grâce à des partenariats stratégiques avec des leaders technologiques tels que Cisco, Dell, Microsoft et Huawei, l'entreprise fournit des solutions de pointe adaptées aux enjeux numériques de ses clients. [2]



Figure 1 : Logo de 3s

Avec un capital de plus de 70 millions de dinars et un chiffre d'affaires dépassant les 150 millions de dinars, 3S ne cesse de se développer et d'investir au-delà du secteur IT. Sa diversification dans les énergies renouvelables, l'industrie bioplastique et la construction navale reflète sa vision stratégique et sa capacité à anticiper les évolutions du marché.

En tant que stagiaire chez 3S, nous aurons l'opportunité de travailler au sein d'une équipe dynamique et hautement qualifiée, composée de plus de 430 collaborateurs, dont plus de 120 ingénieurs. Nous serons immergés dans un environnement stimulant, où l'innovation et l'excellence opérationnelle sont au cœur de nos préoccupations. Nous contribuerons à des projets technologiques de grande envergure, tout en développant nos compétences et en participant à l'essor continu de l'entreprise.

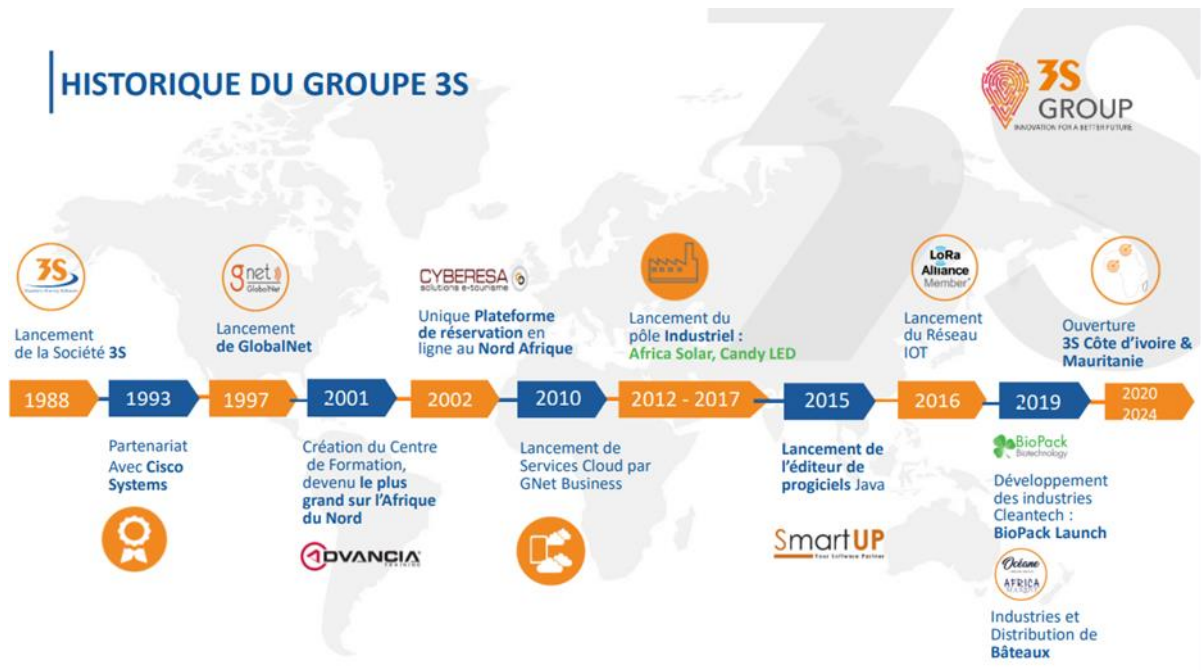


Figure 2 : Historique du groupe 3s [2]

1) Les services fournis par l'organisme d'accueil

Dans le cadre de notre stage, nous avons intégré le service **Développement et Édition de Progiciels** de l'entreprise 3S, un département clé dans la transformation numérique de ses clients. Ce service s'inscrit dans une offre globale couvrant plusieurs domaines stratégiques, décrits ci-dessous :

A) Développement et Édition de Progiciels

3S développe des solutions logicielles sur mesure, notamment des ERP, CRM et applications cloud, permettant aux entreprises d'automatiser et d'optimiser la gestion de leurs activités grâce à des outils performants et évolutifs

B) Intégration IT et Sécurité

3S accompagne les entreprises dans la mise en place et la sécurisation de leurs infrastructures informatiques. Grâce à des solutions avancées en réseautique, cybersécurité et virtualisation, elle assure une connectivité fiable et une protection optimale contre les cybermenaces.

C) Services Internet et Sécurité

L'entreprise propose des solutions d'hébergement cloud, de protection des accès web et mail, ainsi que des outils de supervision pour garantir la disponibilité et la sécurité des services en ligne de ses clients.

D) Solutions Collaboratives et Télécommunications

Pour optimiser la communication d'entreprise, 3S intègre des systèmes de téléphonie IP, de visioconférence et de collaboration adaptés aux besoins des organisations modernes, permettant ainsi une meilleure productivité et une connectivité fluide.

E) Vidéosurveillance et Systèmes de Sécurité

L'entreprise propose des solutions de vidéosurveillance IP, de contrôle d'accès et d'affichage dynamique pour renforcer la sécurité des infrastructures et améliorer la gestion des flux de personnes et d'informations..

F) Formation et Consulting IT

L'entreprise offre des formations certifiantes et du conseil stratégique en IT, aidant les entreprises à renforcer les compétences de leurs équipes et à adopter les meilleures pratiques pour une transformation numérique réussie.

G) Autres Activités Diversifiées

En plus de l'IT, 3S investit dans des secteurs innovants tels que les énergies renouvelables (centrales photovoltaïques), l'éclairage LED, les bioplastiques et la construction navale, diversifiant ainsi son expertise et son impact industriel.

Ces services font de 3S un partenaire technologique de référence, offrant des solutions complètes adaptées aux besoins des entreprises.

2) Les clients de 3S

3S développe des solutions pour plusieurs acteurs majeurs, issus de secteurs variés. Dans le domaine des télécommunications, l'entreprise collabore avec des opérateurs tels qu'Ooredoo, Tunisie Telecom et Topnet.

Elle accompagne également des établissements bancaires et financiers comme la BIAT ou la BH. Le secteur de la grande distribution est aussi représenté, avec des enseignes telles que Carrefour, Monoprix et MG.

Enfin, 3S travaille avec le secteur public, notamment avec La Poste et la SONEDE.



Figure 3 : List des clients de 3S

3) Présentation du domaine métier

Le projet s'inscrit dans le domaine du DevOps et de l'automatisation des processus CI/CD. L'objectif est d'optimiser le déploiement des applications web multi-services en réduisant les délais et les erreurs humaines.

A) Enjeux et Tendances

L'adoption du CI/CD est une tendance majeure dans l'industrie IT, visant à accélérer le cycle de développement tout en améliorant la qualité logicielle. Les enjeux principaux incluent :

- Réduction du time-to-market
- Automatisation des tests et intégration continue
- Scalabilité et gestion efficace des ressources
- Sécurité et conformité dans les environnements DevOps.

B) Acteurs Clés

Dans le cadre de l'automatisation des processus CI/CD, plusieurs types d'acteurs interviennent, chacun ayant un rôle déterminant dans la réussite des projets :

- **Équipes de développement (Développeurs)** : Ce sont les principaux utilisateurs des pipelines¹ CI/CD. Ils conçoivent, développent et testent les applications. Leur collaboration avec les équipes DevOps permet d'assurer une intégration fluide du code dans l'environnement de production.
- **Équipes DevOps** : Responsables de la mise en place, du suivi et de l'optimisation des pipelines CI/CD. Elles assurent la liaison entre le développement et l'exploitation en automatisant les processus de build, test et déploiement.

¹ **Pipeline** : Suite d'étapes automatisées pour tester, construire et déployer une application.

- **Administrateurs systèmes / Ops** : Ils interviennent sur la gestion des infrastructures, qu'elles soient physiques ou cloud, en garantissant la disponibilité, la scalabilité et la sécurité des environnements d'exécution.
- **Responsables sécurité (SecOps)** : Intègrent les bonnes pratiques de sécurité dès les premières phases du cycle de vie des applications. Leur rôle est essentiel dans le contexte DevSecOps pour assurer la conformité et prévenir les vulnérabilités.
- **Chefs de projet / Product Owners** : Ils supervisent l'avancement des développements et veillent à ce que les livrables répondent aux besoins métier. Ils jouent un rôle clé dans la priorisation des tâches et la validation des versions déployées.
- **Outils et plateformes CI/CD** : Bien qu'ils ne soient pas des acteurs humains, les outils comme Jenkins, GitLab CI/CD, GitHub Actions, Docker, Kubernetes, ou encore Ansible jouent un rôle central en orchestrant les différentes étapes de l'automatisation.

III. Analyse de l'existant

Avant de détailler notre solution, il est crucial de comprendre en profondeur le fonctionnement actuel du cycle DevOps au sein de 3S. Cette analyse de l'existant permettra de mettre en lumière à la fois les points forts sur lesquels s'appuyer et les failles majeures qu'il convient de corriger. En examinant chaque étape — de la compilation à la supervision, en passant par les tests, le déploiement et la gestion des accès — nous identifions les zones d'ombre et les goulots d'étranglement. Ce diagnostic exhaustif, appuyé par des retours d'expérience des équipes et des données de production, sert de socle à la conception d'une plateforme unifiée, capable de transformer un processus actuellement morcelé en un continuum fluide, sécurisé et automatisé.

1) Présentation de l'existant

Dans l'état actuel, le cycle DevOps chez 3S comprend cinq phases clés, chacune opérant de manière autonome et sans coordination globale :

Compilation (build) :

- **Processus manuel** : chaque projet est compilé via des configurations spécifiques, gérées à la main.
- **Logs dispersés** : les fichiers de sortie et d'erreur sont répartis sur plusieurs serveurs, sans espace centralisé pour leur consultation.

- Visibilité limitée : il est difficile de suivre l'historique des builds ou de comparer les performances entre projets.

Tests automatisés :

- Lancements indépendants : les suites de tests (unitaires, d'intégration, de performance) sont déclenchées individuellement pour chaque application.
- Résultats isolés : les rapports sont stockés localement, sans consolidation ni accès rapide à un tableau de bord unique.
- Absence de suivi global : aucune synthèse n'existe pour mesurer la tendance de la qualité du code dans le temps.

Déploiement :

- Scripts personnalisés : chaque release exploite des scripts shell développés en interne, sans standardisation.
- Pas de versioning unifié : les mises à jour sont tracées projet par projet, compliquant la gestion des versions.
- Rollback manuel : en cas d'erreur, l'équipe doit exécuter manuellement des commandes pour revenir à une version antérieure.

Supervision :

- Dashboards multiples : les métriques (CPU, mémoire, temps de réponse, taux d'erreur) sont affichées sur différentes interfaces.
- Navigation fastidieuse : pour consulter l'état général du système, il faut jongler entre plusieurs consoles.
- Alertes peu fiables : les notifications d'incident sont dispersées et parfois manquantes, retardant la réaction.

Gestion des accès et configurations :

- Administration manuelle : création et suppression des comptes, attribution des droits et des rôles gérées séparément pour chaque environnement.
- Réplication des paramètres : l'ajout d'un nouveau projet nécessite de répliquer manuellement toutes les configurations existantes.
- Risques de sécurité : doublons de comptes et permissions obsolètes peuvent persister, ouvrant des failles.

Même si chacune de ces étapes (build, tests, déploiement, supervision, gestion des accès) fonctionne relativement bien quand on les regarde séparément, leur absence de lien entre elles crée de sérieux problèmes pour l'ensemble du processus. En l'absence d'un point de coordination unique, les équipes ne voient pas l'ensemble des informations au même endroit. Elles doivent passer d'une console à une autre, ce qui rend la prise de décision lente et laborieuse.

Les incidents mettent plus de temps à être détectés et résolus, car il n'existe pas de flux de travail fluide qui relie automatiquement une étape à la suivante. De plus, la répétition manuelle de certaines tâches — création de comptes, paramétrage, mises à jour — alourdit le travail quotidien et augmente le risque d'erreurs. En résumé, le manque de cohérence entre les différentes phases entraîne une perte de temps, une augmentation de la charge de travail et une baisse de l'efficacité générale de la démarche DevOps.

2) Critique de l'existant

L'analyse des différentes phases met en évidence quatre défis majeurs qui freinent la performance et l'efficacité des équipes DevOps :

Visibilité éclatée :

- Absence de tableau de bord global : Les informations critiques (état des builds, résultats des tests, métriques de performance) sont dispersées dans plusieurs interfaces, obligeant les utilisateurs à naviguer entre elles.
- Perte de temps et erreurs de diagnostic : Sans vue synthétique, les anomalies passent souvent inaperçues jusqu'à la phase de production, entraînant des corrections tardives et coûteuses.

Processus redondants et lourdeurs opérationnelles :

- Multiplication des tâches manuelles : Configuration, déploiement et gestion des accès doivent être reproduits pour chaque projet et chaque outil.
- Complexité croissante : Les scripts et procédures évoluent souvent de façon anarchique, augmentant la charge cognitive et le risque d'erreurs humaines.

Sécurité et gouvernance fragilisées :

- Gestion des droits décentralisée : L'absence d'une politique RBAC unifiée génère des permissions mal alignées, des comptes obsolètes et des failles potentielles.
- Traçabilité limitée : Sans journalisation centralisée, il est difficile de retracer les actions et de vérifier la conformité aux normes de sécurité.

Agilité compromise :

- Time-to-market allongé : La duplication des configurations et des étapes ralentit la mise en place de nouveaux projets ou fonctionnalités.
- Difficulté d'évolution : Les modifications majeures deviennent risquées, car chaque
- Changement doit être testé et déployé manuellement, phase par phase.

Ces freins combinés aboutissent à :

- Perte d'efficacité : Les équipes passent plus de temps sur des tâches répétitives que sur l'innovation.
- Augmentation des coûts : Les retards et les incidents génèrent des surcoûts de support et de maintenance.
- Insatisfaction des parties prenantes : Clients, développeurs et opérationnels souffrent d'un manque de réactivité et de transparence.

3) Solution proposée

Pour corriger ces manquements, nous proposons une plateforme DevOps centralisée qui offre :

1. Tableau de bord unifié Un cockpit global regroupe compilation, tests, déploiement et supervision en une seule interface, offrant une vue synthétique et en temps réel.
2. Pipelines as Code automatisés Des templates paramétrables permettent de générer, versionner et rollbacker les workflows CI/CD sans toucher au script shell.
3. Supervision intelligente et centralisée Collecte consolidée des métriques et logs, alertes configurables et reporting automatisé, pour anticiper les incidents.
4. Gestion centralisée des droits RBAC² fédéré et Single Sign-On (SSO)³ pour tous les services, avec journalisation complète des actions.
5. Portail self-service Interface intuitive pour les développeurs et ops, facilitant la création et la gestion des projets, réduisant la dépendance aux équipes support.

Cette plateforme constitue le pilier d'une stratégie DevOps performante, permettant de passer d'un modèle fragmenté à un écosystème cohérent, agile et sécurisé. En centralisant les

² **RBAC (Role-Based Access Control)** : absence de contrôle d'accès basé sur les rôles centralisé. RBAC définit des rôles attribuant des permissions précises, simplifiant la gestion des droits.

³ **SSO (Single Sign-On)** : authentification unique pour tous les services.

opérations, elle réduira les coûts, améliorera la qualité des livraisons et stimulera l'innovation au sein de 3S.

Bénéfices attendus :

- Réduction de 30 % du temps consacré aux tâches manuelles.
- Diminution de 40 % des incidents en production.
- Meilleure sécurité et traçabilité.
- Collaboration optimisée entre équipes.

IV. Méthodologie de travail :

Ce projet s'inscrit dans le cadre d'un stage d'une durée de **quatre mois**, réalisé en **binôme**. Afin de répondre efficacement à la problématique posée, nous avons accordé une attention particulière au **choix de la méthodologie de travail**. Celle-ci a été sélectionnée non seulement pour sa pertinence par rapport au contexte du projet, mais aussi pour sa capacité à **structurer notre démarche et à valoriser nos avancées** de manière claire et progressive.

Dans cette section, nous présentons la méthodologie de gestion de projet adoptée tout au long de notre travail. Une telle méthodologie regroupe un ensemble de pratiques et d'outils permettant de :

- Définir les tâches à accomplir.
- Planifier leur réalisation dans le temps.
- Identifier et mobiliser les ressources nécessaires.

Il existe plusieurs approches en gestion de projet, chacune étant plus ou moins adaptée selon les besoins, les contraintes et les objectifs du projet.

Il existe plusieurs méthodologies de gestion de projet, chacune adaptée à un contexte spécifique. Parmi les plus utilisées, on retrouve :

Modèle en V : Une approche traditionnelle dérivée du modèle en cascade, mettant l'accent sur une validation stricte après chaque phase de développement. Elle est souvent utilisée dans les projets où la qualité et la traçabilité sont essentielles, comme dans l'aéronautique ou le médical.

2TUP (Two-Track Unified Process) : Une méthode hybride qui combine les avantages des approches traditionnelles et agiles en séparant les cycles de développement et de gestion des exigences. Elle est particulièrement utile dans les projets où les besoins évoluent rapidement.

Méthodes Agiles (Scrum, Kanban, SAFe, etc.) : Des approches flexibles favorisant l'adaptabilité, la collaboration et la livraison continue.

Dans ce projet, nous nous concentrerons principalement sur la comparaison entre la méthode en cascade et Scrum, détaillée dans le tableau ci-dessous

Critères	Méthode Cascade [3]	Scrum [4]
Approche	Séquentielle, linéaire	Itérative, incrémentale
Planification	Planification détaillée en amont, toutes les exigences définies au début	Planification adaptative, le back log de produit évolue
Phases	Phases distinctes et séquentielles (analyse, conception, implémentation, tests, déploiement)	Sprints ⁴ courts et itératifs, avec des revues et des rétrospectives
Flexibilité	Faible, difficile de changer les exigences une fois le projet lancé	Élevée, les changements peuvent être intégrés dans les sprints suivants
Implication du client	Limitée, principalement au début et à la fin du projet	Forte, le Product Owner est activement impliqué tout au long du projet
Livrables	Un seul livrable final à la fin du projet	Livrables fonctionnels à la fin de chaque sprint
Gestion des risques	Risques élevés si les exigences initiales sont incorrectes ou changent	Risques réduits grâce aux itérations courtes et aux retours fréquents
Documentation	Documentation complète et détaillée à chaque phase	Documentation minimale, axée sur les besoins immédiats
Adaptation aux changements	Difficile, coûteux.	Facile, et fait partie du processus.
Retour d'information	Tardif, à la fin du projet.	Rapide et fréquent, à la fin de chaque sprint.
Idéal pour	Projets avec des exigences stables et bien définies	Projets complexes, évolutifs, avec des exigences changeantes

Tableau 1 : comparaison des méthodologies Cascade et Scrum

Scrum et le modèle en cascade sont deux approches fondamentalement différentes en gestion de projet, chacune avec ses avantages et ses limites. Le modèle en cascade suit une structure linéaire et rigide où chaque phase – analyse, conception, développement, test et déploiement – doit être entièrement achevée avant de passer à la suivante. Cette méthode fonctionne bien

⁴ **Sprint** : Période de développement courte avec des objectifs précis.

lorsque les exigences sont clairement définies dès le départ et ne risquent pas de changer en cours de projet. Cependant, elle manque de flexibilité et peut entraîner des délais importants si des ajustements sont nécessaires après coup. À l'inverse, Scrum, qui repose sur une approche agile, est conçu pour offrir une grande réactivité aux changements et aux imprévus. Il divise le projet en sprints courts et itératifs, où l'équipe livre à chaque cycle des incréments fonctionnels du produit. Grâce à des réunions quotidiennes et à une collaboration étroite entre les parties prenantes, Scrum permet d'optimiser le travail en fonction des retours et d'améliorer continuellement le produit. Cette méthode favorise l'innovation, la transparence et l'implication active de l'équipe, garantissant ainsi une meilleure adaptabilité aux besoins des utilisateurs. Contrairement au modèle en cascade, qui impose un cadre figé dès le début, Scrum permet de maximiser la valeur ajoutée tout en minimisant les risques liés aux incertitudes du projet.

1) Adoption de Scrum dans ce projet

Dans ce projet, nous adopterons la méthodologie **Scrum**, une approche Agile qui nous permet d'itérer rapidement et d'ajuster notre travail en fonction des retours des parties prenantes. Scrum offre une approche adaptative et collaborative, idéale pour les projets complexes et évolutifs.

Le cycle de vie d'un projet Scrum repose sur les éléments suivants :

- **Backlog⁵ Produit** : Liste des exigences et améliorations priorisées par le Product Owner⁶.
- **Sprint Planning** : Planification des tâches à réaliser dans un sprint en fonction des priorités du backlog produit et de la capacité de l'équipe.
- **Sprint** : Période de 1 à 4 semaines pendant laquelle l'équipe se concentre sur le développement d'un ensemble défini de fonctionnalités.
- **Daily Scrum** : Réunion quotidienne rapide permettant à l'équipe d'échanger sur l'avancement, les obstacles rencontrés et les ajustements nécessaires.
- **Sprint Review** : Démonstration des fonctionnalités développées aux parties prenantes afin de recueillir leurs retours et ajuster la suite du développement.
- **Sprint Retrospective** : Moment dédié à l'analyse du sprint écoulé pour identifier ce qui a bien fonctionné et les axes d'amélioration à appliquer dans les prochains sprints.

⁵ **Backlog** : Liste priorisée des tâches à réaliser dans le projet.

⁶ **Product Owner (PO)** : Personne en charge de définir les besoins et priorités du produit.

Chaque sprint aboutit à un livrable fonctionnel, ce qui permet d'obtenir des retours précoces et d'ajuster les priorités en conséquence. Contrairement à une approche en cascade, Scrum favorise une forte interaction entre les équipes, une réactivité accrue aux changements et une livraison continue de valeur.

2) Rôles dans Scrum

Dans la méthodologie Scrum, plusieurs rôles clés assurent la réussite du projet :

Product Owner (PO) : Responsable du backlog produit, il définit les priorités et veille à ce que le produit réponde aux besoins des utilisateurs.

Scrum Master : Facilite le processus Scrum, élimine les obstacles et s'assure que l'équipe applique correctement les principes Agile.

Développeurs : Membres de l'équipe chargés de concevoir, développer et tester les fonctionnalités prévues.

Stakeholders (Parties Prenantes) : Clients, utilisateurs ou autres acteurs externes qui fournissent des retours pour orienter le développement.

Conclusion

Dans ce chapitre, nous avons d'abord introduit l'entreprise dans laquelle notre projet a été mené, en mettant en avant son environnement technique.

Par la suite, nous avons examiné l'existant à travers une analyse approfondie de la problématique, une critique des solutions en place et la présentation de notre approche proposée. Enfin, nous avons détaillé la méthodologie adoptée.

Le prochain chapitre sera consacré à l'état de l'art ainsi qu'à l'introduction des technologies utilisées.

Chapitre 2 : Revue de l'état de l'art

Introduction

Dans ce chapitre, nous explorerons les concepts fondamentaux de l'approche CI/CD et son rôle crucial dans l'automatisation des processus de développement et de déploiement des applications web multiservices. Nous commencerons par une présentation détaillée de la culture DevOps, de ses principes essentiels et des avantages qu'elle offre. Nous examinerons ensuite les outils et technologies clés qui permettent de mettre en place une infrastructure CI/CD performante et adaptée aux besoins des équipes de développement.

I. La culture DevOps

1) Définition et Objectifs

DevOps est une méthodologie de développement logiciel qui favorise une collaboration étroite et une intégration continue entre les équipes de développement (Dev) et les équipes des opérations (Ops). L'objectif principal de DevOps est d'améliorer l'efficacité des processus de développement et de déploiement des logiciels en réduisant les délais de livraison tout en améliorant la qualité du produit.[1]

L'adoption de DevOps permet d'intégrer de manière fluide des pratiques comme **l'intégration continue (CI)** et le **déploiement continu (CD)** dans le cycle de vie des applications, ce qui permet de livrer des fonctionnalités de manière rapide, fiable et efficace. Les organisations qui adoptent DevOps sont capables de répondre plus rapidement aux demandes du marché, aux besoins des clients et de s'adapter plus facilement aux changements technologiques. [5]

Les principaux objectifs de la culture DevOps incluent :

- **Accélérer le développement des applications** en automatisant les tests, le déploiement et la gestion des infrastructures.
- **Améliorer la qualité du code** en intégrant des tests automatisés à chaque étape du développement et en facilitant la détection rapide des erreurs.
- **Optimiser les coûts** en réduisant le temps passé sur des processus manuels et en augmentant la productivité des équipes.

2) Principes de DevOps

La culture DevOps repose sur plusieurs principes fondamentaux, qui sont conçus pour améliorer la collaboration entre les équipes et automatiser au maximum les processus afin de garantir un cycle de vie rapide, efficace et de haute qualité pour le logiciel.

A) CI/CD : L'intégration continue et le déploiement continu

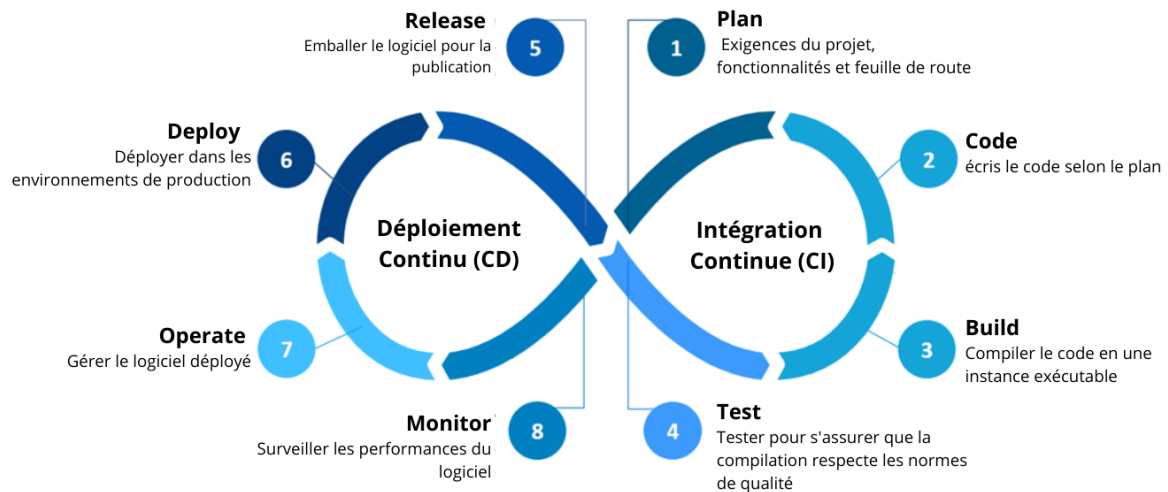


Figure 4: Image de CI/CD pipeline [6]

L'intégration continue (CI) et le déploiement continu (CD) sont au cœur de la culture DevOps.

CI : Les développeurs intègrent fréquemment leurs changements de code dans un dépôt centralisé. À chaque intégration, des tests automatisés sont effectués pour valider les modifications. Cela permet de détecter rapidement les erreurs d'intégration.

CD : Le déploiement continu permet de déployer automatiquement le code validé vers des environnements de test ou de production. Le but est de réduire le temps entre le moment où le code est écrit et celui où il est accessible aux utilisateurs.

B) Automatisation des tests

L'automatisation des tests est cruciale pour DevOps. Dans une chaîne CI/CD, les tests (unitaires, d'intégration, de régression) sont automatisés et s'exécutent à chaque modification du code. Cela permet de garantir qu'aucune fonctionnalité existante ne soit affectée par de nouvelles modifications. En outre, les tests automatisés réduisent le besoin d'interventions manuelles, ce qui permet de se concentrer sur des tâches plus stratégiques avec plus de valeur ajoutée.

C) Collaboration entre les équipes

La culture DevOps vise à éliminer les silos traditionnels entre les équipes de développement et des opérations. Dans un environnement DevOps, les équipes Dev et Ops travaillent ensemble tout au long du cycle de vie du logiciel. Cela facilite la communication, la compréhension mutuelle des objectifs et une réponse plus rapide aux problèmes. La collaboration permet également une meilleure planification et une gestion plus fluide des changements.

D) Retour d'information rapide

DevOps encourage un retour d'information rapide et continu à toutes les étapes du développement. Cela inclut les tests automatisés, la surveillance des applications en production, et la gestion des incidents. Le but est de détecter et corriger les problèmes dès qu'ils apparaissent, avant qu'ils n'affectent les utilisateurs finaux.

E) Avantages de la culture DevOps

L'adoption de DevOps offre des bénéfices considérables pour les équipes de développement, les opérations et l'entreprise dans son ensemble. Les principaux avantages incluent l'amélioration de la qualité du code, la réduction des délais de déploiement, la collaboration renforcée entre les équipes, la réduction des risques, et une meilleure satisfaction client.

3) Amélioration de la qualité du code

DevOps permet une amélioration continue de la qualité du code en intégrant des tests automatisés à chaque étape du cycle de développement. Les avantages incluent :

Détection précoce des erreurs : L'intégration continue avec des tests automatisés permet de détecter rapidement les erreurs et les régressions, avant qu'elles n'atteignent l'environnement de production.

Assurance qualité : Grâce à l'automatisation des tests (tests unitaires, tests d'intégration, tests de performance), la qualité du code est garantie à chaque itération. Cela permet également d'avoir un code plus robuste et résilient.

Les tests automatisés permettent également d'assurer que les nouvelles fonctionnalités n'entravent pas les fonctionnalités existantes, réduisant ainsi les risques d'incidents en production.

A) Réduction des délais de déploiement

L'un des avantages les plus significatifs de DevOps est la **réduction des délais de déploiement**. En automatisant les processus de validation, de tests et de déploiement, DevOps permet des mises à jour plus rapides et plus fréquentes du logiciel.

Livraison rapide : Les équipes peuvent déployer des modifications de manière plus régulière et plus rapide grâce aux pipelines CI/CD automatisés.

Petites mises à jour fréquentes : L'approche DevOps privilégie des livraisons fréquentes et incrémentales, au lieu de déploiements majeurs espacés dans le temps. Cela réduit le risque de bugs en production et améliore l'agilité.

B) Collaboration améliorée entre les équipes

L'un des principes fondamentaux de DevOps est de promouvoir une **collaboration étroite** entre les équipes de développement et des opérations. Ce principe est essentiel pour faciliter les échanges d'informations, réduire les malentendus et garantir une meilleure coordination tout au long du cycle de vie du logiciel.

Communication renforcée : Grâce à des outils collaboratifs et des processus partagés, les équipes Dev et Ops peuvent collaborer en temps réel, ce qui permet de résoudre les problèmes plus rapidement et d'améliorer la prise de décision.

Réduction des conflits : Les équipes travaillant en étroite collaboration peuvent mieux comprendre les priorités des autres, réduisant ainsi les conflits et favorisant la cohésion des équipes.

C) Amélioration de la satisfaction client

DevOps améliore l'expérience utilisateur en permettant une **réactivité accrue** et une **qualité élevée des produits**. Voici les principaux leviers :

Livraison fréquente de nouvelles fonctionnalités : Les utilisateurs bénéficient d'une mise à jour constante des fonctionnalités, de la correction des bugs et de l'amélioration de l'expérience. Cela permet à l'entreprise de répondre rapidement aux besoins des clients.

Moins de temps d'indisponibilité : En automatisant les tests et le déploiement, le nombre d'incidents en production est réduit, ce qui améliore la stabilité des applications et limite les interruptions de service.

Réponse rapide aux retours utilisateurs : DevOps permet aux équipes de traiter rapidement les retours clients, d'adapter les fonctionnalités selon les besoins et d'optimiser en continu l'expérience utilisateur.

D) Réduction des risques et des coûts

La mise en place de DevOps permet non seulement de réduire les risques associés aux déploiements, mais également d'optimiser les coûts liés au cycle de développement.

Moins de risques liés aux déploiements : L'automatisation des tests et des déploiements permet d'effectuer des mises en production plus petites et plus fréquentes, réduisant ainsi les risques liés à des changements importants.

Réduction des coûts à long terme : Bien que l'implémentation de DevOps puisse demander un investissement initial pour la formation et l'infrastructure, les économies réalisées à long terme sont substantielles. Ces économies proviennent de la réduction des erreurs, de l'amélioration de la productivité, et d'une gestion plus efficace des environnements.

E) Meilleure gestion des infrastructures

L'adoption de DevOps transforme également la gestion des infrastructures en rendant cette gestion **plus automatisée et plus cohérente**.

Infrastructure en tant que code (IaC⁷) : DevOps permet de gérer l'infrastructure de manière programmée et automatisée via des outils comme Terraform⁸, ce qui réduit les erreurs humaines et garantit des déploiements plus cohérents.

Scalabilité : DevOps, couplé à des outils comme Kubernetes, permet de gérer l'auto-scaling et l'orchestration des conteneurs, ce qui assure une gestion optimisée des ressources et une capacité d'adaptation rapide aux besoins des utilisateurs.

4) Conclusion sur les Avantages de DevOps

L'adoption de la culture DevOps, intégrant les pratiques d'intégration et de déploiement continus, permet de transformer de manière significative les processus de développement logiciel. Cette méthodologie permet d'améliorer la qualité du code, de réduire les délais de déploiement, d'encourager la collaboration entre les équipes et de répondre rapidement aux besoins du marché. Dans le contexte des applications web multi-services, où les exigences de scalabilité et de gestion des services sont cruciales, DevOps s'avère essentiel pour assurer une gestion fluide et efficace des applications.

II. Choix des outils pour ce projet

Dans le cadre du projet d'automatisation d'applications web multiservices via une approche CI/CD, le choix des outils était crucial pour garantir l'efficacité du processus de développement, de test, de déploiement et de mise en production. Ce projet, réalisé dans le cadre du stage **3S (Standard Sharing Software)**, avait pour objectif d'automatiser le cycle de vie des

⁷ **IaC (Infrastructure as Code)** : Pratique qui consiste à gérer l'infrastructure via du code.

⁸ **Terraform** : Outil d'Infrastructure as Code pour provisionner des ressources automatiquement.

applications en intégrant des technologies modernes, tout en optimisant les flux de travail avec une gestion agile des processus.

Les outils choisis ont été sélectionnés en fonction de leur capacité à répondre aux exigences du projet, qui incluent l'utilisation des technologies suivantes :

- **Développement web avec Node.JS, HTML et CSS** pour le développement du plateforme front-end et back-end.
- **Conteneurisation avec Docker** pour isoler les différents services.
- **Orchestration de conteneurs avec Kubernetes** pour gérer les déploiements à grande échelle.
- **Environnements de test automatisés** avec l'intégration de tests unitaires et fonctionnels dans les pipelines CI/CD.
- **Systèmes d'exploitation Ubuntu** pour les machines virtuelles utilisées pour l'hébergement de l'environnement de développement.
- Outils de gestion de la qualité du code avec SonarQube et gestion des artefacts avec Nexus pour garantir la qualité et la gestion des versions.

1) Jenkins pour la gestion des pipelines CI/CD



Figure 5: logo de Jenkins

Jenkins est une solution populaire et puissante pour la gestion des pipelines CI/CD. Elle est largement utilisée dans les entreprises et les projets de grande envergure grâce à sa flexibilité et à son extensibilité. Jenkins permet d'automatiser les processus de construction, de test et de déploiement des applications, tout en offrant un contrôle granulaire sur chaque étape du pipeline.[6]

Avantages de Jenkins :

- **Extensibilité via des plugins :** Jenkins dispose d'une large bibliothèque de plugins qui permet d'ajouter des fonctionnalités spécifiques selon les besoins, telles que l'intégration avec différents systèmes de contrôle de version, outils de tests, ou plateformes de déploiement.

- **Configuration flexible** : La configuration des pipelines dans Jenkins peut être réalisée via des fichiers *Jenkinsfile* ou via son interface graphique. Cela permet une grande flexibilité dans la définition des étapes du pipeline (construction, tests, déploiement, etc.).
- **Large support de technologies** : Jenkins supporte une multitude d'outils modernes tels que Docker, Kubernetes, Maven, Gradle, et bien d'autres, ce qui permet une intégration facile avec des technologies variées utilisées dans des projets complexes.
- **Pipeline as Code** : Grâce aux *Jenkinsfiles*, les pipelines sont définis comme du code, ce qui permet de versionner, tester et réutiliser les configurations de manière contrôlée.
- **Communauté active et documentation riche** : Jenkins bénéficie d'une grande communauté d'utilisateurs et de développeurs, offrant des ressources abondantes et une large base de connaissances pour résoudre des problèmes spécifiques.

2) SonarQube pour l'analyse de la qualité du code



Figure 6: logo de SonarQube

SonarQube est un outil d'analyse de la qualité du code qui permet de détecter les erreurs, les vulnérabilités et les mauvaises pratiques dans le code source. Il est utilisé dans ce projet pour s'assurer que le code produit respecte des critères de qualité stricts avant de passer à la phase de production. [7]

Avantages de SonarQube :

- **Analyse statique du code** : SonarQube analyse le code source pour détecter les problèmes de performance, les erreurs de syntaxe, les vulnérabilités de sécurité et les duplications. Cela permet d'améliorer la maintenabilité et la sécurité du code.
- **Rapports détaillés et visualisation** : L'outil fournit des rapports détaillés et des graphiques pour visualiser les problèmes détectés, ce qui aide les développeurs à se concentrer sur les problèmes les plus critiques.
- **Compatibilité avec les langages de programmation courants** : SonarQube est compatible avec une large gamme de langages, y compris PHP, ce qui est essentiel dans ce projet de développement web.

3) Nexus pour la gestion des artefacts



Figure 7: logo de SonaType Nexus Repository

Nexus est un gestionnaire d'artefacts qui permet de stocker et de gérer les dépendances et les bibliothèques nécessaires au projet. Il facilite l'intégration de bibliothèques tierces dans les applications et permet de versionner les artefacts produits durant le processus de développement (par exemple, les images Docker et les bibliothèques PHP). [8]

Avantages de Nexus :

- **Gestion centralisée des artefacts** : Nexus permet de centraliser tous les artefacts (par exemple, les fichiers JAR, WAR ou Docker images), ce qui simplifie la gestion des versions et des dépendances.
- **Sécurisation des artefacts** : Nexus permet de contrôler l'accès aux artefacts et d'assurer que seuls les artefacts validés et testés sont utilisés dans les environnements de développement et de production.
- **Compatibilité avec Docker** : Nexus est particulièrement adapté pour gérer des artefacts Docker, ce qui est essentiel pour la conteneurisation des services dans le cadre de ce projet.

4) Ubuntu pour les machines virtuelles



Figure 8: logo de Ubuntu

Le système d'exploitation **Ubuntu** a été choisi pour héberger les machines virtuelles utilisées dans ce projet. Ubuntu est un système d'exploitation stable, sécurisé et largement utilisé dans les environnements de production. [9]

Avantages d'Ubuntu :

- **Stabilité et sécurité** : Ubuntu est reconnu pour sa robustesse et sa sécurité. Cela en fait un excellent choix pour les serveurs de développement et de production.

- **Compatibilité avec les outils DevOps** : Ubuntu est largement compatible avec les outils de développement et d'automatisation comme Docker, Jenkins, GitLab CI, et Kubernetes, ce qui facilite l'intégration de tous les outils du projet.
- **Support communautaire** : Ubuntu bénéficie d'une large communauté de développeurs et d'une documentation riche, ce qui facilite la résolution des problèmes techniques.

5) MobaXterm pour la connexion SSH

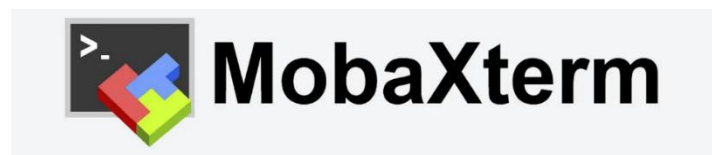


Figure 9 : logo de MobaXterm

MobaXterm est un logiciel qui permet de se connecter à distance à des serveurs via SSH. Dans ce projet, il a été utilisé pour administrer les serveurs Ubuntu et pour effectuer des déploiements à distance. [10]

Avantages de MobaXterm :

- **Connexion sécurisée via SSH** : MobaXterm permet de se connecter facilement aux serveurs distants en toute sécurité, ce qui est essentiel pour gérer les machines virtuelles Ubuntu hébergeant l'environnement de développement.
- **Interface graphique intuitive** : Il offre une interface graphique conviviale pour gérer les connexions SSH et effectuer des actions administratives à distance, facilitant ainsi la gestion des serveurs.
- **Transfert de fichiers** : MobaXterm permet également de transférer des fichiers entre les serveurs locaux et distants, ce qui est très utile pour les opérations de déploiement.

6) Kubernetes pour l'orchestration des conteneurs



Figure 10 : logo de Kubernetes

Kubernetes est un système open-source d'orchestration⁹ de conteneurs qui automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées. Dans le cadre de ce projet, Kubernetes joue un rôle central dans le déploiement des services applicatifs,

⁹ **Orchestration** : Coordination automatique du déploiement et du fonctionnement des services.

permettant une gestion efficace et scalable des conteneurs Docker générés par les pipelines CI/CD.[11]

Avantages de Kubernetes :

- **Orchestration puissante des conteneurs** : Kubernetes permet de gérer automatiquement le cycle de vie des conteneurs, y compris leur déploiement, leur redémarrage, leur mise à l'échelle et leur suppression.
- **Scalabilité dynamique** : Grâce à son système d'autoscaling, Kubernetes adapte automatiquement les ressources selon la charge, garantissant ainsi des performances optimales.
- **Haute disponibilité** : En cas de défaillance d'un nœud, Kubernetes redirige les pods vers d'autres nœuds disponibles pour maintenir la continuité de service.
- **Intégration avec Jenkins** : Les pipelines Jenkins peuvent interagir avec Kubernetes pour déclencher des déploiements automatisés à chaque mise à jour validée.
- **Séparation des environnements** : Chaque service peut être isolé dans un espace de noms (namespace¹⁰), facilitant la gestion des projets multi-services.

7) Docker Hub pour la gestion des images conteneurisées



Figure 11 : logo de Docker Hub

Docker Hub est un registre public d'images Docker permettant de stocker, partager et distribuer des images conteneurisées. Dans ce projet, Docker Hub est utilisé comme source centrale pour récupérer les images officielles (comme PHP, SonarQube, Jenkins, etc.) et comme dépôt pour publier des images personnalisées créées au cours du pipeline CI/CD.[12]

Avantages de Docker Hub :

- **Accès aux images officielles** : Il fournit un large éventail d'images officielles, maintenues par la communauté ou les éditeurs eux-mêmes, garantissant leur fiabilité.
- **Partage facilité** : Les images Docker créées dans Jenkins peuvent être poussées vers Docker Hub, ce qui facilite leur partage entre environnements et collaborateurs.

¹⁰ **Namespace** : Espace logique d'isolation dans Kubernetes pour séparer les services.

- **Intégration avec Jenkins** : Docker Hub s'intègre naturellement dans les pipelines Jenkins, permettant l'automatisation des push et pull¹¹ d'images.
- **Versionnement** : Chaque image peut être versionnée, ce qui permet de suivre précisément les évolutions d'un service déployé.
- **Visibilité et collaboration** : Il permet de gérer les dépôts publics ou privés, selon le besoin de partage ou de confidentialité des projets.

8) Grafana pour la visualisation des métriques



Figure 12 : logo de Grafana

Grafana est une plateforme open-source de visualisation et d'analyse de métriques. Elle permet de créer des tableaux de bord interactifs à partir de données collectées par des outils de monitoring comme Prometheus. Dans ce projet, Grafana est utilisé pour surveiller en temps réel l'état des déploiements et des performances des services déployés sur Kubernetes.[13]

Avantages de Grafana :

- **Visualisation en temps réel** : Grafana permet de créer des dashboards dynamiques et personnalisables pour suivre l'état des ressources (CPU, mémoire, pods, erreurs, etc.).
- **Intégration avec Prometheus** : Il s'intègre parfaitement avec Prometheus, utilisé comme collecteur de métriques, pour fournir une vue d'ensemble complète de l'infrastructure.
- **Alertes configurables** : Il permet de définir des seuils d'alerte et de recevoir des notifications (mail, Slack, etc.) en cas d'anomalies.
- **Multi-sources** : Grafana peut agréger des données provenant de plusieurs sources (Prometheus, InfluxDB, Loki, Elasticsearch, etc.).
- **Interface intuitive** : Son interface ergonomique rend l'analyse des données accessible même aux non-experts.

¹¹ **Push / Pull** : Envoyer (push) ou récupérer (pull) du code depuis un dépôt distant.

9) Prometheus pour la collecte des métriques



Figure 13 : logo de Prometheus

Prometheus est un système open-source de surveillance et de collecte de métriques conçu pour les environnements cloud-natifs. Dans ce projet, il est utilisé pour monitorer l'état de l'infrastructure déployée, en particulier les services tournant sur Kubernetes, et pour fournir les données nécessaires à Grafana.[14]

Avantages de Prometheus :

Collecte efficace des métriques : Prometheus interroge régulièrement les endpoints des services (via HTTP) pour collecter des métriques comme l'usage CPU, mémoire, disponibilité, erreurs, etc.

- **Base de données temporelle intégrée :** Les métriques sont stockées avec un horodatage précis, permettant des analyses fines sur la durée.
- **Alerting intégré :** Il peut déclencher des alertes via Alertmanager en fonction de règles définies (CPU trop élevé, service indisponible, etc.).
- **Intégration avec Kubernetes :** Prometheus détecte automatiquement les services déployés grâce à son intégration native avec Kubernetes.
- **Interopérabilité avec Grafana :** Les données collectées peuvent être directement exploitées par Grafana pour des visualisations avancées.

10) Node.js pour le développement backend



Figure 14 : logo de Node.JS

Node.js est un environnement d'exécution JavaScript open-source, basé sur le moteur V8 de Google Chrome. Il permet d'exécuter du code JavaScript côté serveur, offrant ainsi une solution performante et asynchrone pour le développement d'applications web. Dans le cadre de ce projet, Node.js est utilisé pour développer les API backend qui interagissent avec les services hébergés sur l'infrastructure Kubernetes.

Avantages de Node.js :

- **Modèle non bloquant (asynchrone)** : Node.js gère les opérations I/O de manière asynchrone, ce qui le rend idéal pour des applications nécessitant des échanges rapides avec une base de données ou des services réseau.
- **Écosystème riche (npm)** : Grâce au gestionnaire de paquets npm, il est facile d'intégrer des bibliothèques populaires pour accélérer le développement (ex. : Express.js, Axios, Sequelize, etc.).
- **Scalabilité** : Node.js s'adapte bien à des architectures microservices ou conteneurisées, notamment lorsqu'il est déployé dans des environnements comme Kubernetes.
- **Interopérabilité avec des bases de données et API** : Node.js facilite l'intégration avec des bases de données relationnelles ou NoSQL, ainsi qu'avec des API tierces.
- **Communauté active** : Une grande communauté de développeurs permet une résolution rapide des problèmes et un accès à une documentation abondante.

11) L'architecture de notre projet

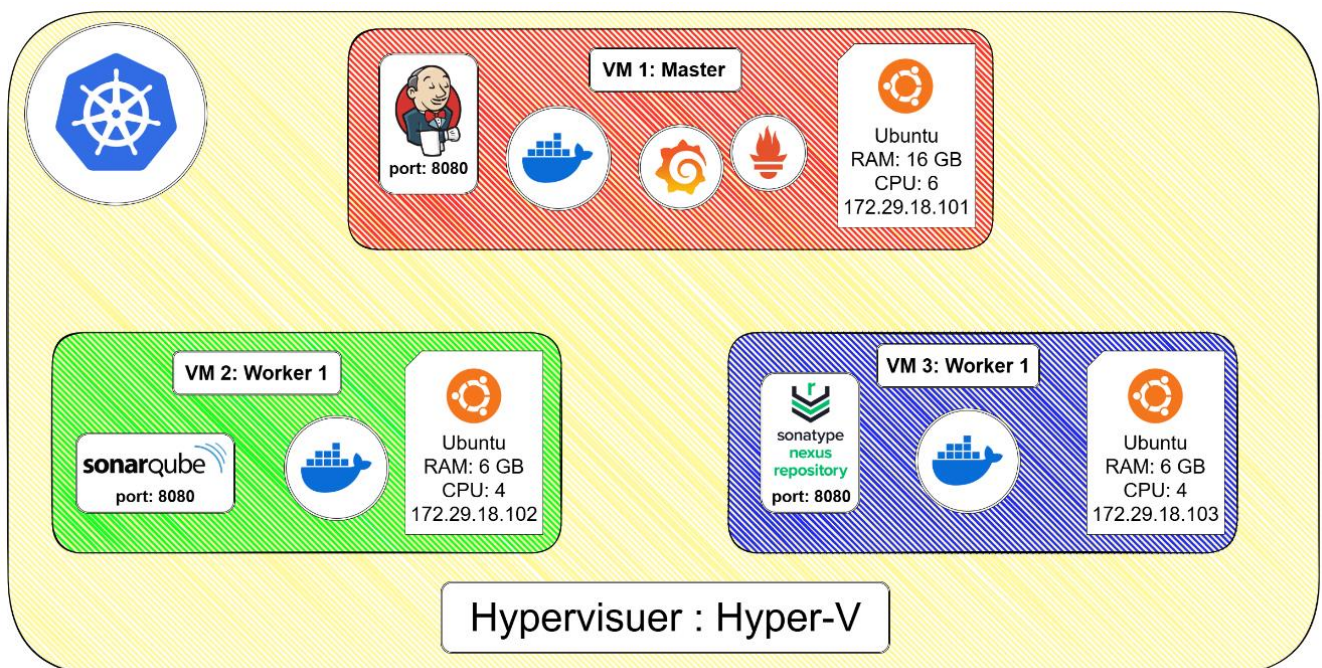


Figure 15 : Architecture physique

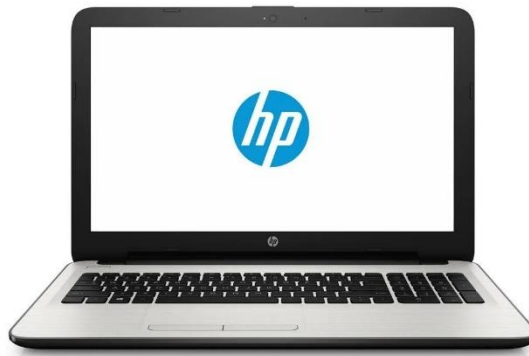
12) Ressources Matérielles

Pour la réalisation de ce projet, nous avons utilisé plusieurs équipements matériels en fonction des besoins de développement, des tests et de la rédaction du rapport.

A) Matériel utilisé en entreprise

Dans l'environnement de travail, nous nous connectons principalement à des machines virtuelle (VM) via SSH à l'aide de MobaXterm pour manipuler notre projet. Ainsi, la puissance de nos ordinateurs personnels est moins critique pour l'exécution du projet, mais nous les utilisons pour certaines tâches complémentaires.

- Laptop 1 (utilisé par Elomri Taysir) : HP 15 (15-ay007)



- Processeur : AMD E2-9000e Radeon R2
- Stockage :
 - SSD Emtec X150 250GB
 - HDD WD Blue 1To
- Mémoire vive : 4GB RAM 1866MHz
- Laptop 2 (utilisé par Hosni Iyed) : Dell Inspiron 13 5379



- Processeur : Intel Core i5-8250U @ 1.60GHz
- Stockage : SSD Samsung 860 Evo 250GB
- Mémoire vive : 8GB RAM 2400MHz

B) Matériel personnel utilisé pour les recherches et tests

En complément, un ordinateur bureautique personnel a été utilisé en dehors de l'entreprise pour :

- Tester différentes technologies avant leur mise en place dans le projet
- Simuler des scénarios de développement pour mieux comprendre les étapes
- Rédiger le rapport et effectuer des recherches techniques
- PC bureautique (utilisé par Hosni Iyed)



- Processeur : Intel Core i7 7700K
- Carte graphique : NVIDIA RTX 4060 Super
- Mémoire vive : 48GB RAM 2666MHz
- Stockage :
 - SSD NVMe Patriot 1To
 - HDD Samsung 3To (ST3000LM016)

L'utilisation de cet ordinateur pour les tests nous a permis de mieux anticiper les choix techniques avant leur déploiement dans l'environnement de travail.

Conclusion

Dans ce chapitre, nous avons défini les notions théoriques essentielles à notre projet et exploré les outils disponibles sur le marché. Après une analyse comparative, nous avons justifié notre choix technologique en fonction des exigences du projet.

Le prochain chapitre se concentrera sur la spécification des besoins et la conception de notre solution.

Chapitre 3 : Sprint 0 - Spécification des besoins

Introduction

Cette partie vise à détailler les spécifications des besoins pour la mise en place de notre **plateforme d'automatisation des applications web multi-services via l'approche CI/CD**. L'objectif est d'établir une base solide pour le développement, en définissant les exigences fonctionnelles et non fonctionnelles, le backlog du produit, l'architecture du système et la planification du projet.

I. Spécification des besoins

1) Besoins fonctionnels

Ce sont les fonctionnalités que le système doit absolument fournir :

1. Gestion de Code Source

Permettre aux développeurs de **pousser** et **cloner** du code via GitHub.

Suivre les changements de code pour chaque commit¹².

2. Intégration Continue avec Jenkins

Déclencher automatiquement un **build**¹³ à chaque commit.

Lancer des tests de qualité de code via SonarQube.

Créer et pousser des images Docker dans un registre.

Déployer automatiquement les artefacts dans Nexus Repository.

3. Analyse de la Qualité du Code

Scanner automatiquement le code avec SonarQube.

Générer un rapport d'analyse consultable.

4. Containerisation et Déploiement

Construire les images Docker des applications.

Déployer les containers via Kubernetes sur les bons Workers.

¹² **Commit** : Sauvegarde d'un ensemble de modifications dans un dépôt Git.

¹³ **Build** : Processus de compilation ou de transformation du code en application exécutable.

Orchestrer les Pods Flask¹⁴ et MongoDB¹⁵.

5. Gestion des Artéfacts

Permettre à Nexus de stocker et versionner les binaires/artéfacts.

6. Supervision

Surveiller l'état des VM, Pods, containers, etc. via Grafana et Prometheus.

Alerter en cas de défaillance ou de surcharge.

7. Gestion des Utilisateurs & Rôles

Authentification des utilisateurs.

Attribution des rôles : Admin, DevOps, Développeur, Superviseur.

Gérer les droits d'accès à Jenkins, GitHub, SonarQube, etc.

2) Besoins non fonctionnels

Ce sont les contraintes de qualité du système :

1. Sécurité

Authentification obligatoire pour chaque utilisateur.

Rôles bien définis avec droits d'accès restreints.

Chiffrement des communications entre Jenkins, Docker, Kubernetes et GitHub.

2. Performance

Temps de réponse rapide pour les builds et les déploiements.

Déploiement parallèle possible sur plusieurs Workers.

3. Scalabilité

Le système doit pouvoir ajouter facilement de nouveaux nœuds Kubernetes.

Jenkins et SonarQube doivent supporter plusieurs pipelines simultanés.

4. Disponibilité & Tolérance aux Pannes

Monitoring continu via Prometheus.

¹⁴ **Flask** : Framework web minimaliste en Python.

¹⁵ **MongoDB** : Base de données NoSQL orientée documents.

Notification automatique en cas de panne.

Redémarrage automatique des Pods plantés.

5. Modularité

Chaque composant (Jenkins, SonarQube, Nexus, etc.) est découplé et conteneurisé.

Les modules peuvent être mis à jour indépendamment.

6. Traçabilité

Historique des builds, déploiements et commits conservé.

Logs¹⁶ disponibles pour tous les composants du pipeline.

II. Backlog du produit

Notre backlog de produit est constitué de plusieurs **User Stories**, chacune associée à une complexité et une priorité métier.

ID	Feature	User Story	Complexité	Priorité
US1	Installation Jenkins	En tant qu'admin, je veux installer Jenkins pour gérer le pipeline CI/CD.	Faible	Élevée
US2	Installation SonarQube	En tant qu'admin, je veux installer SonarQube pour l'analyse statique du code.	Faible	Élevée
US3	Installation Kubernetes	En tant qu'admin, je veux installer Kubernetes pour déployer mes applications.	Moyenne	Élevée
US4	Installation Grafana/Prometheus	En tant qu'admin, je veux installer Grafana et Prometheus pour monitorer mes applications.	Moyenne	Élevée
US5	Configuration DockerHub	En tant qu'admin DevOps, je veux configurer DockerHub pour stocker mes images Docker.	Faible	Élevée
US16	Installation Nexus	En tant qu'admin DevOps, je veux installer Nexus pour stocker et gérer mes artefacts (hors Docker images).	Moyenne	Élevée
US6	Pipeline Jenkins CI/CD	En tant que DevOps, je veux configurer un pipeline automatisé (build, SonarQube, Docker push, déploiement Kubernetes et stockage d'artefacts sur Nexus).	Moyenne	Élevée

¹⁶ **Logs** : Fichiers enregistrant les événements du système ou de l'application.

US7	API REST centralisée	En tant que développeur backend, je veux créer une API REST pour centraliser la gestion des outils DevOps (Jenkins, Kubernetes, SonarQube, Grafana, Prometheus, Nexus).	Élevée	Élevée
US8	Notifications Email	En tant qu'utilisateur, je veux recevoir des notifications par email sur l'état des pipelines et des alertes critiques.	Moyenne	Élevée
US9	Gestion GitHub Webhooks	En tant que DevOps, je veux configurer les Webhooks GitHub pour déclencher automatiquement le pipeline Jenkins.	Moyenne	Élevée
US10	Gestion des logs	En tant que développeur backend, je veux sauvegarder et gérer les événements Webhooks et logs dans une base de données.	Moyenne	Moyenne
US11	Maquette UI Dashboard	En tant que développeur frontend, je veux concevoir une maquette ergonomique du dashboard centralisé.	Moyenne	Élevée
US12	Dashboard temps réel	En tant qu'utilisateur, je veux visualiser en temps réel l'état des outils (CI/CD, monitoring, analyses) via un seul dashboard.	Élevée	Élevée
US13	Actions depuis dashboard	En tant qu'utilisateur, je veux déclencher les pipelines Jenkins directement depuis l'interface du dashboard centralisé.	Moyenne	Élevée
US14	Affichage des notifications	En tant qu'utilisateur, je veux visualiser clairement les alertes récentes directement dans mon dashboard.	Moyenne	Élevée
US15	Gestion des préférences utilisateur	En tant qu'utilisateur, je veux gérer facilement mes préférences et paramètres depuis le dashboard (notifications, préférences utilisateur).	Moyenne	Moyenne

Tableau 2 : Backlog des fonctionnalités CI/CD

III. Diagramme de cas d'utilisation général

Le schéma ci-dessous représente les interactions entre les utilisateurs et notre système :

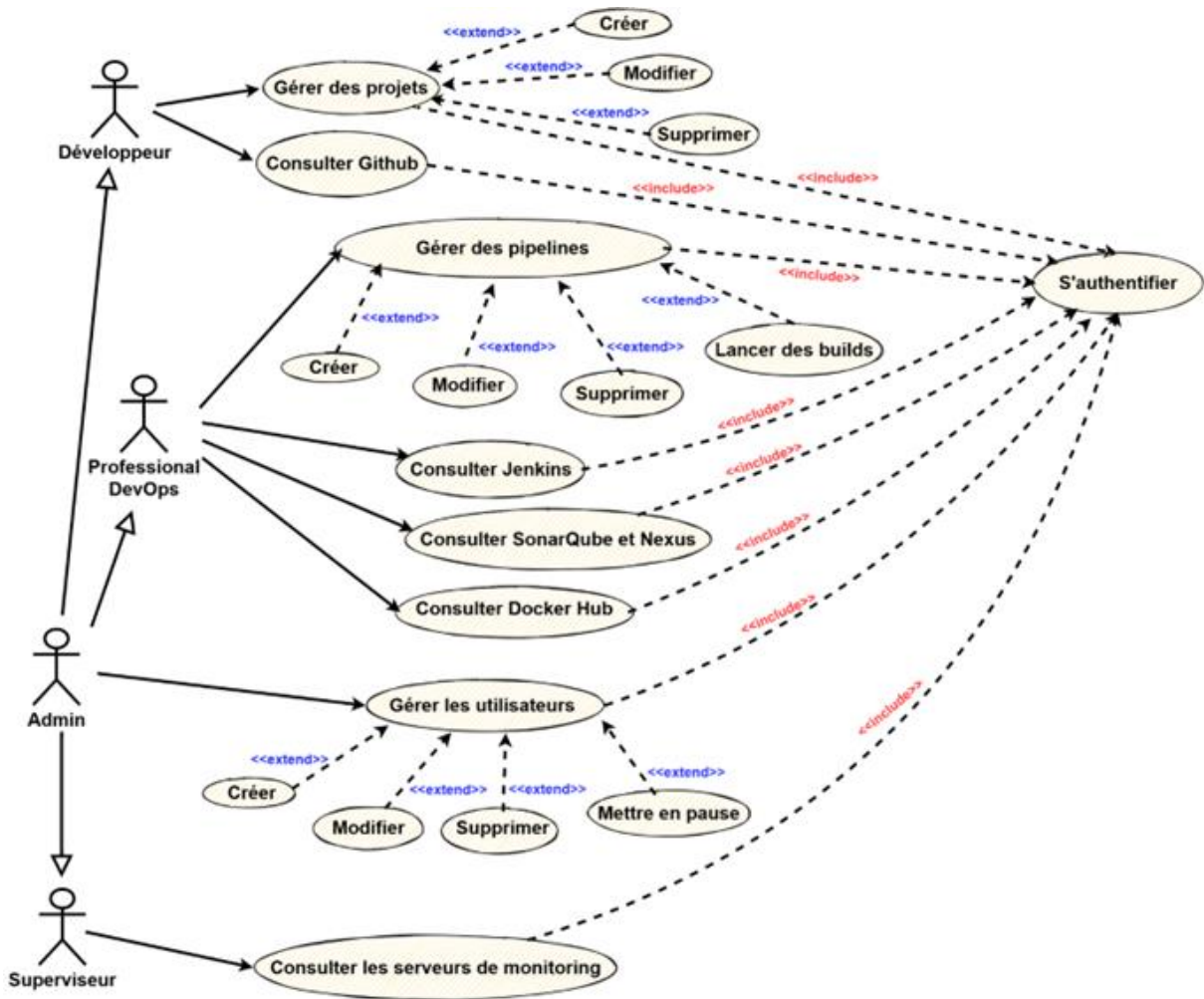


Figure 16: Cas d'utilisation et systèmes impliqués

1) Description scénariste du cas d'utilisation : Créer un pipeline

Description :

Ce cas d'utilisation décrit le processus par lequel un professionnel DevOps peut créer un nouveau pipeline via l'interface de l'application, en interagissant avec l'API Jenkins.

Acteur principal :

Professionnel DevOps

Précondition :

L'utilisateur doit être authentifié dans le système.

Scénario nominal :

1. Le DevOps accède à la page d'accueil de la plateforme.

2. Il clique sur l'option "**Créer un Pipeline**", ce qui ouvre un formulaire contenant les champs requis (nom de la pipeline, script de configuration, etc.).
3. Il remplit les champs demandés.
4. Il soumet le formulaire.
5. Le système vérifie la validité des données saisies.
6. Il envoie ensuite une requête à l'API Jenkins pour créer le pipeline avec les informations fournies.
7. En cas de succès, un message de confirmation est affiché.
8. Le DevOps peut alors voir son nouveau pipeline dans la liste des pipelines disponibles.

Postcondition :

Un pipeline est créée avec succès et est prête à être utilisée ou configurée.

Exceptions :

- Si des champs sont mal remplis, un message d'erreur s'affiche et l'utilisateur est invité à corriger les informations.
- Si une erreur survient lors de l'appel à l'API Jenkins, le système affiche un message d'échec.

2) Description scénariste du cas d'utilisation : Afficher le contenu d'un projet

Description :

Ce cas d'utilisation explique comment un développeur peut consulter le contenu (fichiers ou dossiers) d'un projet GitHub spécifique directement via l'interface de la plateforme.

Acteur principal :

Développeur

Précondition :

Le développeur doit être authentifié et avoir sélectionné un projet GitHub à afficher.

Scénario nominal :

1. Le développeur accède à la page d'accueil de la plateforme.
2. Il navigue vers la section "Projets".
3. Il choisit le projet GitHub qu'il souhaite explorer.
4. Le système interroge l'API GitHub pour récupérer la liste des fichiers et dossiers du projet.

5. La liste récupérée est affichée dans l'interface.
6. Le développeur sélectionne un fichier ou un dossier pour en consulter le contenu.
7. Le système effectue une seconde requête à l'API GitHub pour obtenir le contenu sélectionné.
8. Le contenu du fichier ou dossier choisi est ensuite affiché dans l'interface de l'application.

Postcondition :

Le contenu du fichier ou dossier sélectionné est visible dans l'interface.

Exceptions :

- Si le fichier ou le dossier sélectionné n'existe pas, le système affiche un message indiquant que le contenu est indisponible.

IV. Architecture du système

Composants principaux :

- Serveur Jenkins :
 - C'est le cœur de l'architecture. Il est responsable de l'exécution des pipelines de déploiement.
 - Il orchestre les différentes étapes du processus, en déclenchant les actions sur les autres composants.
- Travailleurs Jenkins (Travailleur 1 et Travailleur 2) :
 - Ce sont des agents qui exécutent les tâches définies dans les pipelines Jenkins.
 - Ils peuvent être sur des machines distinctes, ce qui permet de paralléliser les opérations et d'accélérer le processus de déploiement.
 - Dans cette figure, les travailleurs servent à exécuter les tâches de déploiement et d'analyse.
- Dépôt GitHub :
 - C'est là où le code source de l'application est stocké.
 - Jenkins récupère le code à partir de ce dépôt pour le construire et le déployer.
- SonarQube :
 - C'est un outil d'analyse de code qui vérifie la qualité du code source.
 - Jenkins envoie le code à SonarQube pour analyse et récupère les résultats.
- Sonatype Nexus :

- C'est un gestionnaire de référentiel qui stocke les artefacts (par exemple, les fichiers binaires) générés lors de la construction de l'application.
- Les travailleurs Jenkins déploie les artefacts sur Nexus et les récupèrent pour le déploiement.
- Cluster¹⁷ Kubernetes :
 - C'est une plateforme d'orchestration de conteneurs qui permet de déployer et de gérer des applications conteneurisées.
 - Les travailleurs Jenkins utilise Kubernetes pour déployer l'application.
- Application Déployée :
 - C'est l'application qui a été déployée avec succès sur le cluster Kubernetes.

Flux de travail :

1. Déclenchement du pipeline :
 - Le serveur Jenkins déclenche l'exécution du pipeline, soit manuellement, soit automatiquement (par exemple, lors d'une modification du code dans GitHub).
2. Récupération du code :
 - Le travailleur 1 récupère le code source à partir du dépôt GitHub.
3. Analyse du code :
 - Le travailleur 1 envoie le code à SonarQube pour analyse de qualité.
 - Le travailleur 1 récupère les résultats de l'analyse.
4. Construction et stockage des artefacts :
 - Le travailleur 2 construit l'application et stocke les artefacts résultants dans Sonatype Nexus.
5. Déploiement sur Kubernetes :
 - Le travailleur 2 récupère les artefacts de Sonatype Nexus et les utilise pour déployer l'application sur le cluster Kubernetes.
6. Application déployée :
 - L'application est maintenant en cours d'exécution sur le cluster Kubernetes.

¹⁷ **Cluster** : Groupe de machines connectées travaillant ensemble.

V. Planification de la release

Sprint	ID des tâches	Durée	Objectif principal	Livrable attendu
Sprint 1 : Installation des outils	US1, US2, US3, US4, US5, US16	4 Semaines	Installer les outils DevOps nécessaires à la CI/CD.	Jenkins, SonarQube, Kubernetes, Grafana, Prometheus, DockerHub et Nexus installés et opérationnels.
Sprint 2 : Configuration pipeline CI/CD	US6, US9	4 Semaines	Configurer et automatiser le pipeline complet CI/CD.	Pipeline CI/CD automatisé, déclenchement via GitHub Webhooks et stockage des artefacts sur Nexus opérationnel.
Sprint 3 : Développement Backend/API & Notifications	US7, US8, US10	4 Semaines	Développer l'API backend, gestion notifications par email et logs.	API REST centralisée (incluant Nexus), notifications par email fonctionnelles, gestion des logs intégrée.
Sprint 4 : Développement Frontend et Dashboard centralisé	US11, US12, US13, US14, US15	4 Semaines	Développer l'interface utilisateur centralisée (Dashboard).	Dashboard interactif en temps réel avec déclenchement des pipelines, gestion Nexus, et affichage des notifications.

Tableau 3 : Planification des sprints du projet

Conclusion

Ce Sprint 0 nous a permis d'établir clairement les exigences fonctionnelles et non fonctionnelles, de structurer précisément le backlog produit, et de définir une planification détaillée des prochaines étapes.

Grâce à cette base solide, nous sommes prêts à débiter le Sprint 1 : Installation des outils et à mener à bien le développement d'une plateforme CI/CD centralisée, robuste et répondant pleinement aux attentes des utilisateurs.

Chapitre 4 : Sprint 1 - Installation des outils DevOps

Introduction

Ce premier sprint constitue la fondation technique du projet. Il est dédié à l'installation, à la configuration et à la vérification des outils essentiels à une chaîne CI/CD moderne et automatisée :

Docker, Jenkins, SonarQube, Nexus, Prometheus, Grafana et Kubernetes.

Chaque section est accompagnée de captures d'écran réalisées tout au long du processus pour illustrer l'état d'avancement et les résultats obtenus.

I. Installation de Docker

Docker est utilisé pour la conteneurisation des applications, ce qui facilite leur portabilité, leur déploiement et leur gestion. Dans notre projet, Docker permet de standardiser les environnements de build et d'exécution tout au long du pipeline CI/CD.

1) Installation des paquets requis :

```
sudo apt install -y ca-certificates curl gnupg lsb-release
```

- **sudo apt install**: Cette commande est utilisée pour installer de nouveaux paquets.
- **-y**: Cet argument répond automatiquement "oui" à toutes les invites d'installation.
- **ca-certificates**: Fournit les certificats CA nécessaires pour vérifier les connexions SSL.
- **curl**: Un outil en ligne de commande pour transférer des données avec des URL.
- **gnupg**: Gère les clés GPG pour la signature et le cryptage.
- **lsb-release**: Fournit des informations sur la distribution Linux.

```
worker1@worker1:~$ sudo apt install -y ca-certificates curl gnupg lsb-release
[sudo] password for worker1:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.6).
curl set to manually installed.
gnupg is already the newest version (2.4.4-2ubuntu17.2).
gnupg set to manually installed.
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
worker1@worker1:~$
```

Figure 17 : Installation des paquets requis

2) Ajout de la clé GPG de Docker

```
sudo mkdir -p /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

- `sudo mkdir -p /etc/apt/keyrings`: Crée-le répertoire `/etc/apt/keyrings` si nécessaire.
- `curl -fsSL ...`: Télécharge la clé GPG de Docker.
- `sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`: Convertit et enregistre la clé GPG.

```
worker1@worker1:~$ sudo mkdir -p /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
worker1@worker1:~$
```

Figure 18 : Ajout de la clé GPG de Docker

3) Ajout du dépôt Docker

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu \  
$(lsb_release -cs) stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

- `echo ...`: Construit la chaîne de caractères pour la ligne du dépôt.
- `dpkg --print-architecture`: Détermine l'architecture du système.
- `lsb_release -cs`: Obtient le nom de la version de la distribution.
- `sudo tee ...`: Ajoute la ligne au fichier de configuration du dépôt.

```
worker1@worker1:~$ echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/li  
nux/ubuntu \  
$(lsb_release -cs) stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
worker1@worker1:~$
```

Figure 19 : Ajout du dépôt Docker

4) Installation du moteur Docker

```
sudo apt update  
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-  
compose-plugin
```

- `sudo apt update`: Met à jour la liste des paquets disponibles.
- `sudo apt install -y ...`: Installe les paquets Docker.
- `docker-ce`: Le moteur Docker.
- `docker-ce-cli`: L'interface en ligne de commande de Docker.
- `containerd.io`: Le runtime de conteneur.

- **docker-buildx-plugin**: Plugin pour les builds multi-architecture.
- **docker-compose-plugin**: Plugin pour Docker Compose.

```
worker1@worker1:~$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras libltdl7 libslirp0 pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin
  libltdl7 libslirp0 pigz slirp4netns
0 upgraded, 10 newly installed, 0 to remove and 1 not upgraded.
Need to get 120 MB of archives.
After this operation, 440 MB of additional disk space will be used.
Get:1 http://tn.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 containerd.io amd64 1.7.27-1 [30.5 MB]
Get:3 http://tn.archive.ubuntu.com/ubuntu noble/main amd64 libltdl7 amd64 2.4.7-7build1 [40.3 kB]
Get:4 http://tn.archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu3 [63.8 kB]
Get:5 http://tn.archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build2 [34.9 kB]
Get:6 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-buildx-plugin amd64 0.23.0-1~ubuntu.24.04~noble [34.6 MB]
Get:7 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-cli amd64 5:28.1.1-1~ubuntu.24.04~noble [15.8 MB]
Get:8 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce amd64 5:28.1.1-1~ubuntu.24.04~noble [19.2 MB]
Get:9 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-ce-rootless-extras amd64 5:28.1.1-1~ubuntu.24.04~noble [6,092 kB]
Get:10 https://download.docker.com/linux/ubuntu noble/stable amd64 docker-compose-plugin amd64 2.35.1-1~ubuntu.24.04~noble [13.8 MB]
Fetched 120 MB in 53s (2,269 kB/s)
Selecting previously unselected package pigz.
(Reading database ... 124849 files and directories currently installed.)
Preparing to unpack .../0-pigz_2.8-1_amd64.deb ...
Unpacking pigz (2.8-1) ...
Selecting previously unselected package containerd.io.
```

Figure 20 : Installation du moteur Docker

5) Vérification du statut de Docker

```
sudo systemctl status docker
```

- **sudo systemctl status docker** : Affiche le statut du service Docker

```
worker1@worker1:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-04-22 11:02:13 UTC; 8min ago
   TriggeredBy: ● docker.socket
     Docs: https://docs.docker.com
    Main PID: 22773 (dockerd)
      Tasks: 9
     Memory: 20.1M (peak: 21.6M)
        CPU: 501ms
    CGroup: /system.slice/docker.service
            └─22773 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Apr 22 11:02:12 worker1 dockerd[22773]: time="2025-04-22T11:02:12.919569973Z" level=info msg="detected 127.0.0.53 nameserver, >
Apr 22 11:02:12 worker1 dockerd[22773]: time="2025-04-22T11:02:12.951725482Z" level=info msg="Creating a containerd client" a>
Apr 22 11:02:12 worker1 dockerd[22773]: time="2025-04-22T11:02:12.992552356Z" level=info msg="Loading containers: start."
Apr 22 11:02:13 worker1 dockerd[22773]: time="2025-04-22T11:02:13.665053194Z" level=info msg="Loading containers: done."
Apr 22 11:02:13 worker1 dockerd[22773]: time="2025-04-22T11:02:13.698329319Z" level=info msg="Docker daemon" commit=01f442b c>
Apr 22 11:02:13 worker1 dockerd[22773]: time="2025-04-22T11:02:13.698441743Z" level=info msg="Initializing buildkit"
Apr 22 11:02:13 worker1 dockerd[22773]: time="2025-04-22T11:02:13.736810505Z" level=info msg="Completed buildkit initializati>
Apr 22 11:02:13 worker1 dockerd[22773]: time="2025-04-22T11:02:13.745741457Z" level=info msg="Daemon has completed initializa>
Apr 22 11:02:13 worker1 systemd[1]: Started docker.service - Docker Application Container Engine.
Apr 22 11:02:13 worker1 dockerd[22773]: time="2025-04-22T11:02:13.746440319Z" level=info msg="API listen on /run/docker.sock"
lines 1-22/22 (END)
```

Figure 21 : Vérification du statut de Docker

6) Utilisation de Docker sans sudo :

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

- `sudo usermod -aG docker $USER` : Ajoute l'utilisateur actuel au groupe docker.
- `newgrp docker` : Applique les modifications de groupe sans se déconnecter.

```
worker1@worker1:~$ sudo usermod -aG docker $USER
worker1@worker1:~$ newgrp docker
```

Figure 22 : Utilisation de Docker sans sudo

II. Mise en place de Jenkins

Jenkins est le moteur principal du pipeline CI/CD. Il déclenche automatiquement les étapes de build, test et déploiement. Pour cela, nous avons construit une image Docker personnalisée contenant tous les outils nécessaires.

1) Création du volume persistant Jenkins

```
mkdir -p ~/jenkins_home
```

```
sudo chown -R 1000:1000 ~/jenkins_home
```

- `mkdir -p ~/jenkins_home` : Crée le répertoire pour le volume Jenkins.
- `sudo chown -R 1000:1000 ~/jenkins_home` : Change la propriété du répertoire.

```
master@master:~$ mkdir -p ~/jenkins_home
master@master:~$ sudo chown -R 1000:1000 ~/jenkins_home
master@master:~$
```

Figure 23 : Création du volume persistant Jenkins

2) Création du Dockerfile personnalisé

```
FROM jenkins/jenkins:lts
```

```
USER root
```

```
# Install Docker CLI, zip, and dependencies
```

```
RUN apt-get update && \
```

```
Dockerfile
```

```
apt-get install -y docker.io zip unzip curl gnupg2 software-properties-common && \
apt-get clean
```

```
# Install SonarScanner manually from new location
```

```
RUN curl -sSLO /tmp/sonar-scanner.zip  
https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-sca> unzip  
/tmp/sonar-scanner.zip -d /opt_ && \  
  
ln -s /opt/sonar-scanner-4.8.0.2856-linux /opt/sonar-scanner && \  
  
rm /tmp/sonar-scanner.zip  
  
ENV PATH="/opt/sonar-scanner/bin:$PATH"  
  
# Add Jenkins user to Docker group RUN usermod -aG docker jenkins  
  
USER jenkins
```

- **FROM jenkins/jenkins:lts**: Image de base Jenkins.
- **USER root**: Change l'utilisateur pour root.
- **RUN apt-get ...**: Installe les outils nécessaires.
- **RUN curl ...**: Télécharge et installe SonarScanner.
- **ENV PATH="/opt/sonar-scanner/bin:\$PATH"**: Met à jour la variable PATH.
- **RUN usermod -aG docker jenkins**: Ajoute l'utilisateur Jenkins au groupe Docker.
- **USER jenkins**: Change l'utilisateur pour Jenkins.

```
master@master:~$ mkdir jenkins-docker && cd jenkins-docker  
master@master:~/jenkins-docker$ nano Dockerfile
```

```
GNU nano 7.2 Dockerfile  
FROM jenkins/jenkins:lts  
  
USER root  
  
# Install Docker CLI, zip, and dependencies  
RUN apt-get update && \  
    apt-get install -y docker.io zip unzip curl gnupg2 software-properties-common && \  
    apt-get clean  
  
# Install SonarScanner manually from new location  
RUN curl -sSLo /tmp/sonar-scanner.zip https://binaries.sonarsource.com/Distribution/sonar-scanner-cli/sonar-sca  
    unzip /tmp/sonar-scanner.zip -d /opt && \  
    ln -s /opt/sonar-scanner-4.8.0.2856-linux /opt/sonar-scanner && \  
    rm /tmp/sonar-scanner.zip  
  
ENV PATH="/opt/sonar-scanner/bin:$PATH"  
  
# Add Jenkins user to Docker group  
RUN usermod -aG docker jenkins  
  
USER jenkins
```

Figure 24 : Création du Dockerfile personnalisé

3) Construction de l'image personnalisée

```
docker build -t my-jenkins-with-tools .
```

- **docker build -t my-jenkins-with-tools .**: Construit l'image Docker avec le tag my-jenkins-with-tools.


```
master@master:~/jenkins-docker$ docker build -t my-jenkins-with-tools .
[+] Building 100.5s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 715B                                0.0s
=> [internal] load metadata for docker.io/jenkins/jenkins:lts    0.9s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> CACHED [1/4] FROM docker.io/jenkins/jenkins:lts@sha256:7aa631e4f036a348a42c3cdf8c31862141ea33605cbf91 0.0s
=> [2/4] RUN apt-get update && apt-get install -y docker.io zip unzip curl gnupg2 software-properiti 75.9s
=> [3/4] RUN curl -sLo /tmp/sonar-scanner.zip https://binaries.sonarsource.com/Distribution/sonar-scan 20.9s
=> [4/4] RUN usermod -aG docker jenkins                          0.3s
=> exporting to image                                             2.4s
=> => exporting layers                                              2.4s
=> => writing image sha256:2add7c4a5ff604d627cc935e71e742ab082c7bee4e4d1b65324257014b6da243 0.0s
=> => naming to docker.io/library/my-jenkins-with-tools          0.0s
master@master:~/jenkins-docker$
```

Figure 25 : Construction de l'image personnalisée

4) Lancement du conteneur Jenkins

```
docker run -d \
--name jenkins \
-p 8080:8080 -p 50000:50000 \
-v /var/run/docker.sock:/var/run/docker.sock \
-v ~/jenkins_home:/var/jenkins_home \
my-jenkins-with-tools
```

- `docker run -d ...`: Lance le conteneur en arrière-plan.
- `--name jenkins`: Nomme le conteneur.
- `-p 8080:8080 -p 50000:50000`: Expose les ports 8080 et 50000.
- `-v /var/run/docker.sock:/var/run/docker.sock`: Monte le socket Docker.
- `-v ~/jenkins_home:/var/jenkins_home`: Monte le volume Jenkins.
- `my-jenkins-with-tools`: Image à utiliser.

```
master@master:~/jenkins-docker$ nano Dockerfile
master@master:~/jenkins-docker$ docker run -d \
--name jenkins \
-p 8080:8080 -p 50000:50000 \
-v /var/run/docker.sock:/var/run/docker.sock \
-v ~/jenkins_home:/var/jenkins_home \
my-jenkins-with-tools
ac9436d0c1e3180b110e8ed72611cb63d914e8824266da714a29b9c654fb7446
master@master:~/jenkins-docker$
```

Figure 26 : Lancement du conteneur Jenkins

5) Accès à Jenkins via le navigateur

URL : <http://localhost:8080>

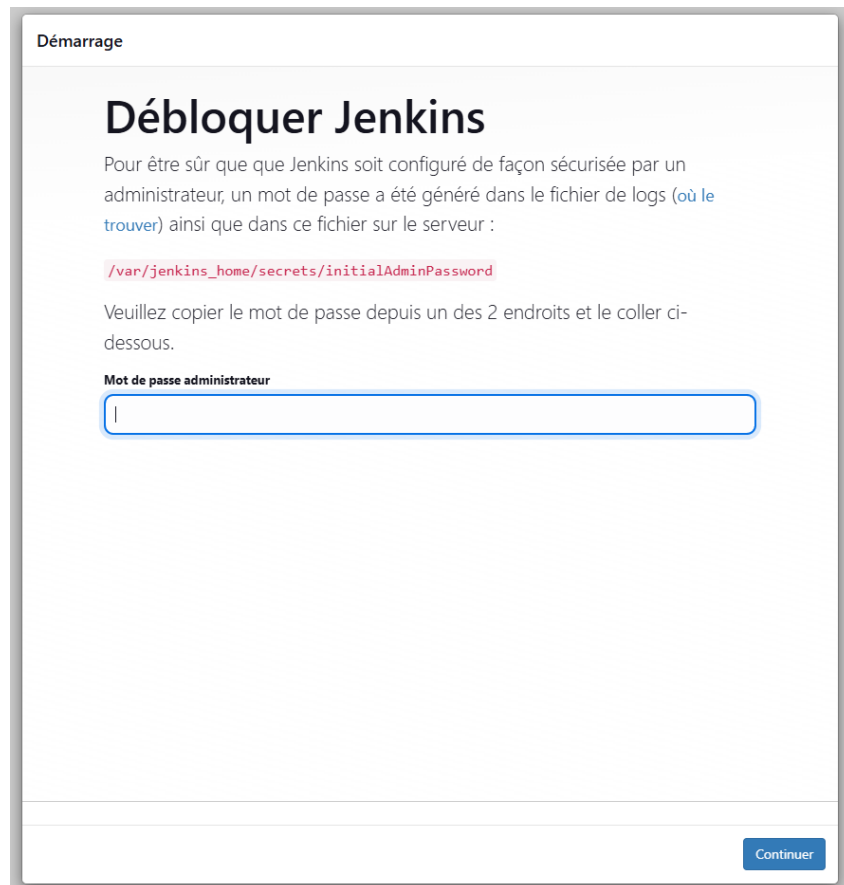


Figure 27 : Accès à Jenkins via le navigateur

6) Récupération du mot de passe administrateur Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

- `docker exec -it jenkins ...`: Exécute une commande dans le conteneur Jenkins.
- `cat /var/jenkins_home/secrets/initialAdminPassword`: Affiche le mot de passe initial.

```
master@master:~/jenkins-docker$ docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
212036a0ba214a54841c4c0c1db421fe
master@master:~/jenkins-docker$
```

Figure 28 : Récupération du mot de passe administrateur Jenkins

III. Déploiement de SonarQube

1) Téléchargement de l'image

```
docker pull sonarqube:lts
```

- `docker pull sonarqube:lts` : Télécharge la dernière image de la version LTS de SonarQube depuis Docker Hub

```
worker1@worker1:~$ docker pull sonarqube:lts
lts: Pulling from library/sonarqube
30a9c22ae099: Pull complete
6dbdd66677ae: Pull complete
3398a057cfd5: Pull complete
c3e4752518b6: Pull complete
f755fa637644: Pull complete
101e6de839e3: Pull complete
0355c6d609c4: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:189210f59439288b7467c9924bdf802591f93b7a10be6859fe35bcf7d2de4d36
Status: Downloaded newer image for sonarqube:lts
docker.io/library/sonarqube:lts
worker1@worker1:~$
```

Figure 29 : Téléchargement de SonarQube

2) Exécution du conteneur

```
docker run -d \
  --name sonarqube \
  -p 9000:9000 \
  -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
  sonarqube:lts
```

- `docker run -d ...` : Lance le conteneur en arrière-plan.
- `--name sonarqube` : Nomme le conteneur "sonarqube".
- `-p 9000:9000` : Expose le port 9000 pour l'interface web de SonarQube.
- `-e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true` : Désactive les vérifications de bootstrap d'Elasticsearch.
- `sonarqube:lts` : Spécifie l'image Docker à utiliser.

```
worker1@worker1:~$ docker run -d \
  --name sonarqube \
  -p 9000:9000 \
  -e SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true \
  sonarqube:lts
eaec5c65f702d0b73256501c96b950ca9b3a6a53d4adf321522a48eeb408cbf7
worker1@worker1:~$
```

Figure 30 : Exécution du conteneur SonarQube

3) Accès à SonarQube via le navigateur

Accès : <http://localhost:9000>

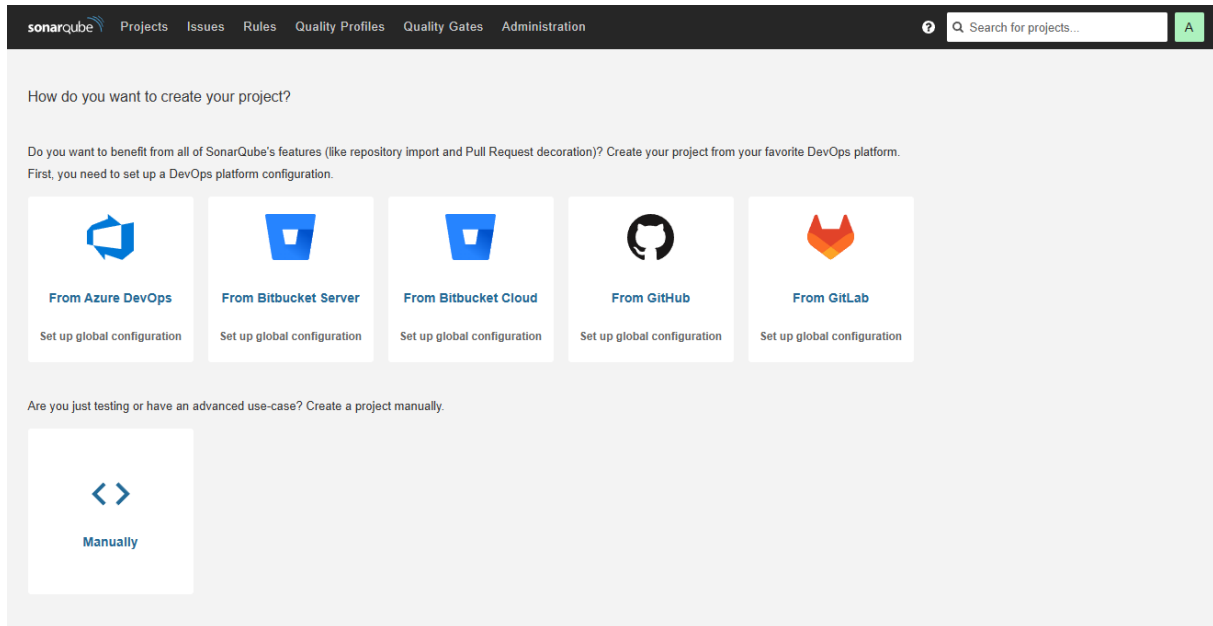


Figure 31 : Accès à SonarQube via le navigateur

IV. Mise en place de Nexus Repository

7) Téléchargement de l'image Nexus

```
docker pull sonatype/nexus3
```

- `docker pull sonatype/nexus3`: Télécharge l'image Docker de Nexus Repository Manager.

```
worker2@worker2:~$ docker pull sonatype/nexus3
Using default tag: latest
latest: Pulling from sonatype/nexus3
9d561c17444a: Pull complete
adc97f92dbce: Pull complete
8050bcef7fd3: Pull complete
a33c00ab1adf: Pull complete
98524a11fd20: Pull complete
03a187f95303: Pull complete
8ecf7f1ce625: Pull complete
Digest: sha256:0db7836f72ec4e85e2770fa22ab75823c239c45cf8162983caab092dcb88846d
Status: Downloaded newer image for sonatype/nexus3:latest
docker.io/sonatype/nexus3:latest
worker2@worker2:~$
```

Figure 32 : Téléchargement de l'image Nexus

8) Lancement du conteneur

```
docker run -d \
  --name nexus \
  --restart always \
  -p 8081:8081 \
  -p 5000:5000 \
  -v nexus-data:/nexus-data \
  sonatype/nexus3
```

- `docker run -d ...` : Lance le conteneur en arrière-plan.
- `--name nexus` : Nomme le conteneur "nexus".
- `--restart always` : Configure Docker pour **redémarrer automatiquement** le conteneur si celui-ci s'arrête ou si le système redémarre.
- `-p 8081:8081` : Fait le **mapping du port 8081 du conteneur vers le port 8081 de l'hôte**. C'est le port utilisé par l'interface web de Nexus Repository Manager..
- `-p 5000:5000` : Mappe un port supplémentaire (souvent utilisé pour un **dépôt Docker privé** configuré dans Nexus). Ce port n'est pas requis par défaut, mais peut être utile selon la configuration souhaitée.
- `-v nexus-data:/nexus-data` : Monte un volume pour stocker les données de Nexus.
- `sonatype/nexus3` : Spécifie l'image Docker à utiliser.

```
worker2@worker2:~$ docker run -d \
--name nexus \
--restart always \
-p 8081:8081 \
-p 5000:5000 \
-v nexus-data:/nexus-data \
sonatype/nexus3
Unable to find image 'sonatype/nexus3:latest' locally
latest: Pulling from sonatype/nexus3
9d561c17444a: Pull complete
adc97f92dbce: Pull complete
8050bcef7fd3: Pull complete
a33c00ab1adf: Pull complete
98524a11fd20: Pull complete
03a187f95303: Pull complete
8ecf7f1ce625: Pull complete
Digest: sha256:0db7836f72ec4e85e2770fa22ab75823c239c45cf8162983caab092dcb88846d
Status: Downloaded newer image for sonatype/nexus3:latest
```

Figure 33 : Lancement du conteneur nexus

9) Accès à SonaType nexus via le navigateur

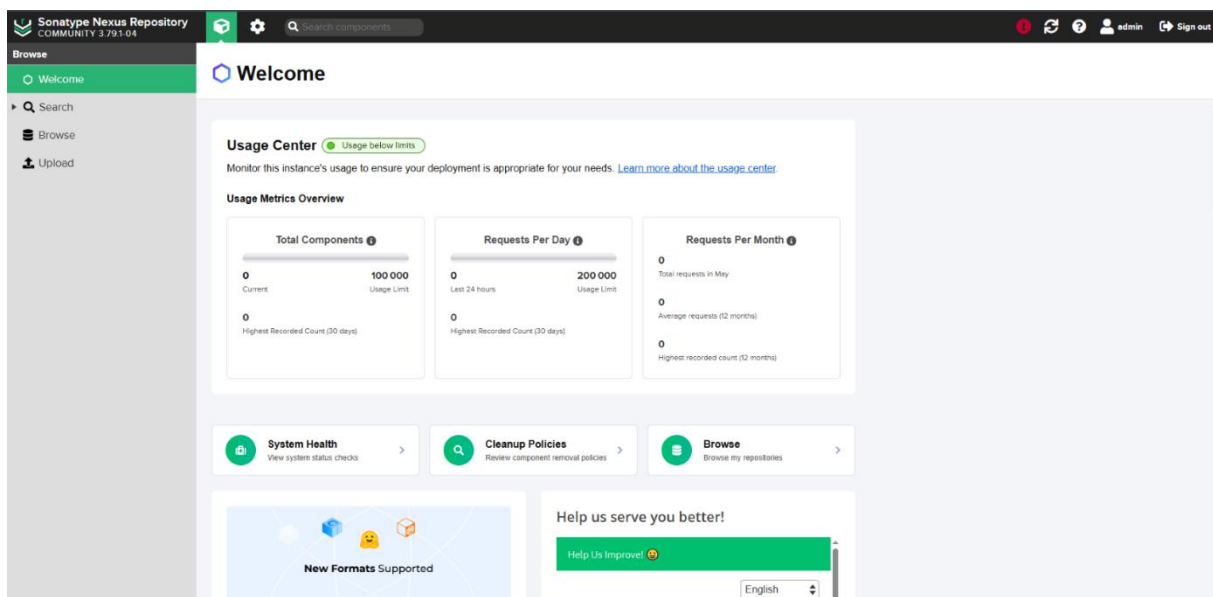


Figure 34 : Accès à SonaType nexus via le navigateur

V. Installation de Kubernetes avec kubeadm

Kubernetes est l'orchestrateur principal de notre infrastructure. Il nous permet de gérer automatiquement le cycle de vie des conteneurs, de manière distribuée et résiliente. Dans ce projet, nous avons mis en place un cluster composé d'un nœud maître et de deux nœuds workers. Voici les étapes détaillées, illustrées par les captures d'écran.

10) Configuration des noms d'hôtes et du fichier /etc/hosts

sudo hostnamectl set-hostname master

- `sudo hostnamectl set-hostname master`: Définit le nom d'hôte de la machine sur "master"

Ouvre-le fichier /etc/hosts avec un éditeur comme nano ou vim et **ajoute à la fin** du fichier les lignes suivantes (en remplaçant <master_ip>, etc. par les adresses IP réelles) :

```
<master_ip> master
<worker1_ip> worker1
<worker2_ip> worker2
```

- `/etc/hosts` : Fichier système qui contient des correspondances entre les adresses IP et les noms d'hôte.

The screenshot shows a terminal window with the command `sudo hostnamectl set-hostname master` being executed. The output shows the password prompt and the command being successful. Below the terminal, the `/etc/hosts` file is open in the nano editor. The file contains the following content:

```
GNU nano 7.2 /etc/hosts
127.0.0.1 localhost
127.0.1.1 master

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
192.168.1.72 master
192.168.1.70 worker1
192.168.1.71 worker2
```

The nano editor interface shows the file name `/etc/hosts` and the line number 1. The bottom status bar displays various keyboard shortcuts for editing and navigation.

Figure 35 : Configuration des noms d'hôtes et du fichier /etc/hosts

11) Désactivation du swap

Kubernetes exige que le swap soit désactivé sur tous les nœuds :

```
sudo swapoff -a  
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

- `sudo swapoff -a`: Désactive temporairement tous les espaces d'échange (swap).
- `sudo sed -i '/ swap / s/^/#/' /etc/fstab`: Commente (désactive) définitivement la ligne de swap dans le fichier `/etc/fstab`.

```
worker1@worker1:~$ sudo swapoff -a  
worker1@worker1:~$ sudo sed -i '/ swap / s/^/#/' /etc/fstab  
worker1@worker1:~$
```

Figure 36 : Désactivation du swap

12) Chargement des modules noyaux et configuration sysctl

Ces paramètres permettent au cluster d'avoir le routage réseau nécessaire entre les pods :

```
sudo modprobe overlay  
sudo modprobe br_netfilter  
cat <<EOF | sudo tee /etc/sysctl.d/kubernetes.conf  
net.bridge.bridge-nf-call-ip6tables = 1  
net.bridge.bridge-nf-call-iptables = 1  
net.ipv4.ip_forward = 1  
EOF  
  
sudo sysctl --system
```

- `sudo modprobe overlay`: Charge le module du noyau "overlay" pour la gestion des systèmes de fichiers overlay.
- `sudo modprobe br_netfilter`: Charge le module du noyau "br_netfilter" pour permettre le filtrage du trafic bridge.
- `cat <<EOF | sudo tee /etc/sysctl.d/kubernetes.conf ... EOF`: Crée un nouveau fichier de configuration `/etc/sysctl.d/kubernetes.conf` avec les paramètres réseau spécifiés.
- `sudo sysctl --system`: Recharge la configuration sysctl à partir de tous les fichiers de configuration.

```
worker1@worker1:~$ sudo modprobe overlay
sudo modprobe br_netfilter
worker1@worker1:~$ sudo tee /etc/sysctl.d/kubernetes.conf <<EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-map-count.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
* Applying /etc/sysctl.d/10-zero-page.conf ...
* Applying /usr/lib/sysctl.d/50-pid-max.conf ...
* Applying /usr/lib/sysctl.d/99-protect-links.conf ...
* Applying /etc/sysctl.d/99-sysctl.conf ...
* Applying /etc/sysctl.d/kubernetes.conf ...
* Applying /etc/sysctl.conf ...
kernel.apparmor_restrict_unprivileged_userns = 1
net.core.default_qdisc = fq_codel
kernel.printk = 4 4 1 7
net.ipv6.conf.all.use_tempaddr = 2
net.ipv6.conf.default.use_tempaddr = 2
kernel.kptr_restrict = 1
kernel.sysrq = 176
vm.max_map_count = 1048576
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.all.rp_filter = 2
kernel.yama.ptrace_scope = 1
vm.mmap_min_addr = 65536
kernel.pid_max = 4194304
fs.protected_fifos = 1
fs.protected_hardlinks = 1
fs.protected_regular = 2
fs.protected_symlinks = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
worker1@worker1:~$
```

Figure 37 : Chargement des modules noyaux et configuration sysctl

13) Installation d'exécution de conteneur

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release
curl https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo systemctl enable containerd
sudo systemctl restart containerd
```

- `sudo apt update`: Met à jour la liste des paquets disponibles.
- `sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release`: Installe les paquets nécessaires pour accéder aux dépôts HTTPS.
- `curl ... | sudo gpg --dearmor ...`: Ajoute la clé GPG du dépôt Docker.

- `echo "deb ... | sudo tee ..."`: Ajoute le dépôt Docker à la liste des sources de paquets.
- `sudo apt install -y containerd`: Installe le runtime de conteneur containerd.
- `sudo mkdir -p /etc/containerd`: Crée le répertoire de configuration de containerd.
- `containerd config default | sudo tee ...`: Génère la configuration par défaut de containerd et l'enregistre dans un fichier.
- `sudo systemctl enable containerd`: Active le service containerd pour qu'il démarre au démarrage du système.
- `sudo systemctl restart containerd`: Redémarre le service containerd.

```
worker1@worker1:~$ sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release
Hit:1 http://tn.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://download.docker.com/linux/ubuntu noble InRelease
Hit:3 http://tn.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://tn.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Ign:6 https://packages.cloud.google.com/apt kubernetes-xenial InRelease
Err:7 https://packages.cloud.google.com/apt kubernetes-xenial Release
 404 Not Found [IP: 216.58.212.110 443]
Reading package lists... Done
E: The repository 'https://apt.kubernetes.io kubernetes-xenial Release' does not have a Release file.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
apt-transport-https is already the newest version (2.7.14build2).
ca-certificates is already the newest version (20240203).
curl is already the newest version (8.5.0-2ubuntu10.6).
gnupg is already the newest version (2.4.4-2ubuntu17.2).
lsb-release is already the newest version (12.0-2).
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
worker1@worker1:~$ sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
File '/etc/apt/keyrings/docker.gpg' exists. Overwrite? (y/N) y
worker1@worker1:~$ sudo apt update
sudo apt install -y containerd.io
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://tn.archive.ubuntu.com/ubuntu noble InRelease
Hit:5 http://tn.archive.ubuntu.com/ubuntu noble-updates InRelease
Ign:4 https://packages.cloud.google.com/apt kubernetes-xenial InRelease
Hit:6 http://tn.archive.ubuntu.com/ubuntu noble-backports InRelease
Err:7 https://packages.cloud.google.com/apt kubernetes-xenial Release
 404 Not Found [IP: 216.58.212.110 443]
Reading package lists... Done
E: The repository 'https://apt.kubernetes.io kubernetes-xenial Release' does not have a Release file.
N: Updating from such a repository can't be done securely, and is therefore disabled by default.
N: See apt-secure(8) manpage for repository creation and user configuration details.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
containerd.io is already the newest version (1.7.27-1).
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
worker1@worker1:~$ sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
worker1@worker1:~$ sudo systemctl restart containerd
sudo systemctl enable containerd
worker1@worker1:~$ sudo systemctl restart containerd
sudo systemctl enable containerd
worker1@worker1:~$
```

Figure 38 : Installation d'exécution de conteneur Kubernetes

14) Ajout du dépôt Kubernetes et installation de kubeadm, kubelet et kubectl

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | \ sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /" | \
sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update
sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

- `sudo mkdir -p /etc/apt/keyrings`: Crée le répertoire pour les clés GPG des dépôts APT.
- `curl ... | sudo gpg --dearmor ...`: Télécharge et ajoute la clé GPG du dépôt Kubernetes.
- `echo "deb ... | sudo tee ..."`: Ajoute le dépôt Kubernetes à la liste des sources de paquets.
- `sudo apt update`: Met à jour la liste des paquets disponibles.
- `sudo apt install -y kubelet kubeadm kubectl`: Installe les outils Kubernetes :
 - `kubelet`: L'agent qui s'exécute sur chaque nœud et gère les conteneurs.
 - `kubeadm`: L'outil en ligne de commande pour initialiser et gérer un cluster Kubernetes.
 - `kubectl`: L'outil en ligne de commande pour interagir avec le cluster Kubernetes.
- `sudo apt-mark hold kubelet kubeadm kubectl`: Empêche la mise à jour automatique de ces paquets.

```
worker1@worker1:~$ sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /" | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /
worker1@worker1:~$ sudo apt update
Hit:1 http://tn.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 https://download.docker.com/linux/ubuntu noble InRelease
Hit:3 http://tn.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://tn.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:5 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.30/deb InRelease [1,189 B]
Get:7 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.30/deb Packages [16.6 kB]
Fetched 17.8 kB in 1s (21.7 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
worker1@worker1:~$
```

```
worker1@worker1:~$ sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  conntrack cri-tools kubernetes-cni
The following NEW packages will be installed:
  conntrack cri-tools kubeadm kubectl kubelet kubernetes-cni
0 upgraded, 6 newly installed, 0 to remove and 1 not upgraded.
Need to get 92.8 MB of archives.
```

```
User sessions running outdated binaries:
worker1 @ session #16: sshd[2621]
worker1 @ session #17: sshd[2623,2745]
worker1 @ session #2: login[1249]
worker1 @ user manager service: systemd[1461]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kubelet set on hold.
kubeadm set on hold.
kubectl set on hold.
worker1@worker1:~$
```

Figure 39 : Ajout du dépôt Kubernetes et installation de kubeadm, kubelet et kubectl

15) Initialisation du nœud master

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

- `sudo kubeadm init ...`: Initialise le nœud actuel comme nœud maître du cluster Kubernetes.
- `--pod-network-cidr=10.244.0.0/16`: Spécifie la plage d'adresses IP à utiliser pour les pods.

```
master@master:~$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16
I0422 13:54:28.587364 33358 version.go:256] remote version is much newer: v1.32.3; falling back to: stable-1.3
[init] Using Kubernetes version: v1.30.11
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action in beforehand using 'kubeadm config images pull'
W0422 13:54:29.244442 33358 checks.go:844] detected that the sandbox image "registry.k8s.io/pause:3.8" of the
ime is inconsistent with that used by kubeadm. It is recommended to use "registry.k8s.io/pause:3.9" as the CRI sa
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [kubernetes.kubernetes.default.kubernetes.default.svc kub
t.svc.cluster.local master] and IPs [10.96.0.1 192.168.1.72]
```

Figure 40 : Initialisation du nœud master (Kubernetes)

16) Configuration de kubectl sur le master

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

- `mkdir -p $HOME/.kube` : Crée le répertoire `.kube` dans le répertoire personnel de l'utilisateur.
- `sudo cp -i ...` : Copie le fichier de configuration administrateur Kubernetes vers le répertoire de l'utilisateur.
- `sudo chown $(id -u):$(id -g) ...` : Change la propriété du fichier de configuration pour l'utilisateur actuel.

17) Installation du plugin réseau Flannel

```
kubectl apply -f
https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-
flannel.yml
```

- `kubectl apply -f ...` : Déploie le plugin réseau Flannel sur le cluster Kubernetes. Flannel est utilisé pour permettre la communication entre les pods.

```
master@master:~$ mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
[sudo] password for master:
master@master:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
master      NotReady control-plane 14m   v1.30.11
```

Figure 41 : Installation du plugin réseau Flannel (Kubernetes)

18) Ajout des nœuds workers au cluster

Sur chaque worker, on exécute la commande `kubeadm join` fournie à la fin de l'initialisation du master.

```
worker2@worker2:~$ sudo kubeadm join 192.168.1.72:6443 --token 7xo6tp.i4tzqsmxnron5vm7 \
--discovery-token-ca-cert-hash sha256:4507023f0946fa4cfff46563239e6f28138f03e57c7f5d45c128d3157a254bb7d
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 1.010571131s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

worker2@worker2:~$
```

Figure 42 : Ajout des nœuds workers au cluster Kubernetes

19) Vérification du cluster

kubectl get nodes

- `kubectl get nodes`: Affiche la liste de tous les nœuds du cluster Kubernetes.

```
master@master:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
master      Ready     control-plane  22m   v1.30.11
worker1     Ready     <none>    65s   v1.30.11
worker2     Ready     <none>    58s   v1.30.11
master@master:~$
```

Figure 43 : Vérification du cluster Kubernetes

VI. Installation de Prometheus et Grafana via Helm et Kubernetes

Durant les premières phases du projet, nous avons initialement envisagé l'installation de Prometheus et Grafana via des conteneurs Docker. Cependant, cette solution s'est révélée limitée dans le cadre d'une infrastructure DevOps scalable et centralisée. Nous avons donc opté pour une **installation plus robuste via Helm Charts** au sein de notre cluster Kubernetes.

1) Création du fichier de configuration : grafana-public.yaml

Ce fichier nous permet de personnaliser le comportement de Grafana : exposition via NodePort, configuration de l'authentification, accès en lecture seule, activation de l'exploration des fichiers, etc.

nano grafana-public.yaml

```
grafana:  
  service:  
    type: NodePort  
    port: 80  
  adminUser: admin  
  adminPassword: prom-operator  
  grafana.ini:  
    server:  
      root_url: http://localhost:3000  
    security:  
      allow_embedding: true  
      disable_gravatar: true  
    auth.anonymous:  
      enabled: true  
      org_role: Viewer  
    users:  
      allow_sign_up: false  
      auto_assign_org: true  
      auto_assign_org_role: Viewer  
  sidecar:  
    dashboards:  
      enabled: true  
    datasources:  
      enabled: true
```

```
GNU nano 7.2
kubeEtd:
  enabled: false

grafana:
  service:
    type: NodePort
    port: 80

  adminUser: admin
  adminPassword: prom-operator

grafana.ini:
  server:
    root_url: http://localhost:3000
  security:
    allow_embedding: true
    disable_gravatar: true
  auth.anonymous:
    enabled: true
    org_role: Viewer
  users:
    allow_sign_up: false
    auto_assign_org: true
    auto_assign_org_role: Viewer

sidecar:
  dashboards:
    enabled: true
  datasources:
    enabled: true

prometheus:
  prometheusSpec:
    maximumStartupDurationSeconds: 300
```

Figure 44 : grafana-public.yaml

2) Installation via Helm

Une fois le fichier YAML prêt, nous avons utilisé la commande suivante pour déployer la stack Prometheus + Grafana dans le namespace `monitoring` :

```
helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--create-namespace \
-f grafana-public.yaml
```

Ce déploiement inclut à la fois Prometheus, Grafana, Alertmanager et les configurations nécessaires à leur bon fonctionnement dans un cluster Kubernetes.

Pour avoir un accès local à Grafana via port-forward :

```
export POD_NAME=$(kubectl --namespace monitoring get pod -l
"app.kubernetes.io/name=grafana,app.kubernetes.io/instance=monitoring" -o name)
kubectl --namespace monitoring port-forward $POD_NAME 3000
```

```

master@master:~$ nano grafana-public.yaml
master@master:~$ helm install monitoring prometheus-community/kube-prometheus-stack \
--namespace monitoring \
--create-namespace \
-f grafana-public.yaml
NAME: monitoring
LAST DEPLOYED: Tue May 20 12:04:22 2025
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=monitoring"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets monitoring-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo

Access Grafana local instance:

  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=monitoring" -o name)
  kubectl --namespace monitoring port-forward $POD_NAME 3000

Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
master@master:~$ kubectl get svc monitoring-grafana -n monitoring
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
monitoring-grafana   NodePort    10.109.160.137   <none>           80:30698/TCP     47s

```

Figure 45 : Installation via Helm

Cette méthode d'installation s'est révélée bien plus adaptée à notre besoin de centralisation, de scalabilité et de gestion simplifiée au sein de notre cluster Kubernetes. Elle nous a également permis d'exploiter pleinement l'**intégration native avec Grafana** pour générer dynamiquement des dashboards liés à Prometheus, utilisés par la suite dans notre interface superviseur.

VII. Résumé des outils

Outil	Port	Accès web	Statut installation
Docker	-	-	Installé
Jenkins	8080	http://localhost:8080	Installé
SonarQube	9000	http://localhost:9000	Installé
Nexus	8081	http://localhost:8081	Installé
Prometheus	-	-	Installé
Grafana	30698	http://localhost:30698	Installé
Kubernetes	-	-	Cluster prêt

Tableau 4 : Résumé des outils

Conclusion

L'ensemble des outils DevOps nécessaires a été installé, configuré et vérifié avec succès. Le cluster Kubernetes, incluant son nœud maître et ses nœuds workers, est opérationnel et prêt à accueillir des processus de déploiement automatisés orchestrés via Jenkins. Cette fondation technique solide garantit une continuité fluide pour les futures phases du projet.

Chapitre 5 : Sprint 2 – Mise en place d'un pipeline CI/CD complète avec Jenkins

Introduction

Dans ce chapitre, nous expliquons comment mettre en place un pipeline CI/CD fonctionnel avec Jenkins. Durant ce sprint, plusieurs pipelines ont été créés pour tester des applications web utilisant différentes technologies. Toutes ces applications ont été forkées depuis des dépôts publics GitHub, exclusivement à des fins de test et de validation des pipelines.

Nous nous concentrerons ici sur une application web développée avec le framework Flask et utilisant MongoDB comme base de données. Ce cas d’étude servira de fil conducteur pour illustrer les différentes étapes de mise en œuvre du pipeline CI/CD avec Jenkins.

I. Architecture cible du pipeline

1) Schéma logique de l'architecture

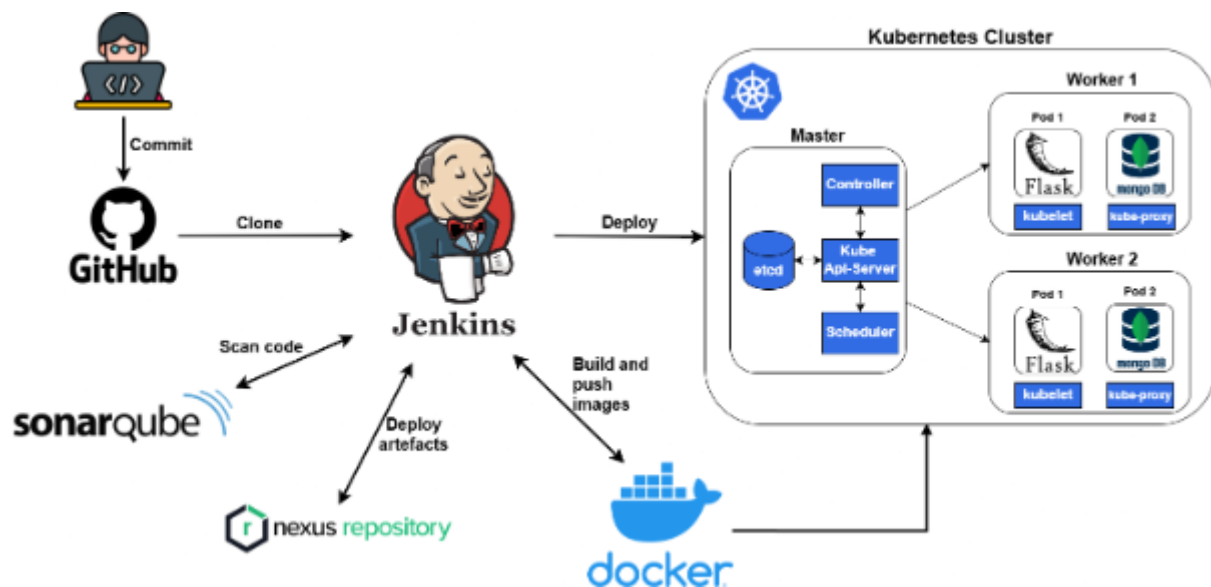


Figure 46 : L'architecture logique

2) Fonctionnement global du pipeline

La chaîne CI/CD suit un processus automatisé déclenché par une action de type *push* sur GitHub. Ce processus comprend les étapes suivantes :

- Jenkins récupère le code source via GitHub.
- Le code est analysé avec SonarQube pour détecter d'éventuelles vulnérabilités ou mauvaises pratiques.
- Une image Docker de l'application est construite.
- Cette image est publiée à la fois sur Docker Hub et dans un repository privé Nexus.

- Kubernetes se charge ensuite de récupérer l’image depuis le registre et de la déployer automatiquement dans le cluster.

Ce pipeline permet d’assurer une livraison continue et fiable des modifications apportées au code.

II. Installation des plugins Jenkins

1) Contexte

Avant de configurer le pipeline CI/CD (après avoir installé tous les prérequis lors du sprint précédent), la première étape consiste à installer les plugins indispensables à Jenkins.

2) Liste des plugins indispensables

Pour garantir l’efficacité et la fiabilité de notre pipeline CI/CD, il est essentiel de disposer de plugins adaptés. Le tableau ci-dessous récapitule les extensions à installer en priorité et leurs rôles.

Plugin	Fonction
Docker Pipeline	Gérer les étapes de build et de push des images Docker
Kubernetes CLI Plugin	Interagir avec le cluster Kubernetes
GitHub Integration Plugin	Connecter Jenkins aux dépôts GitHub
SonarQube Scanner for Jenkins	Lancer des analyses de qualité de code avec SonarQube
Nexus Artifact Uploader Plugin	Transférer les artefacts (jar, war, images Docker...) vers Nexus
Email Extension Plugin	Configurer et envoyer des notifications par e-mail

Tableau 5 : Liste des plugins indispensables

3) Procédure d’installation

1. Accéder à **Tableau de board > Administrer Jenkins > plugins**.
2. Dans l’onglet plugins disponibles, saisissez le nom de chaque plugin dans le champ de recherche.
3. Cocher les plugins listés ci-dessus, puis cliquez sur Installer.

4. Patienter jusqu’à la fin de l’installation, puis vérifier dans l’onglet plugins installés que tous sont bien présents.
5. Si nécessaire, redémarrer Jenkins pour finaliser l’intégration.

4) Illustration de l’installation

Voici un exemple de capture d’écran montrant l’installation d’un plugin :

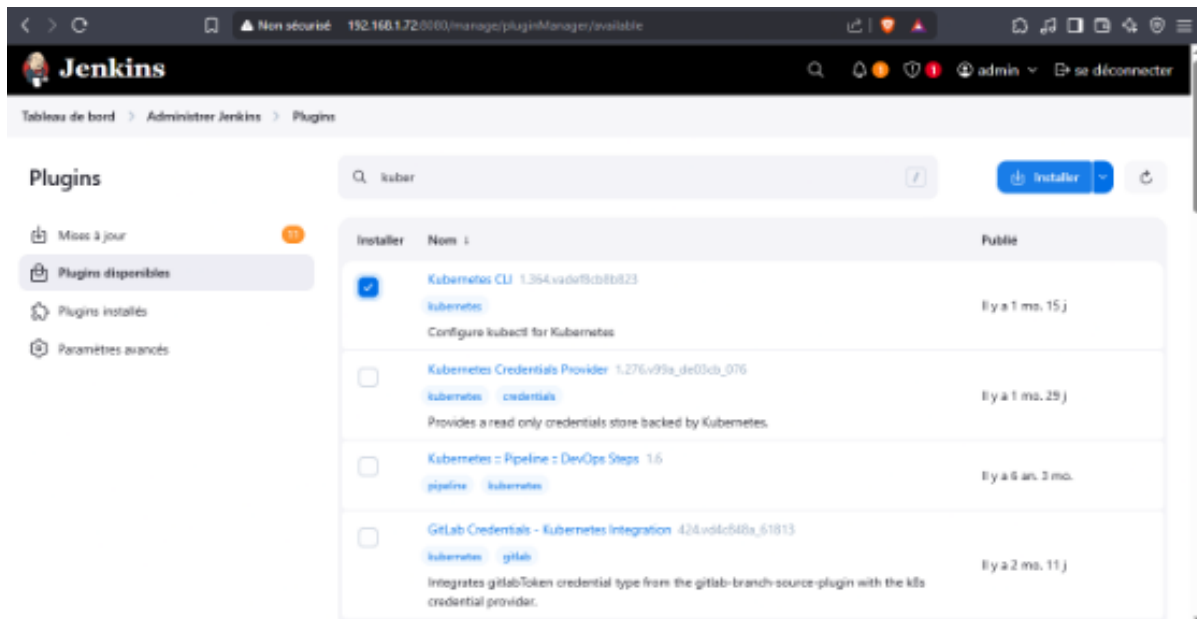


Figure 47 : Illustration de l’installation d’un plugin Jenkins

III. Ajout des identifiants d’accès (Credentials)

La deuxième étape essentielle dans la mise en place de notre pipeline CI/CD consiste à configurer les credentials nécessaires pour l’authentification de Jenkins. Selon les services, les modes d’identification varient :

- **Nexus** : identifiant et mot de passe
- **SonarQube** : token d’accès
- **GitHub & DockerHub** : nom d’utilisateur et token.

Avant l’enregistrement de ces informations dans Jenkins, il convient de générer ou de récupérer les identifiants et jetons pour chaque plateforme. Cette opération garantit une authentification automatique et sécurisée. Les sections suivantes décrivent la procédure de récupération des jetons sur SonarQube, DockerHub et GitHub, puis leur configuration dans Jenkins.

1) Petit rappel sécurité :

- Les tokens et mots de passe ne doivent jamais être partagés dans des documents publics ni figurer dans les journaux de build.
- Les informations d'authentification doivent être stockées dans Jenkins pour éviter leur inclusion accidentelle dans le code source.
- Les jetons doivent être générés avec les permissions minimales requises (permission du moindre privilège).

2) Token SonarQube

1. Se Connecter à l'instance SonarQube.
2. Cliquer sur l'avatar (en haut à droite) > **My Account** > **Security**.
3. Dans **Generate Tokens**, saisissez un nom (par ex. "Jenkins CI") et cliquer sur **Generate**.
4. Copier le jeton généré.

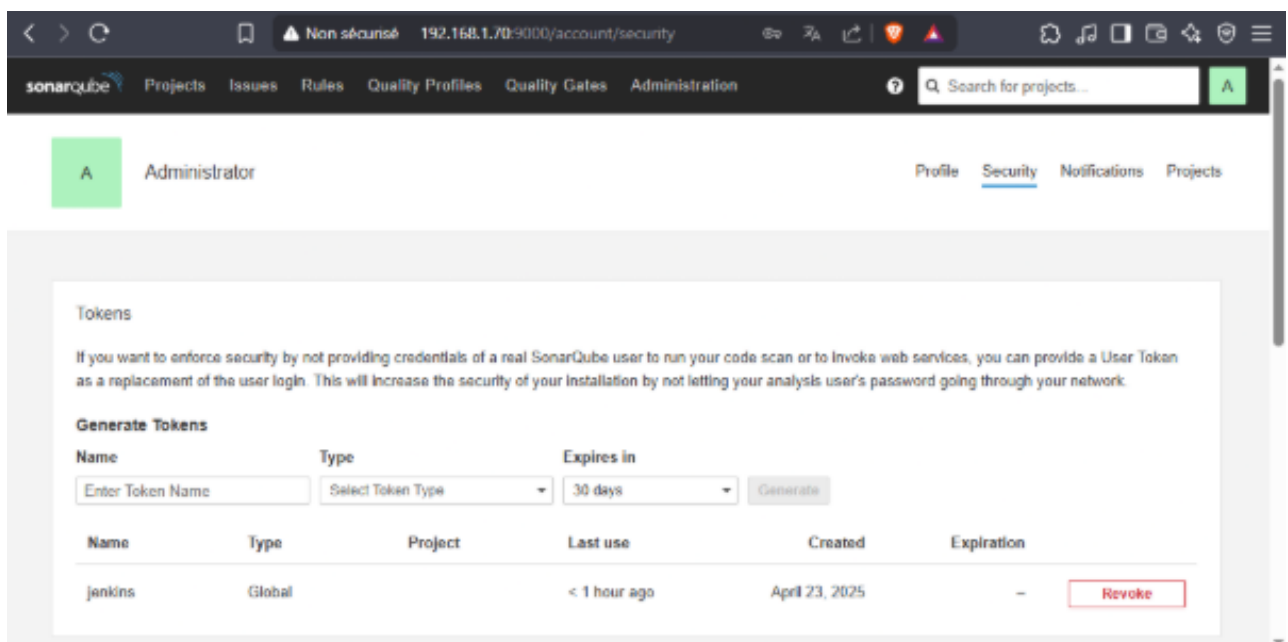


Figure 48 : Token SonarQube

3) Token DockerHub

1. Se connecter à DockerHub.
2. Naviguer vers **Account Settings** → **Security** → **Personal Access Token**.
3. Définir un nom pour le jeton puis générer.
4. Copier le jeton affiché.

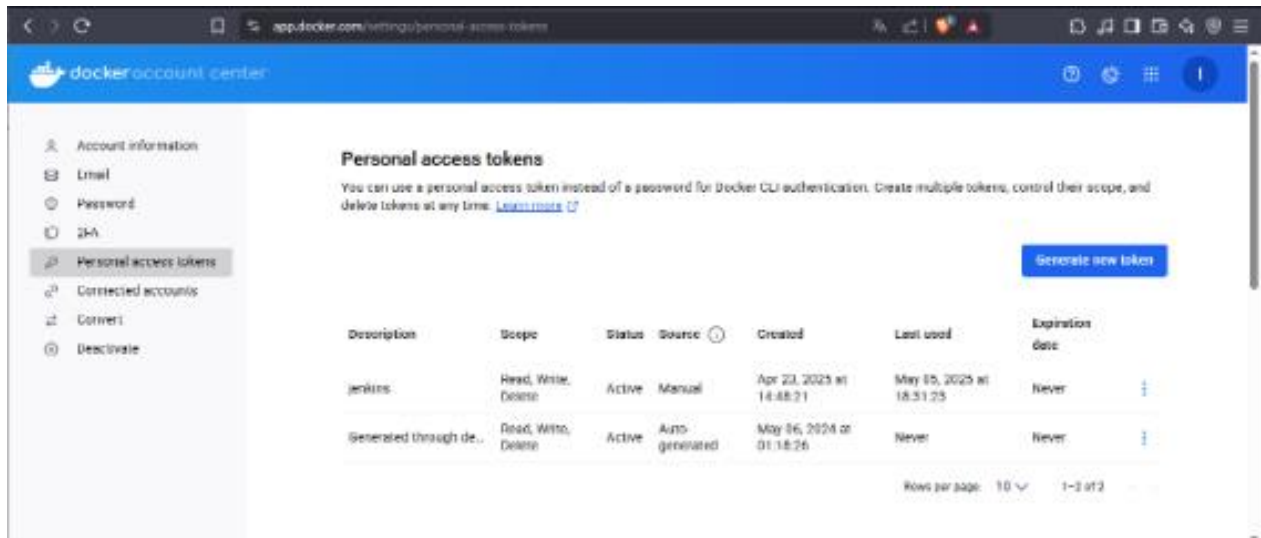


Figure 49 : Token DockerHub

4) Token GitHub

1. Se connecter à GitHub.
2. Ouvrir **Avatar** → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**.
3. Sélectionner les scopes requis (repo, workflow, write:packages, etc.) puis générer le jeton.
4. Copier le jeton.

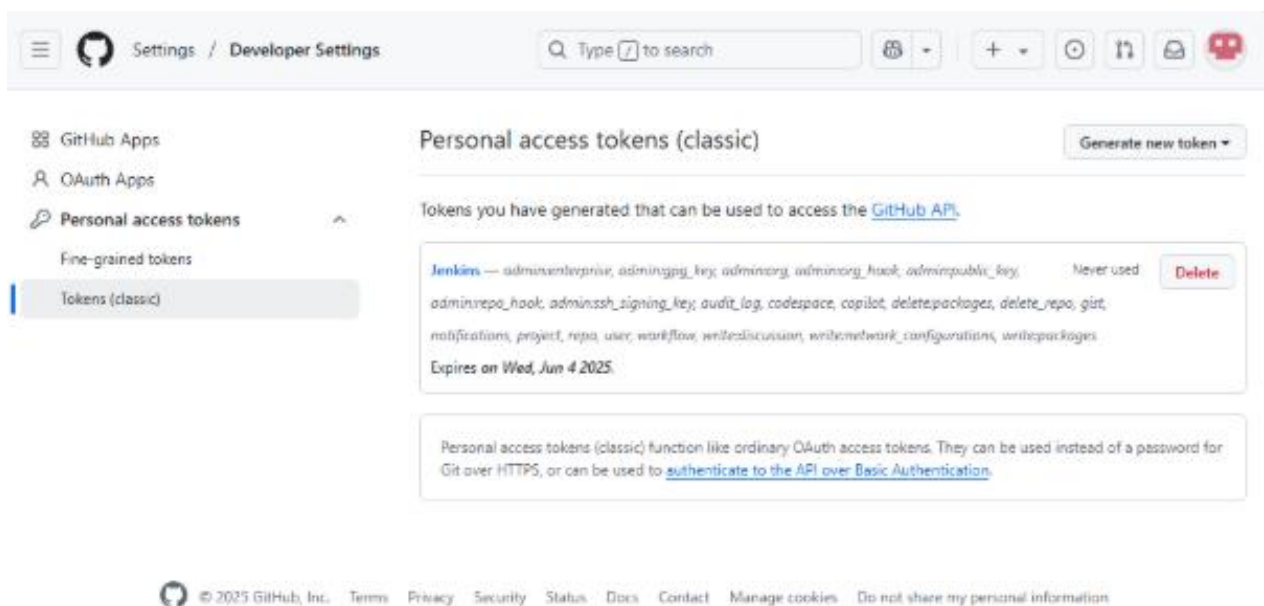


Figure 50 : Token GitHub

5) Ajout Credentials dans Jenkins

1. Se rendre dans **Manage Jenkins** → **Credentials** → **System** → **Global credentials (unrestricted)**.
2. Cliquer sur **Add Credentials**.
3. Pour chaque service :
 - a. **Kind**: *Username with password* (Nexus, GitHub, DockerHub) ou *Secret Text* (SonarQube).
 - b. **Username** : identifiant (ou « token » pour *Secret Text*).
 - c. **Password/Secret** : jeton ou mot de passe copié.
4. **ID** et **Description** : libellés explicites (par ex. *sonarqube-token*, *dockerhub-token*, *github-token*).
5. Enregistrer et vérifier la présence de chaque credential.

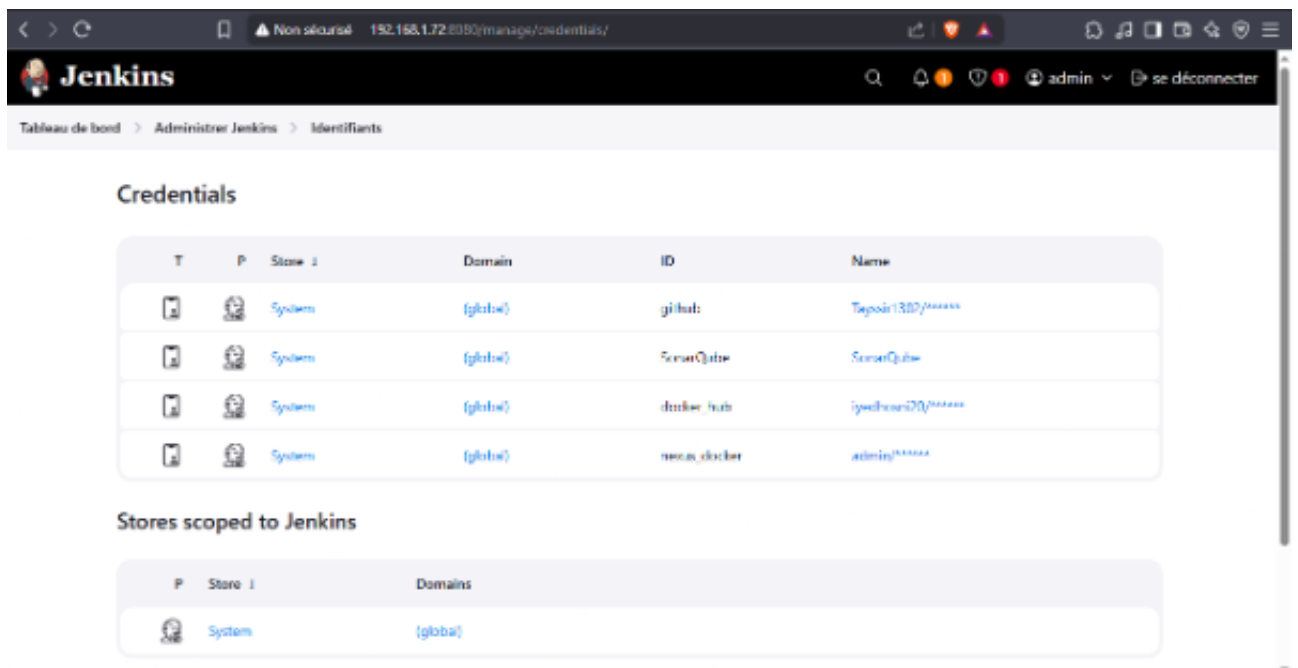


Figure 51 : Credentials dans Jenkins

IV. Préparation du projet Flask/MongoDB

La troisième étape consiste à préparer l’environnement de travail du projet : effectuer un fork du dépôt original, puis intégrer les fichiers essentiels (Dockerfile, requirements.txt, etc.). Cette préparation garantit que le code source est correctement configuré et prêt à être pris en charge par le pipeline CI/CD.

1) Fork du repository GitHub

1. Se rendre sur le dépôt GitHub original.
2. Cliquer sur **Fork**, sélectionner le compte personnel ou de l'organisation.

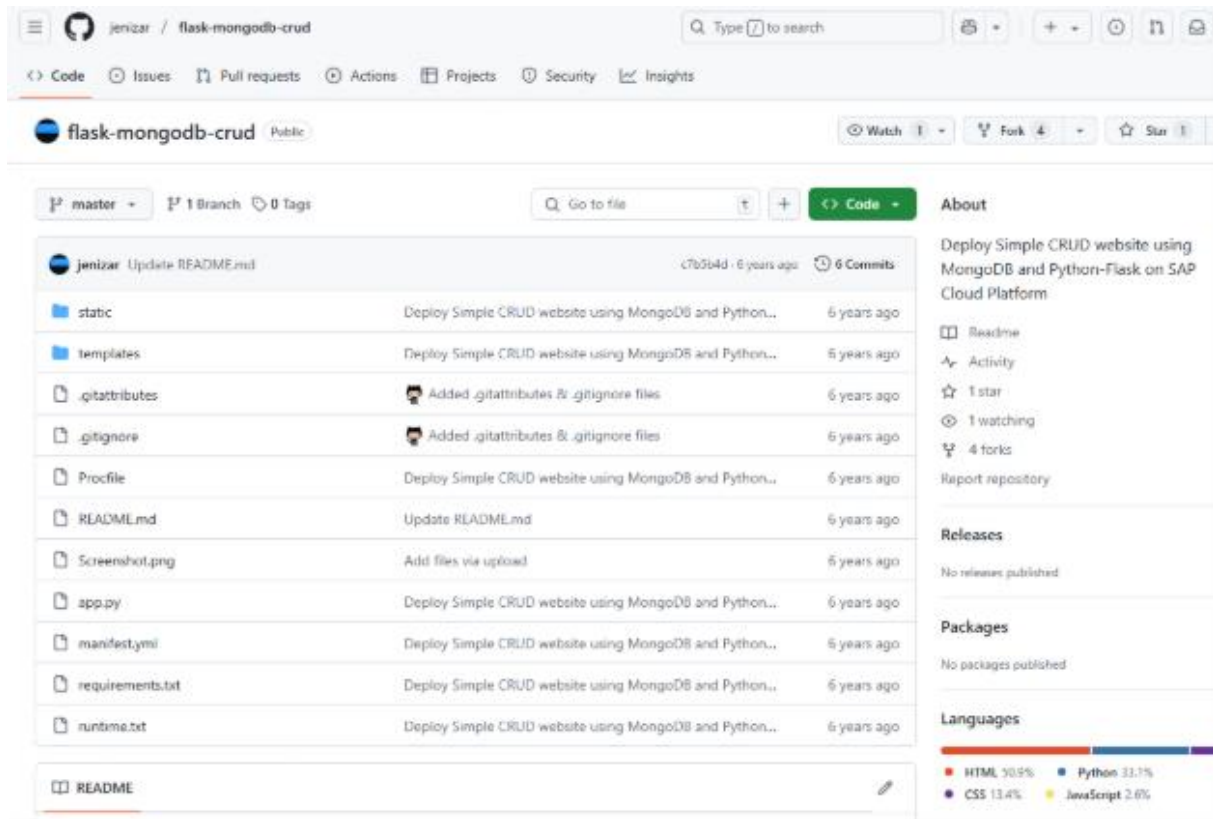


Figure 52 : Fork du repository GitHub

2) Ajout du Dockerfile

Objectif :

- Containeriser l'application afin de garantir un environnement d'exécution homogène et reproductible.
- Faciliter l'intégration dans la chaîne CI/CD grâce à la gestion d'images Docker.

Dockerfile:

```
FROM python:3.9-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

- **FROM python:3.9-slim** : image de base légère avec Python 3.9 pour réduire la taille de l'image finale.

- **WORKDIR /app** : définit le répertoire de travail à l'intérieur du conteneur.
- **COPY . .** : copie l'ensemble du code source du projet dans le répertoire de travail.
- **RUN pip install --no-cache-dir -r requirements.txt** : installe les dépendances listées sans conserver le cache pip.
- **EXPOSE 5000** : informe Docker que l'application écoute sur le port 5000.
- **CMD ["python", "app.py"]** : commande de démarrage de l'application Flask

Requirements.txt :

L'utilisation d'un fichier requirements.txt est optionnelle et dépend de la manière dont l'application gère ses dépendances.

```
flask
pymongo
pytest
pytest-cov
requests
```

3) Ajout de deployment.yaml

Ce fichier définit les ressources Kubernetes nécessaires au déploiement de l'application Flask et de la base de données MongoDB :

- Deployments pour assurer le nombre de réplicas et la mise à l'échelle automatique.
- Services pour exposer l'application et la base de données au sein du cluster.

Deployment.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: mongodb
spec:
  selector:
    app: mongodb
  ports:
    - port: 27017
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongodb
spec:
  selector:
    matchLabels:
      app: mongodb
  replicas: 1
  template:
```

```
  metadata:
    labels:
      app: mongodb
  spec:
    containers:
      - name: mongodb
        image: mongo:5.0
        ports:
          - containerPort: 27017
        volumeMounts:
          - name: mongo-persistent-storage
            mountPath: /data/db
    volumes:
      - name: mongo-persistent-storage
        emptyDir: {}
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-mongodb-test-project
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-mongodb-test-project
  template:
    metadata:
      labels:
        app: flask-mongodb-test-project
    spec:
      containers:
        - name: flask-mongodb-test-project
          image: iyedhosni20/flask-mongodb-test-project:latest
          ports:
            - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: flask-mongodb-test-project-service
spec:
  selector:
    app: flask-mongodb-test-project
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 31711
  type: NodePort
```

4) Modification de la connexion à la base de données

Dans un environnement Kubernetes, chaque service est exécuté dans un pod distinct et accessible via un Service interne. Contrairement à une connexion locale (localhost), la base de données MongoDB est hébergée dans son propre pod et exposée sous un nom DNS défini par le Service Kubernetes. Pour garantir la découverte de service, la résilience et la portabilité de l’application, il est impératif de rediriger la connexion vers cette nouvelle adresse.

Mise en œuvre technique

- Création d’un Service Kubernetes nommé mongo-service pointant vers le pod MongoDB sur le port 27017.
- Mise à jour de l’URI de connexion dans le code Flask :
- Conservation de l’utilisation de pymongo.MongoClient pour établir la connexion, sans modifier la bibliothèque cliente.

Cette adaptation assure que l’application Flask communique directement avec le pod MongoDB au sein du cluster Kubernetes, bénéficiant ainsi du service de découverte, de la scalabilité et de la haute disponibilité.

```
16 - client = MongoClient("mongodb://MNHZEJgGbE97H4x:k859FABWg4n2DUVD@10.11.241.41:58384/S6qQHHSP44UX7RXa") #host uri
17 - db = client.S6qQHHSP44UX7RXa #Select the database
18 - todos = db.todo #Select the collection name

13 + client = pymongo.MongoClient("mongodb://mongodb:27017")
14 + db = client.S6qQHHSP44UX7RXa
15 + todos = db.todo
```

Figure 53 : Mise à jour de l’URI de connexion dans le code Flask

V. Configuration des notifications par email

Les notifications par email permettent de tenir l’équipe informée en temps réel de l’état des builds et déploiements Jenkins :

- **Alertes sur échecs** pour intervenir rapidement en cas de problème.
- **Confirmations de succès** pour valider les livraisons et déploiements automatisés.

1) Procédure

Dans Jenkins, aller dans **Manage Jenkins > Configure System**.

- **Paramétrer le serveur SMTP**
 - **SMTP server:** smtp.gmail.com

- **Use SMTP Authentication** : cocher, puis renseigner l'adresse email et le mot de passe ou token d'application.
- **Use SSL** : cocher
- **Port** : 465
- **Définir l'adresse de réponse**
 - **Reply-To Address**: Address e-mail (ex. iyed.hosni123@gmail.com)
- **Enregistrer la configuration**
 - Cliquer sur **Save** en bas de la page.

Remarque :

- Pour Gmail, il est recommandé d'utiliser un « App Password » généré depuis un compte Google avec l'authentification à deux facteurs activés.
- Veiller à ne pas exposer ces identifiants dans des logs ou fichiers publics.

Notification par email

Serveur SMTP

smtp.gmail.com

Suffixe par défaut des emails des utilisateurs

Avancé / Initial

☒ Use SMTP Authentication

Nom d'utilisateur

iyed.hosni123@gmail.com

Mot de passe

Concealed

Change Password

☒ Utiliser SSL

☐ Use TLS

Port SMTP

465

Adresse de réponse

iyed.hosni123@gmail.com

Jeu de caractères

Figure 54 : Configuration des notifications par email

VI. Création de la pipeline Jenkins

1) Création du projet Pipeline

- Naviguer vers **Tableau de bord > Tous > Nouveau Item**
- Ajouter le nom de pipeline
- Cliquer sur **Pipeline**

- Cliquer sur **OK**

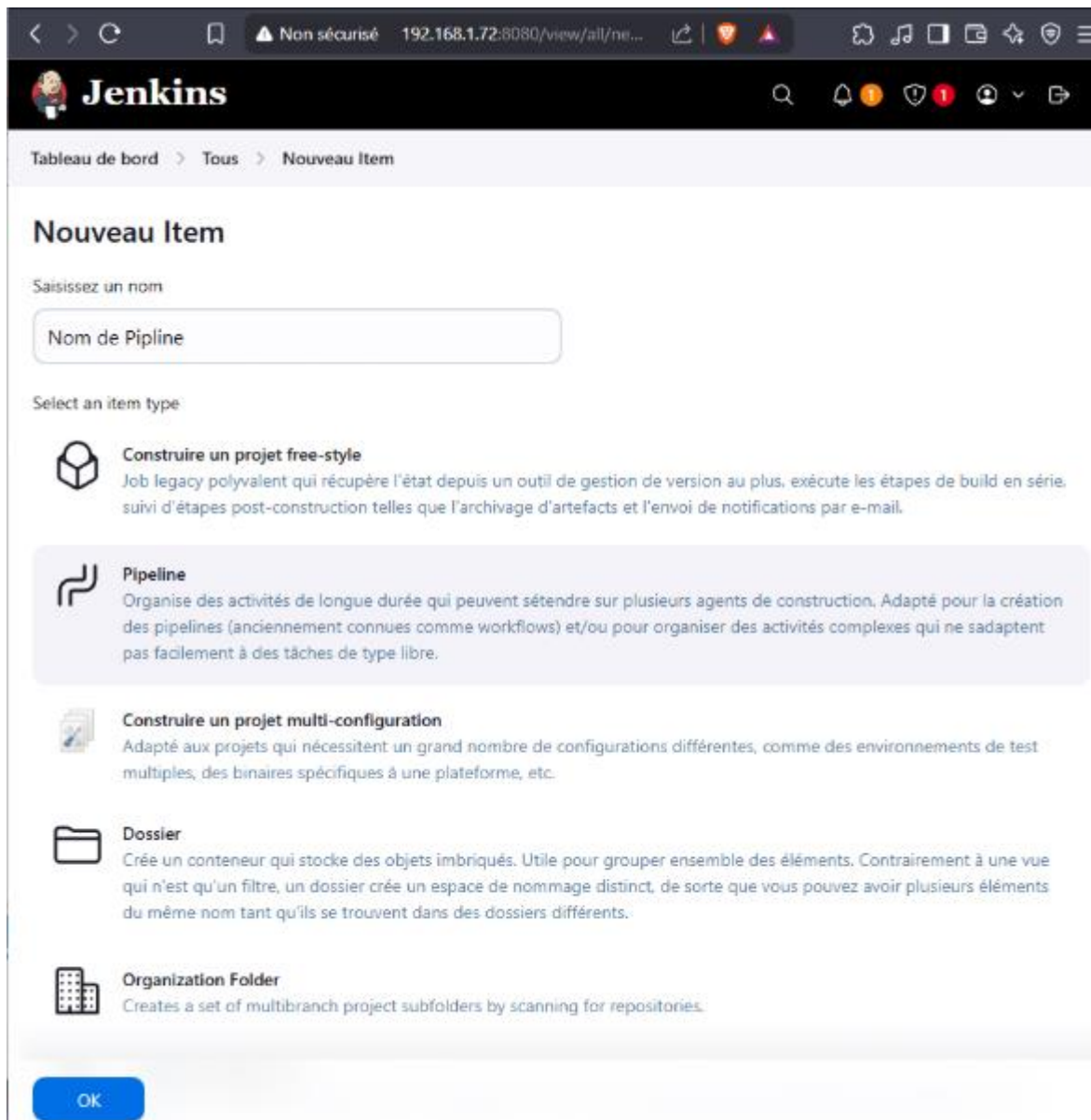


Figure 55 : Création du projet Pipeline

2) Ajout du script Pipeline

- Le script est directement intégré via l'éditeur de Jenkins (sandbox) pour centraliser la configuration et faciliter les ajustements.
- L'option *GitHub hook trigger* est activée pour déclencher automatiquement les builds lors des pushes.

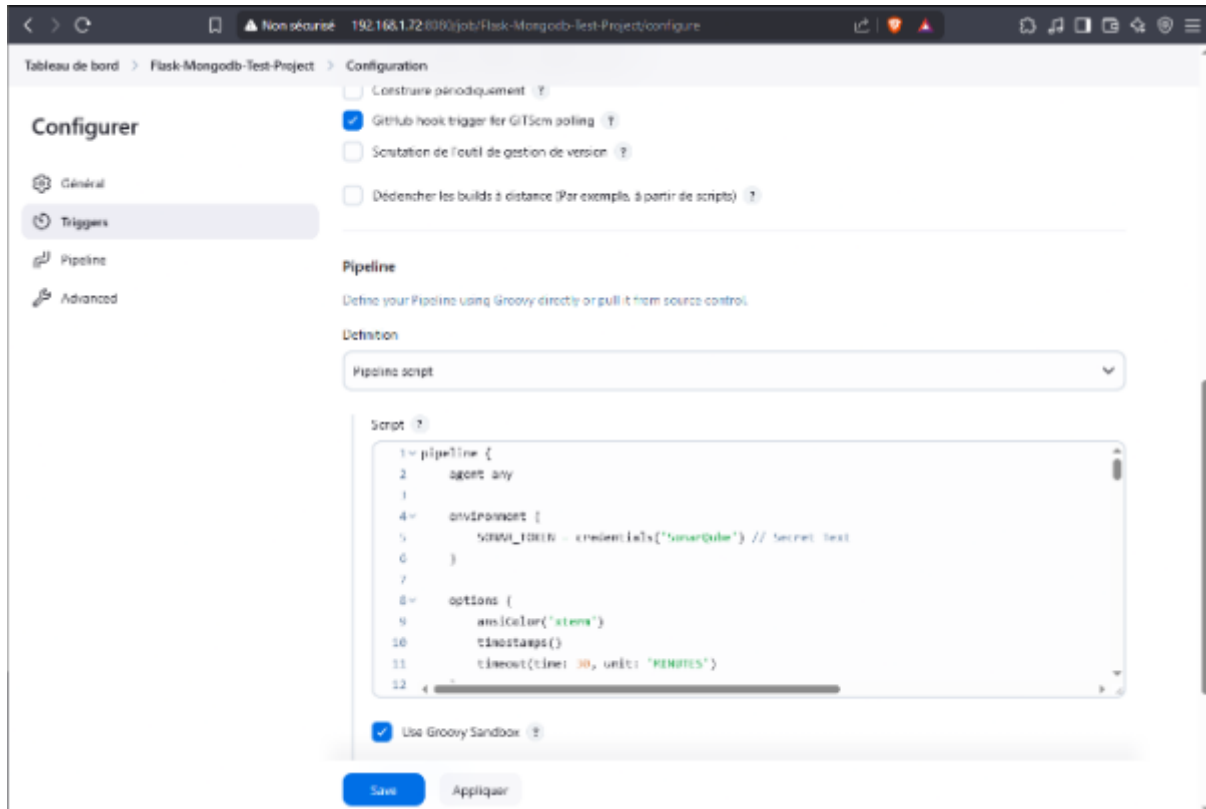


Figure 56 : Ajout du script Pipeline

Le script du pipeline :

```

pipeline { agent any
environment {
    SONAR_TOKEN = credentials('SonarQube') // Secret Text
}

options {
    ansiColor('xterm')
    timestamps()
    timeout(time: 30, unit: 'MINUTES')
}

stages {
    stage('Checkout') {
        steps {
            git branch: 'master',
            url: 'https://github.com/Tayssir1302/flask-mongodb-crud.git',
            credentialsId: 'github'
        }
    }

    stage('Build Docker Image') {
        steps {
            echo "🔧 Building Docker image..."
            sh 'docker build -t iyedhosni20/flask-mongodb-test-project:latest .'
            sh 'docker tag iyedhosni20/flask-mongodb-test-project:latest
192.168.1.71:5000/flask-mongodb-test-project:latest'
        }
    }
}
    
```

```

    }
  }

  stage('SonarQube Analysis') {
    steps {
      echo "🔍 Running SonarQube analysis..."
      sh '''
        /opt/sonar-scanner/bin/sonar-scanner \
        -Dsonar.projectKey=flask-mongodb-test-project \
        -Dsonar.sources=. \
        -Dsonar.host.url=http://192.168.1.70:9000 \
        -Dsonar.login=${SONAR_TOKEN}
      '''
    }
  }

  stage('Push to DockerHub') {
    steps {
      echo "📦 Pushing image to DockerHub..."
      script {
        withCredentials([usernamePassword(credentialsId: 'docker_hub',
usernameVariable: 'DOCKERHUB_USER', passwordVariable:
'DOCKERHUB_TOKEN')]) {
          sh '''
            echo "$DOCKERHUB_TOKEN" | docker login -u
"$DOCKERHUB_USER" --password-stdin
            docker push iyedhosni20/flask-mongodb-test-project:latest
          '''
        }
      }
    }
  }

  stage('Push to Nexus') {
    steps {
      echo "📦 Pushing image to Nexus Docker Registry..."
      script {
        withCredentials([usernamePassword(credentialsId: 'nexus_docker',
usernameVariable: 'NEXUS_USER', passwordVariable: 'NEXUS_PASS')]) {
          sh '''
            echo "$NEXUS_PASS" | docker login 192.168.1.71:5000 -u
"$NEXUS_USER" --password-stdin
            docker push 192.168.1.71:5000/flask-mongodb-test-project:latest
          '''
        }
      }
    }
  }

  stage('Deploy to Kubernetes') {
    steps {
      echo "🚀 Deploying to Kubernetes..."
    }
  }

```

```

    sh '''
        kubectl apply -f devops/deployment.yaml
        kubectl wait --for=condition=ready pod -l app=flask-mongodb-test-project --
timeout=180s

        until kubectl get pod -l app=flask-mongodb-test-project -o
jsonpath='{.items[0].status.containerStatuses[0].ready}' | grep true; do
            echo "⌚ Waiting for pod readiness..."
            sleep 10
        done

        kubectl rollout restart deployment flask-mongodb-test-project

        until kubectl get pod -l app=flask-mongodb-test-project -o
jsonpath='{.items[0].status.containerStatuses[0].ready}' | grep true; do
            echo "⌚ Waiting for rollout..."
            sleep 10
        done
    '''
}

stage('App Access Info') {
    steps {
        echo "✅ Application deployed!"
        sh '''
            NODE_PORT=$(kubectl get svc flask-mongodb-test-project-service -o
jsonpath='{.spec.ports[0].nodePort}')
            NODE_IP=$(kubectl get nodes -o
jsonpath='{.items[0].status.addresses[0].address}')
            echo "🌐 Access your app at: http://\$NODE\_IP:\$NODE\_PORT"
        '''
    }
}

post {
    failure {
        mail to: 'tayssiromri3@gmail.com',
            subject: "❌ Build Failed: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
            body: "Build failed. Check Jenkins for details."
    }
}
}

```


VII. Configuration du webhook GitHub

Le webhook GitHub est nécessaire pour déclencher automatiquement les builds lorsque qu'un développeur pousse un commit sur le dépôt GitHub.

1) Hébergement de Jenkins avec ngrok

GitHub ne peut pas communiquer avec Jenkins lorsqu'il est hébergé en réseau local. Pour le rendre accessible depuis Internet, Ngrok a été utilisé afin d'exposer Jenkins au web.

```
ngrok http 8080
```

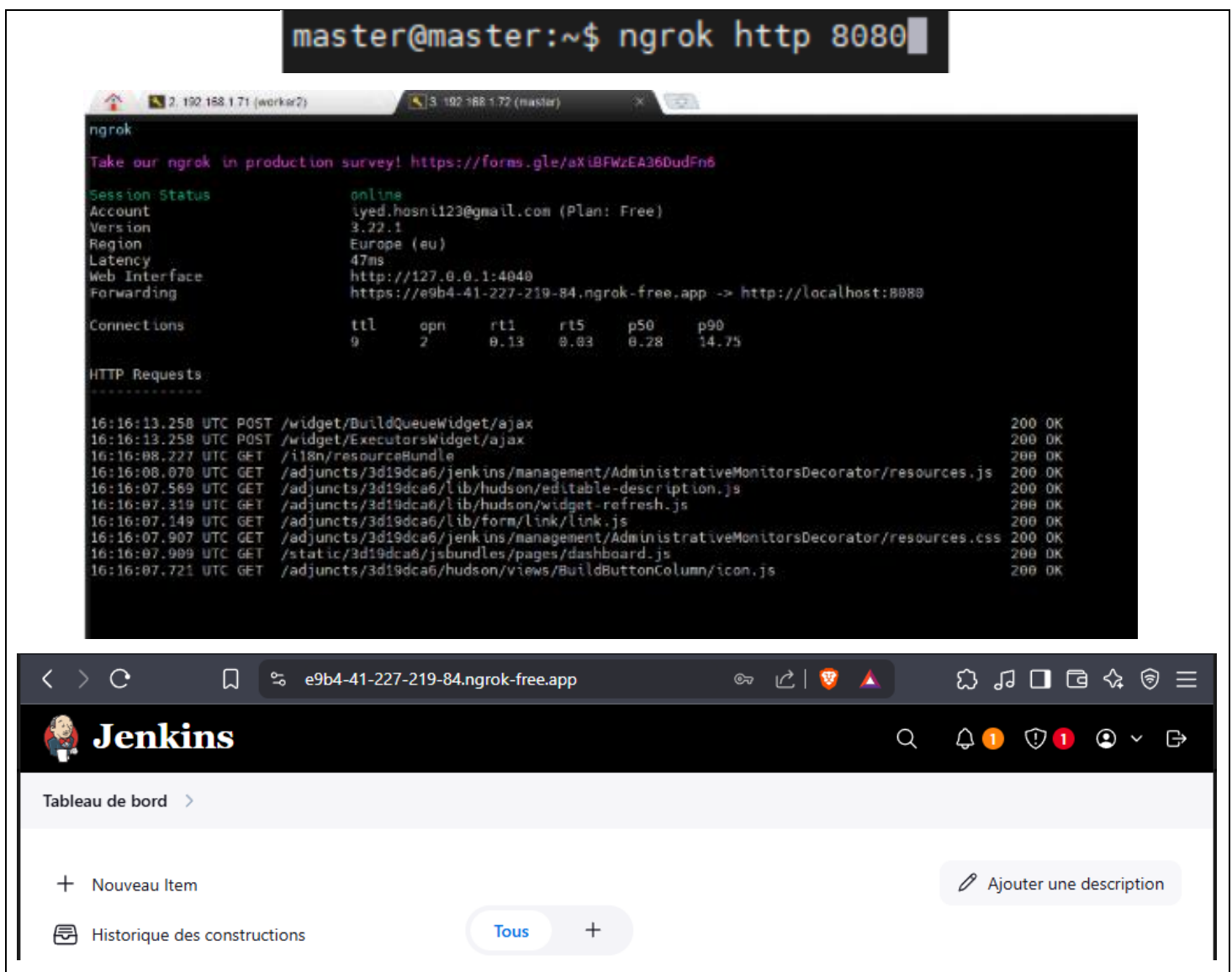


Figure 57 : Hébergement de Jenkins avec ngrok

2) Ajout du webhook GitHub

1. Accédez à **votre dépôt GitHub**, puis allez dans **Settings > Webhooks**.
2. Cliquez sur **"Add webhook"**.
3. Dans le champ **"Payload URL"**, saisissez l'URL fournie par Ngrok, suivie de `/github-webhook/`
Exemple : `https://xxxx.ngrok.io/github-webhook/`
4. Dans le champ **"Content type"**, sélectionnez **application/json** dans le menu déroulant.
Cela permet à GitHub d'envoyer les données au format JSON.
5. Dans la section **"Which events would you like to trigger this webhook?"**, sélectionnez **"Send me everything"** pour recevoir toutes les notifications d'événements.
6. Cliquez sur **"Add webhook"** pour enregistrer.

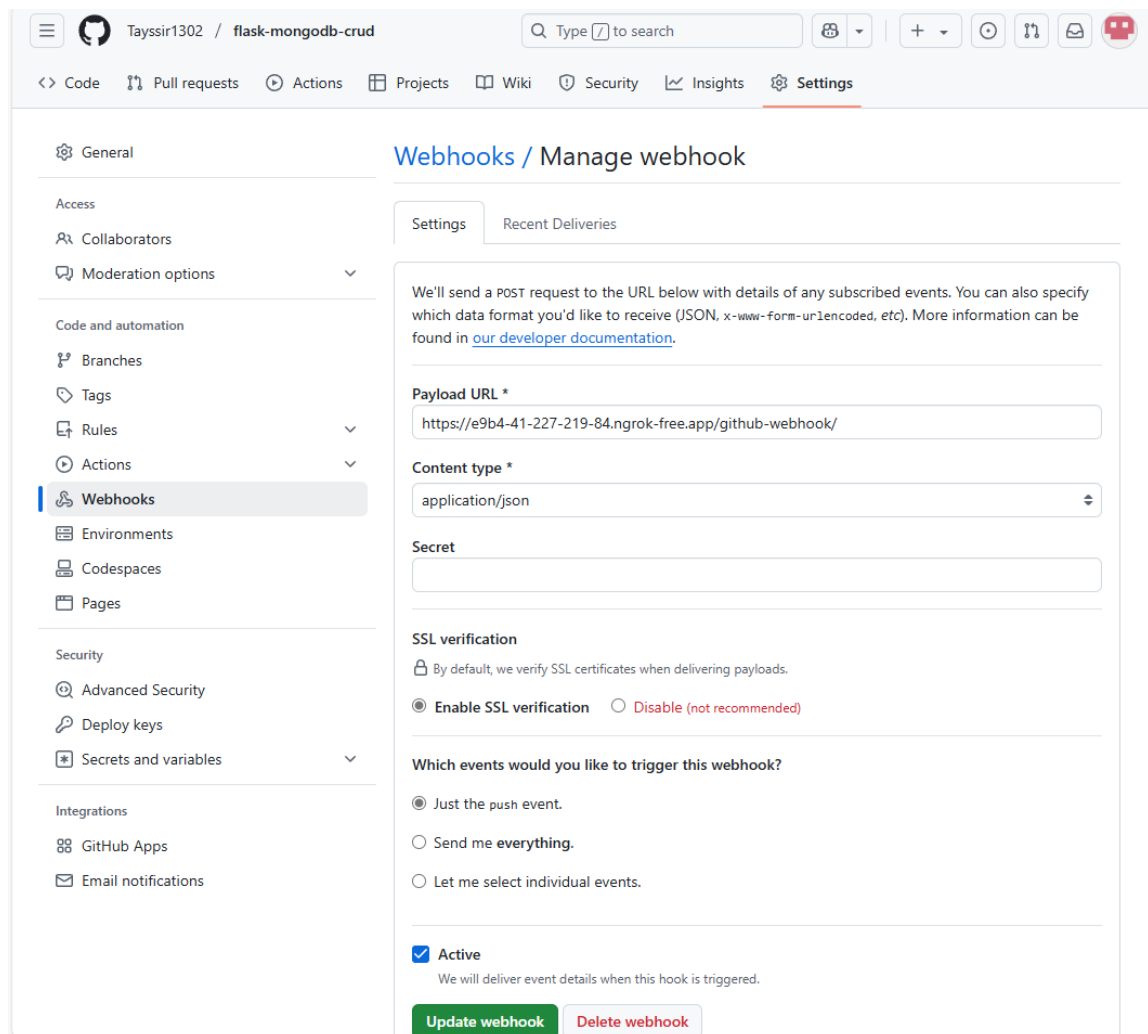


Figure 58 : Ajout du webhook GitHub

VIII. Phase de test du pipeline CI/CD

1) pousse de test avec GitHub Desktop

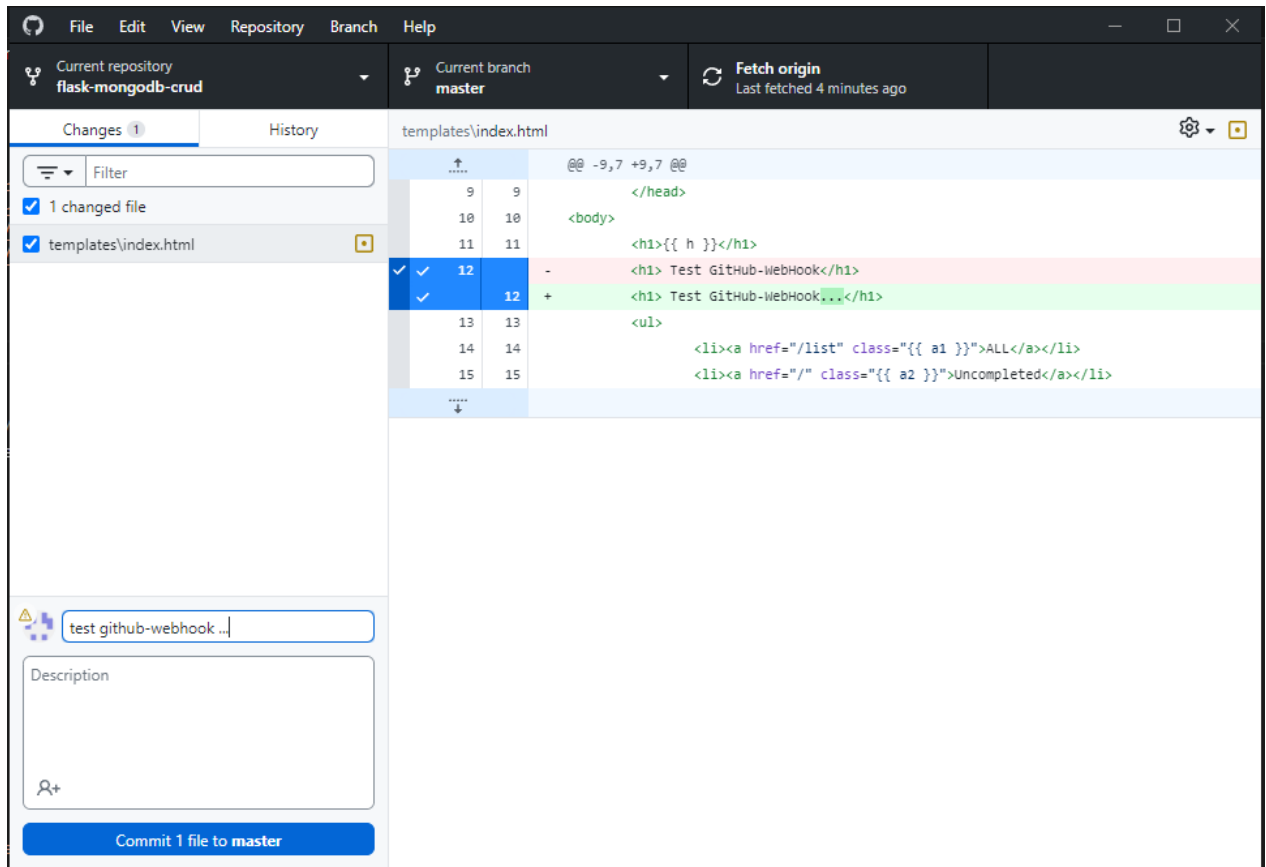


Figure 59 : pousse de test avec GitHub Desktop

2) Validation du webhook GitHub

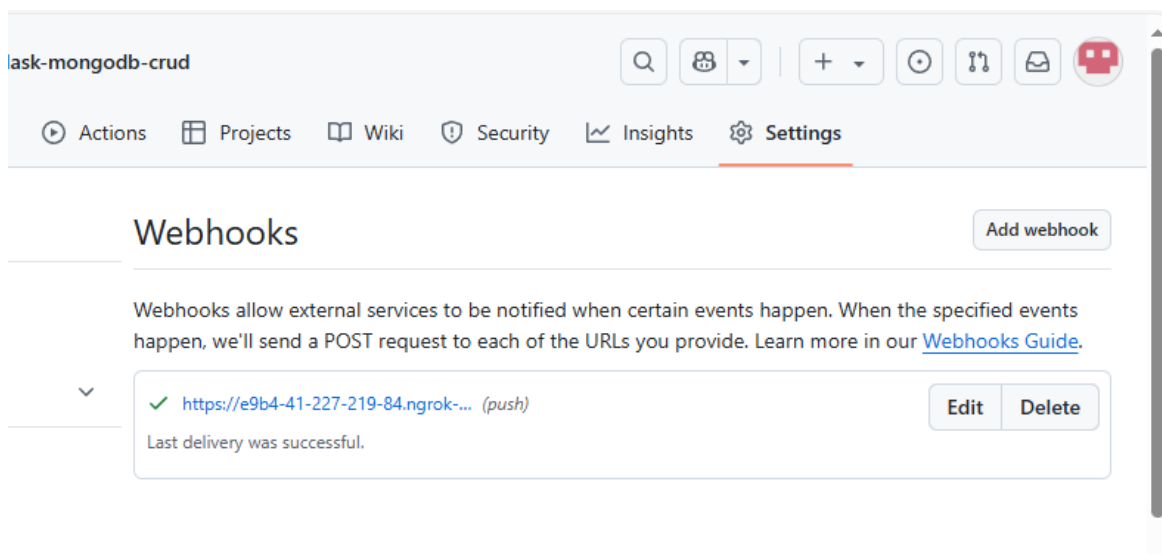
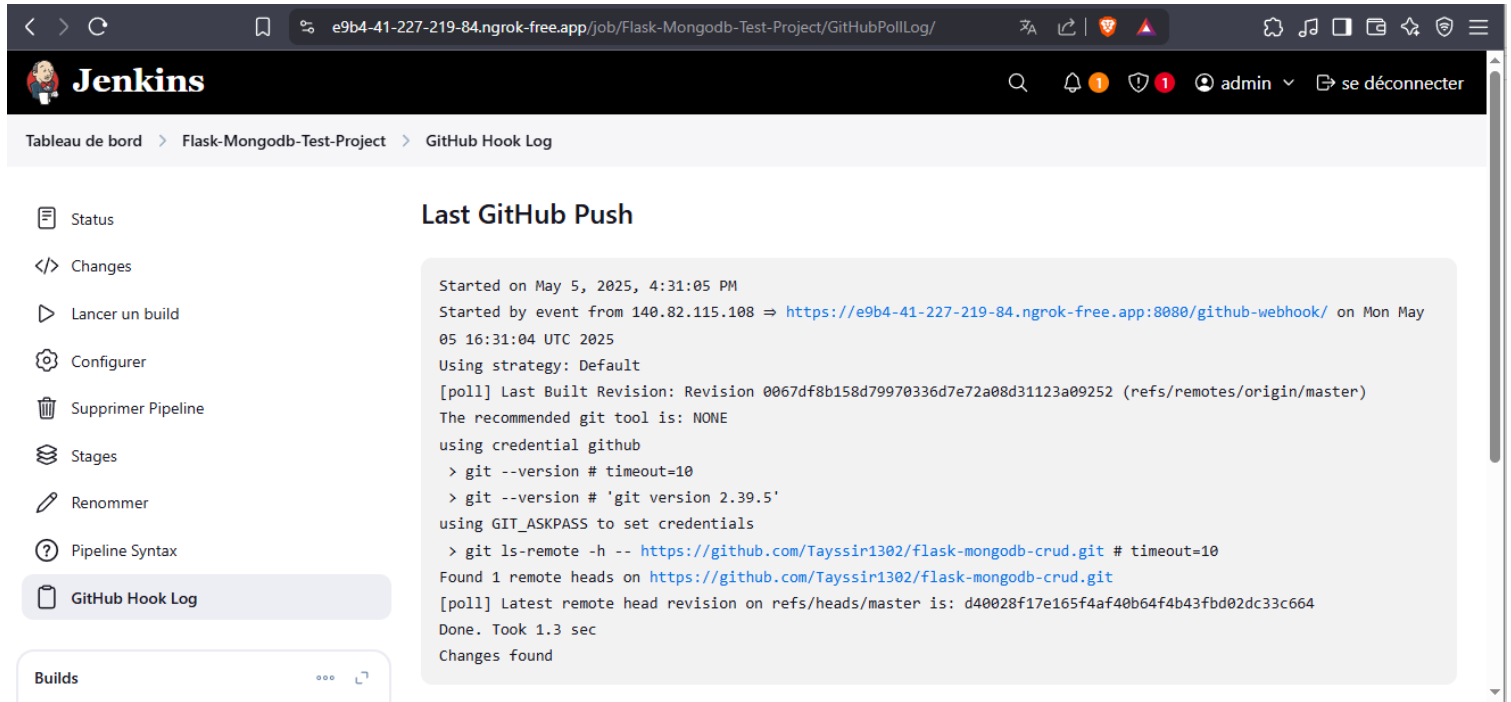


Figure 60 : Validation du webhook GitHub

3) Détection par Jenkins



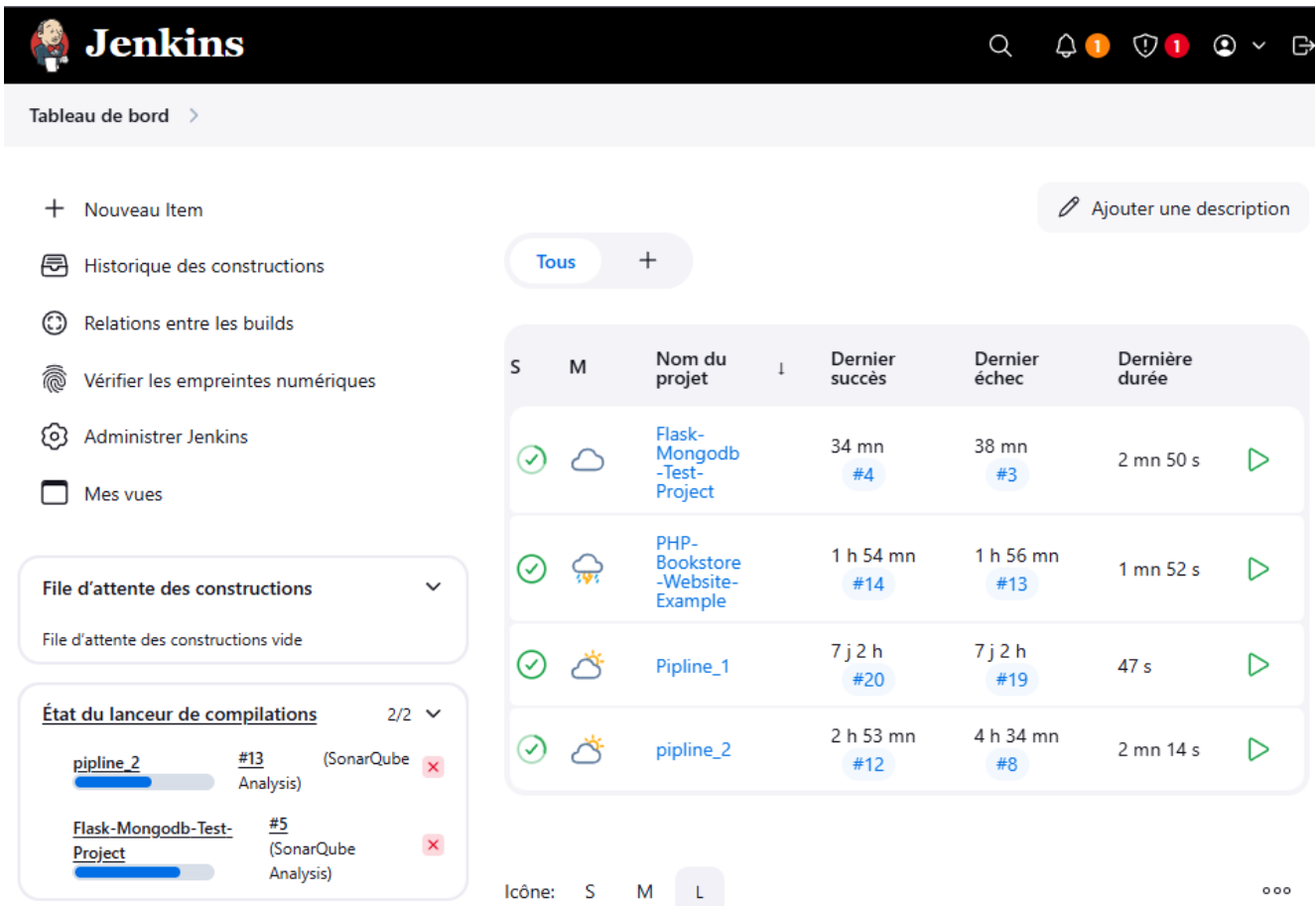
The screenshot shows the Jenkins web interface. The top navigation bar includes the Jenkins logo, a search icon, and user information (admin, se déconnecter). The breadcrumb trail is: Tableau de bord > Flask-Mongodb-Test-Project > GitHub Hook Log. The left sidebar contains a list of actions: Status, Changes, Lancer un build, Configurer, Supprimer Pipeline, Stages, Renommer, Pipeline Syntax, and GitHub Hook Log (which is highlighted). The main content area is titled 'Last GitHub Push' and displays a log of a recent push event. The log text is as follows:

```

Started on May 5, 2025, 4:31:05 PM
Started by event from 140.82.115.108 => https://e9b4-41-227-219-84.ngrok-free.app:8080/github-webhook/ on Mon May
05 16:31:04 UTC 2025
Using strategy: Default
[poll] Last Built Revision: Revision 0067df8b158d79970336d7e72a08d31123a09252 (refs/remotes/origin/master)
The recommended git tool is: NONE
using credential github
> git --version # timeout=10
> git --version # 'git version 2.39.5'
using GIT_ASKPASS to set credentials
> git ls-remote -h -- https://github.com/Tayssir1302/flask-mongodb-crud.git # timeout=10
Found 1 remote heads on https://github.com/Tayssir1302/flask-mongodb-crud.git
[poll] Latest remote head revision on refs/heads/master is: d40028f17e165f4af40b64f4b43fbd02dc33c664
Done. Took 1.3 sec
Changes found
    
```

Figure 61 : Détection par Jenkins

4) Suivi de l'exécution du build



The screenshot shows the Jenkins 'Historique des constructions' (Build History) page. The top navigation bar includes the Jenkins logo, a search icon, and user information. The breadcrumb trail is: Tableau de bord >. The left sidebar contains a list of actions: Nouveau Item, Historique des constructions, Relations entre les builds, Vérifier les empreintes numériques, Administrer Jenkins, and Mes vues. The main content area displays a table of build history. The table has columns: S (Success), M (Manual), Nom du projet, Dernier succès, Dernier échec, and Dernière durée. The table lists four builds:

S	M	Nom du projet	Dernier succès	Dernier échec	Dernière durée
✓	☁	Flask-Mongodb-Test-Project	34 mn #4	38 mn #3	2 mn 50 s
✓	☁	PHP-Bookstore-Website-Example	1 h 54 mn #14	1 h 56 mn #13	1 mn 52 s
✓	☁	Pipeline_1	7 j 2 h #20	7 j 2 h #19	47 s
✓	☁	pipeline_2	2 h 53 mn #12	4 h 34 mn #8	2 mn 14 s

Below the table, there is a section titled 'État du lanceur de compilations' (2/2) showing the status of two compilation launchers: 'pipeline_2' (SonarQube Analysis) and 'Flask-Mongodb-Test-Project' (SonarQube Analysis). Both are shown with a progress bar and a status icon (red X).

Figure 62 : Suivi de l'exécution du build

5) Résultat final du build

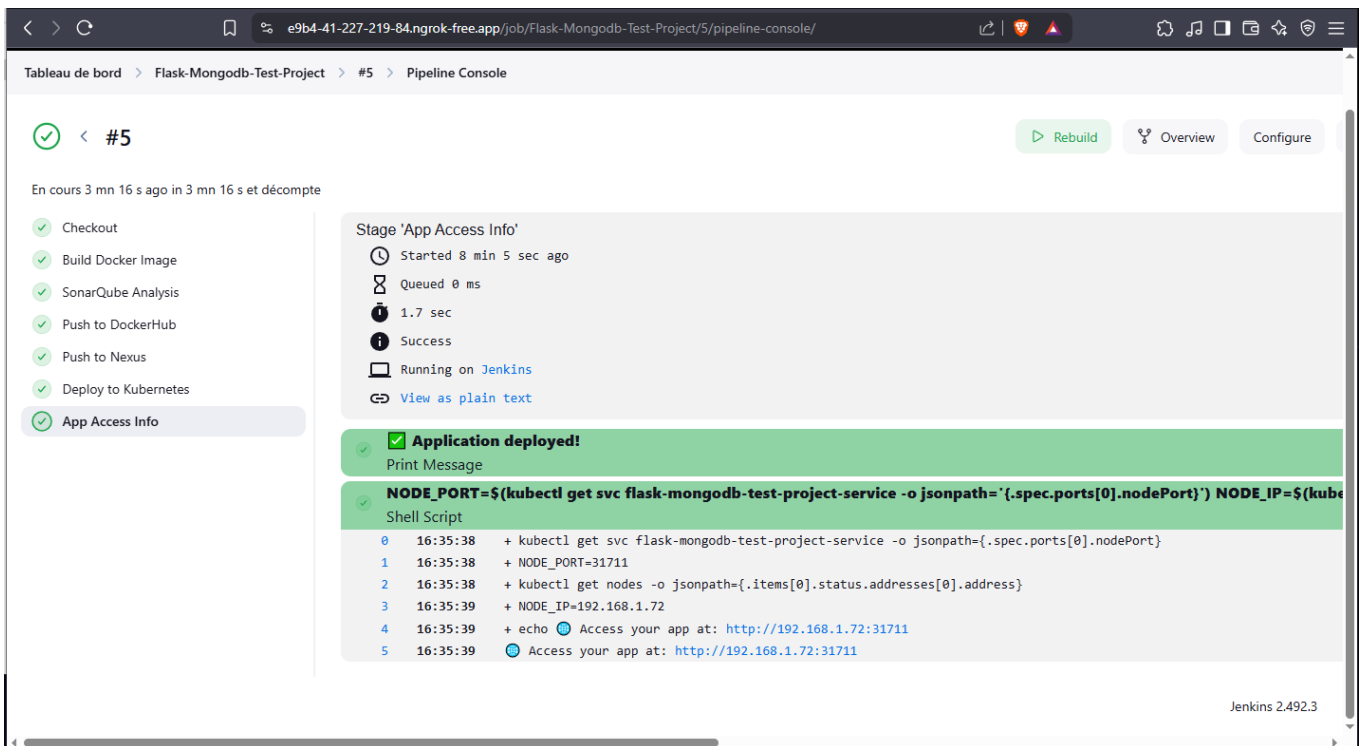


Figure 63 : Build de l'application **Flask-MongoDB** réussi

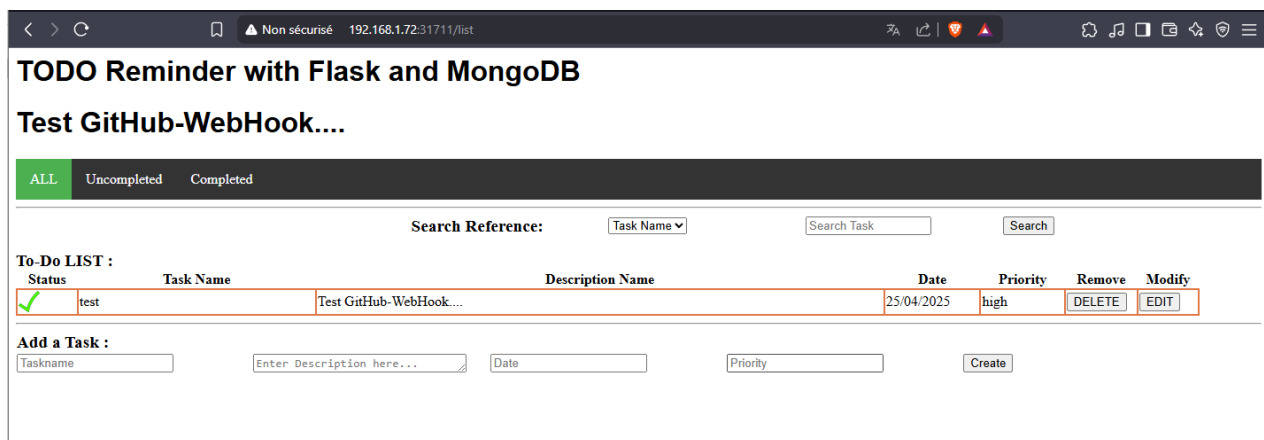


Figure 64 : Application web **Flask-MongoDB** hébergée

6) Vérifier SonarQube et Nexus

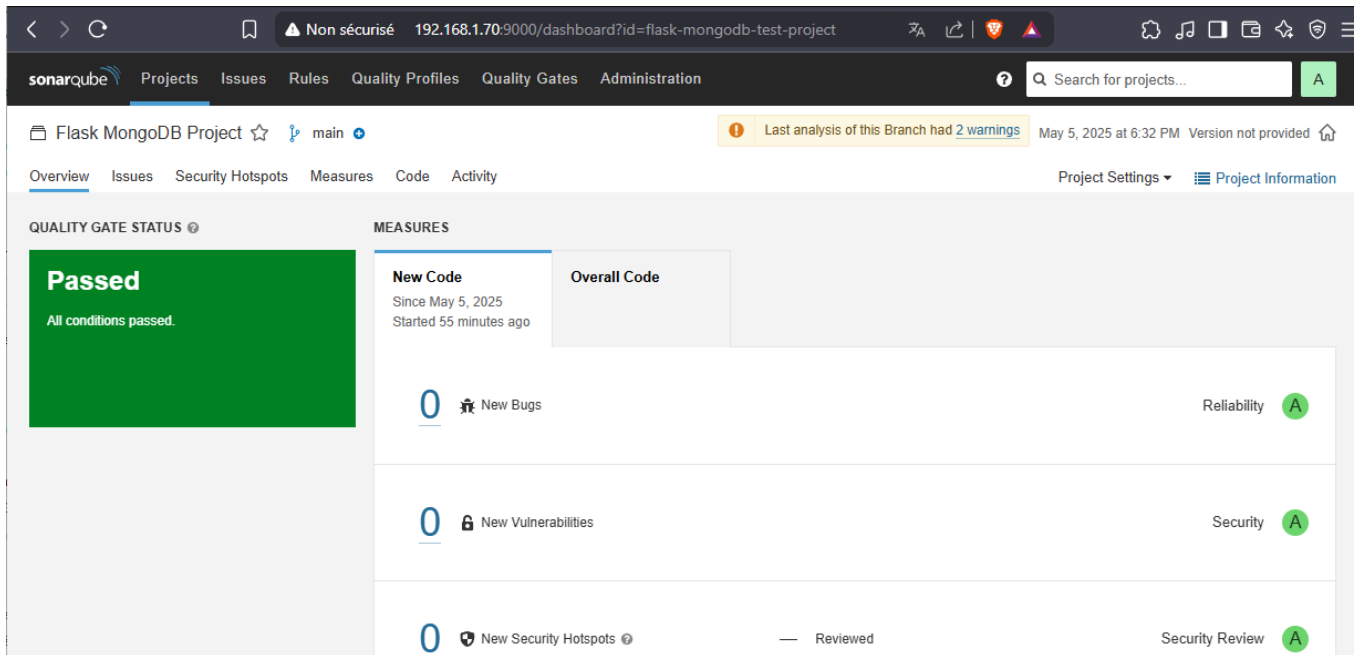


Figure 65 : Analyse réussie du projet Flask-MongoDB dans SonarQube

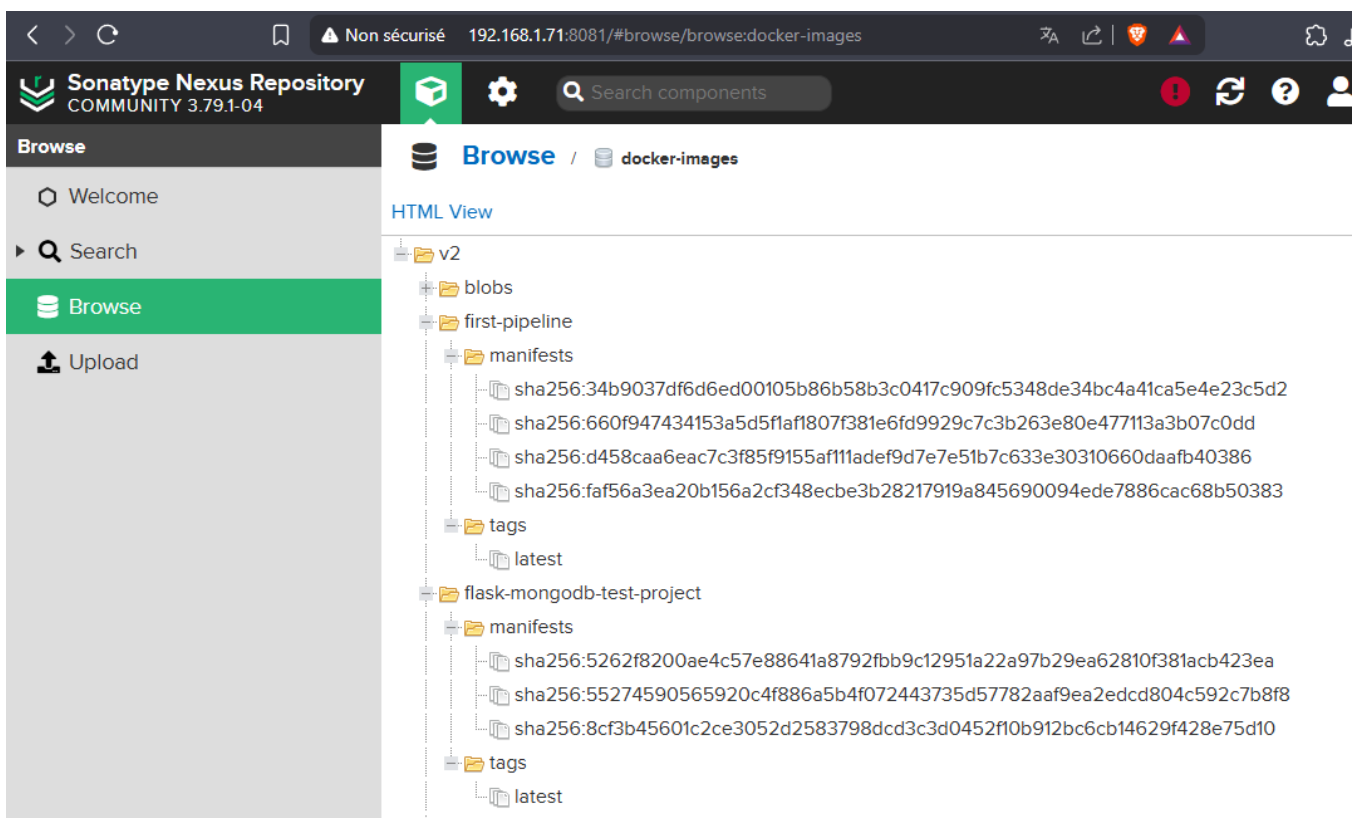


Figure 66 : Déploiement du projet Flask-MongoDB dans le repository Nexus

7) Teste pipeline avec autre application (PHP)

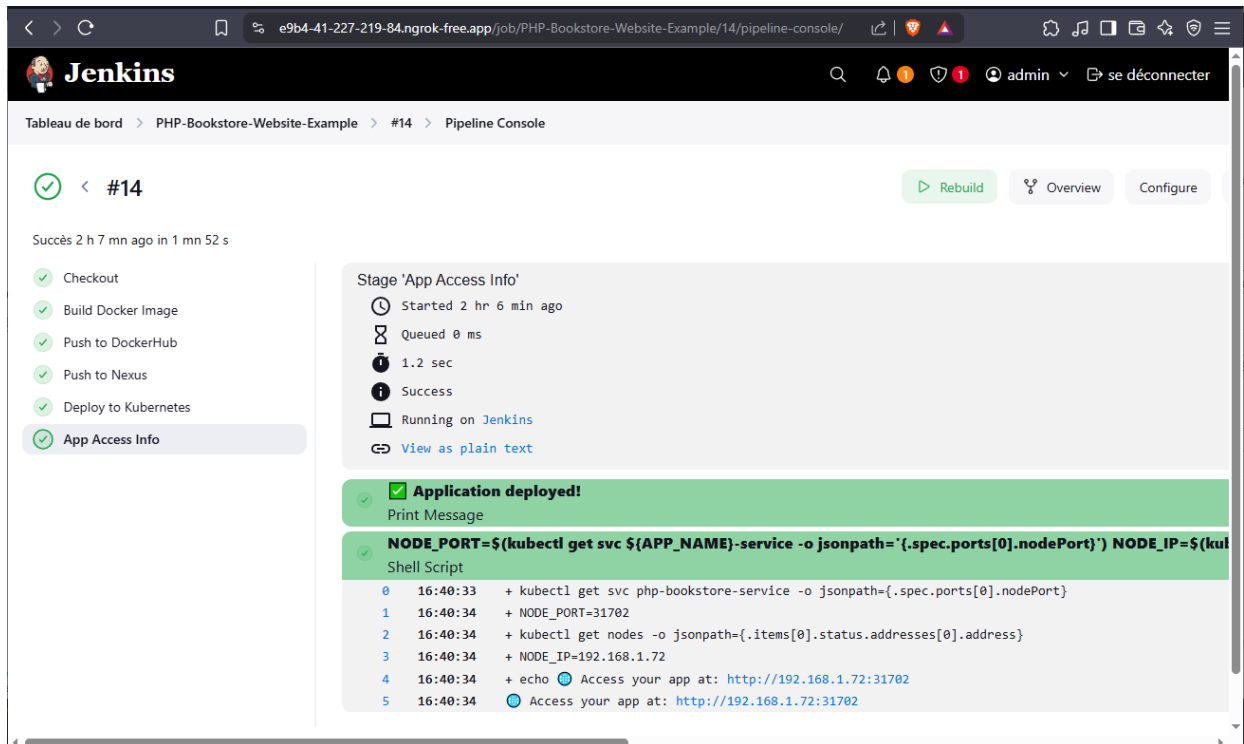


Figure 67 : Build de l'application PHP-MySQL réussi

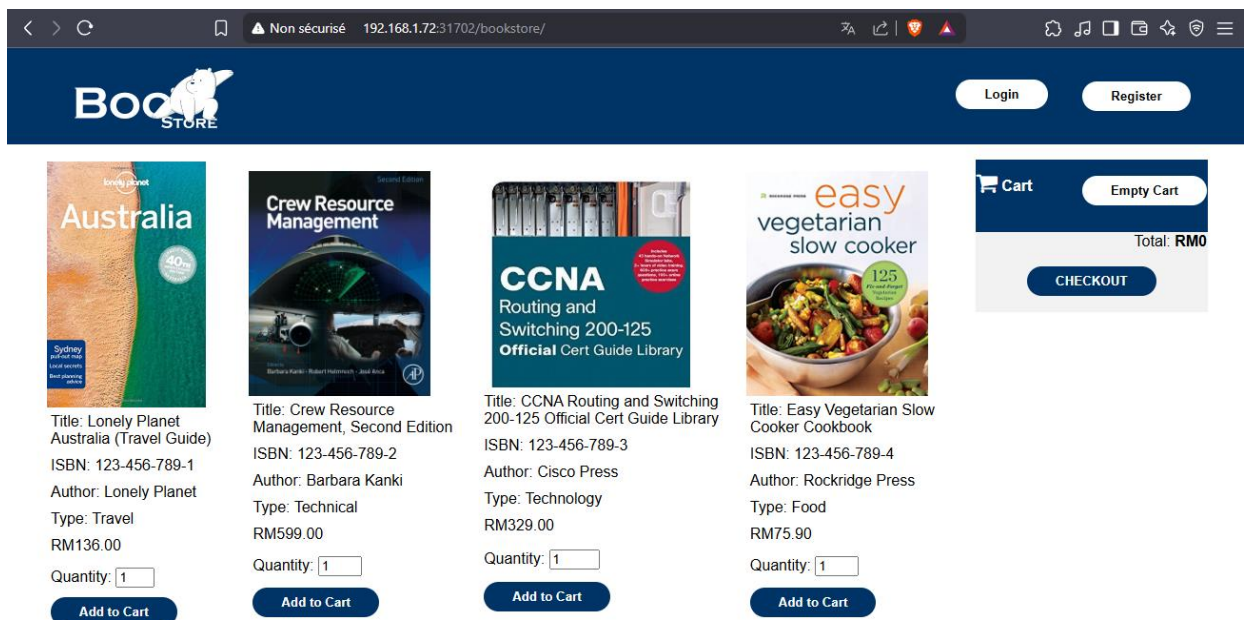


Figure 68 : Application web PHP-MySQL hébergée

8) Vérifier les pods de kubernetes

kubectl get pods


```

master@master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
first-pipeline-7c845d6f67-jxg64    1/1     Running   0           59m
flask-mongodb-test-project-74d79db6b7-xp7cd  1/1     Running   0           59m
mongodb-86f4474bb7-6zssc           1/1     Running   0           7d7h
mysql-6896f5bd4b-p9kc6             1/1     Running   0           4h
php-bookstore-58d558f964-gkn87      1/1     Running   0           174m
master@master:~$

```

Figure 69 : Les pods de kubernetes

9) Test de l'alerte email

Pour vérifier les notifications par e-mail que nous avons configurées précédemment sur Jenkins, nous avons dû provoquer volontairement l'échec d'un pipeline, puis le corriger par la suite. En effet, Jenkins n'envoie une notification que lorsqu'il y a un changement d'état, c'est-à-dire d'un succès à un échec ou d'un échec à un succès.

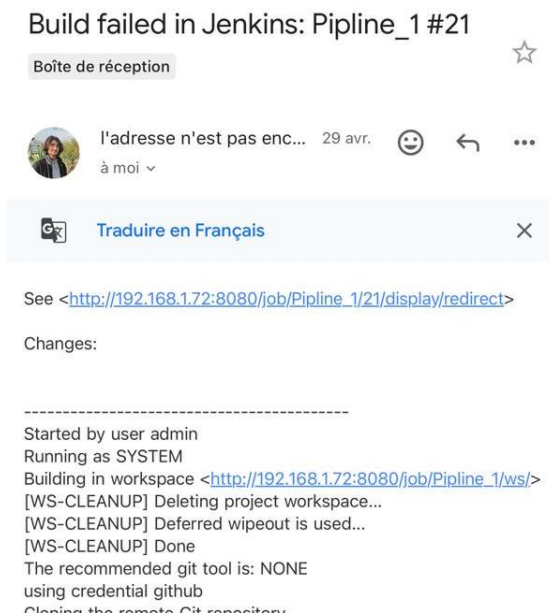


Figure 70 : Notification d'échec du pipeline Jenkins



Figure 71 : Notification de reprise normale du pipeline Jenkins

Conclusion

Ce sprint a permis de créer une pipeline CI/CD stable, couvrant les étapes clés : build, analyse, livraison, et déploiement. L’intégration entre Jenkins, Docker, SonarQube, Nexus et Kubernetes s’est avérée efficace.

Dans un pipeline réel, il est courant d’ajouter des tests unitaires, exécutés automatiquement dans le script Jenkins. Toutefois, comme nous n’avons pas développé ces applications, nous ne disposons pas des tests nécessaires. Ce travail reste donc un test d’intégration technique.

Les sprints 3 et 4 porteront sur une plateforme centralisée pour visualiser tous les tableaux de bord (Jenkins, SonarQube, etc.) et gérer les pipelines depuis une seule interface.

Chapitre 6 : Sprint 3 - Développement Backend / API & Notification

Introduction

Ce chapitre présente la mise en place du backend de la plateforme, qui permet de centraliser les outils, tableaux de bord et alertes utilisés par l'équipe 3S dans un seul espace sécurisé et facile à utiliser. Développé en Node.js, ce backend gère les API REST, les connexions aux services internes et externes, ainsi que l'envoi de notifications en temps réel. L'objectif est de garantir une communication fluide et sécurisée entre les composants de la plateforme, tout en assurant une réactivité optimale en cas d'incidents ou d'événements critiques.

I. L'architecture du Backend

Dans le cadre de la modélisation du backend, deux types de diagrammes complémentaires ont été utilisés afin d'illustrer à la fois la structure de l'application et son modèle de données :

- **Le diagramme d'architecture modulaire:** présenté ci-dessous, reflète fidèlement l'organisation du projet Node.js. Ce backend repose sur une approche modulaire et fonctionnelle, typique de l'écosystème JavaScript, dans laquelle chaque fonctionnalité est encapsulée dans des modules spécifiques (routes/, controllers/, services/, middleware/, socket/, etc.). Cette architecture vise à assurer la lisibilité, la scalabilité et la maintenabilité du code. N'étant pas fondée sur la programmation orientée objet, cette structure ne se prête pas naturellement à une représentation UML classique par diagramme de classes. Le diagramme modulaire met donc plutôt en avant la séparation des responsabilités et les flux entre les différents composants techniques.
- **Le diagramme de classes:** Il représente uniquement les entités persistées localement dans la base (telles qu'User, Notification, ou GitHubRepository), qui correspondent directement aux fichiers définis dans le dossier **/models** de l'application. Ces entités sont les seules à disposer d'un schéma structuré en base de données, contrairement aux données issues des services externes (Jenkins, SonarQube, GitHub, etc.) qui sont utilisées de manière dynamique. Ce diagramme permet donc de visualiser la structure des données internes à la plateforme, leurs attributs et les relations entre elles.

Il est important de noter que la plateforme a été pensée pour être utilisée **par une équipe complète travaillant sur les mêmes projets**. Ainsi, les utilisateurs ayant un **rôle identique** (ex. : développeur, DevOps, superviseur...) accèdent **aux mêmes interfaces, jeux de données et notifications**. Ce fonctionnement basé sur les rôles permet une collaboration fluide, une cohérence des informations affichées et une gestion centralisée des permissions.

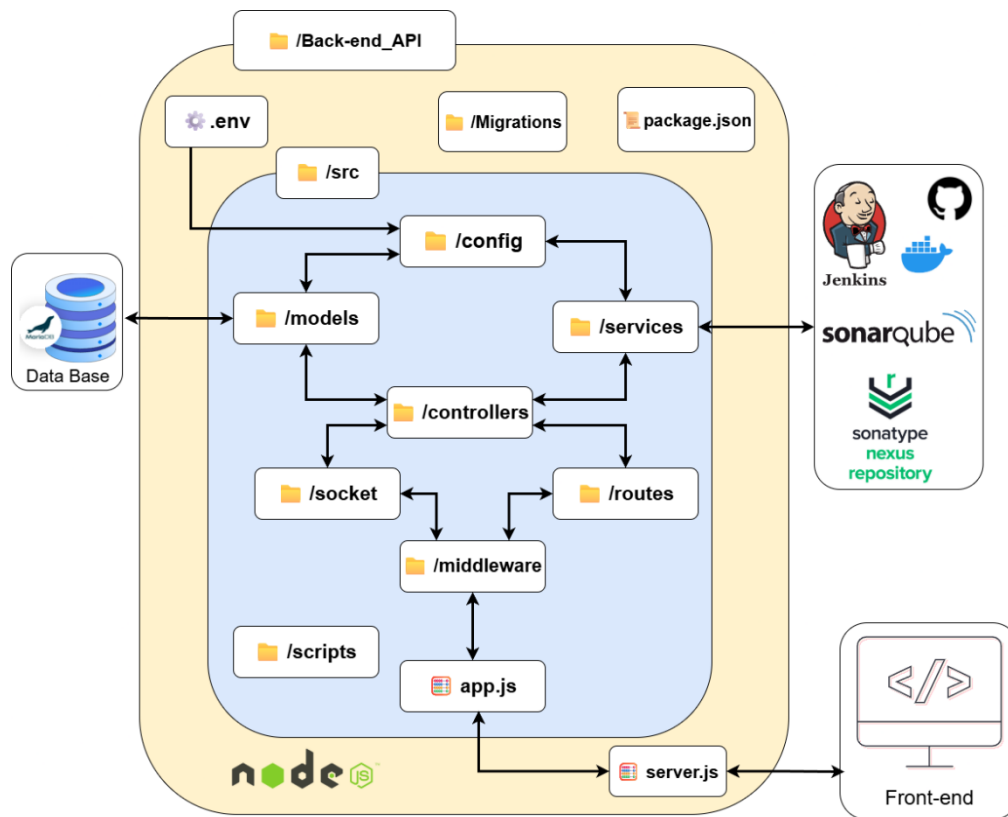


Figure 72 : diagramme modulaire

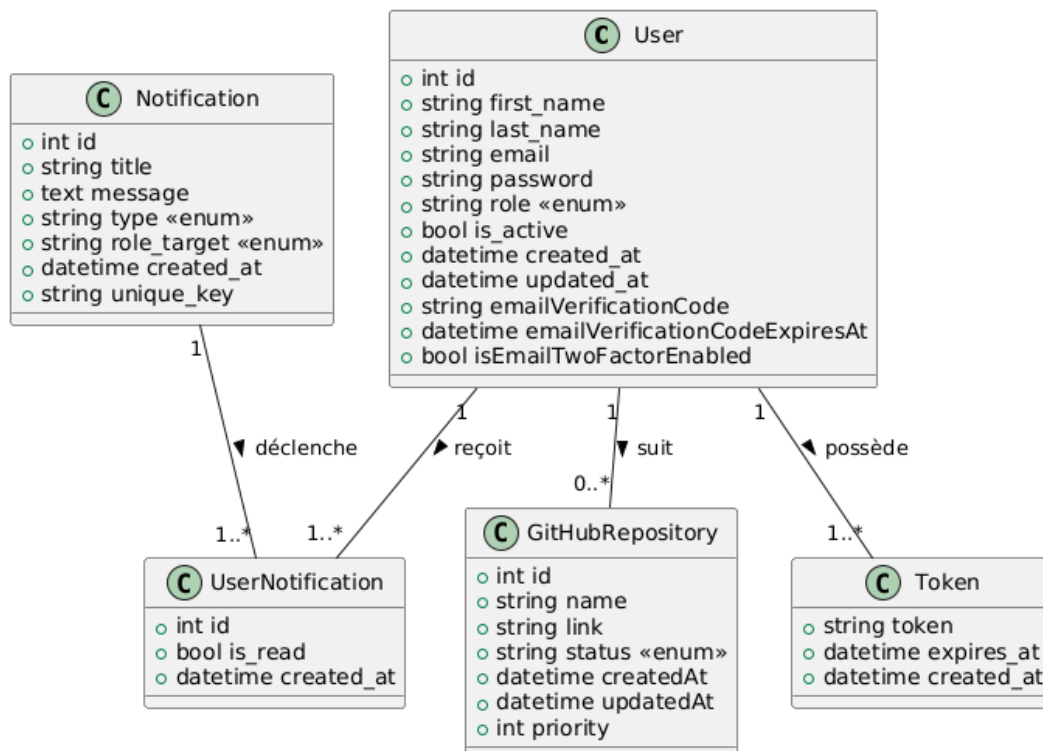


Figure 73 : diagramme de classes

II. Structure technique du backend

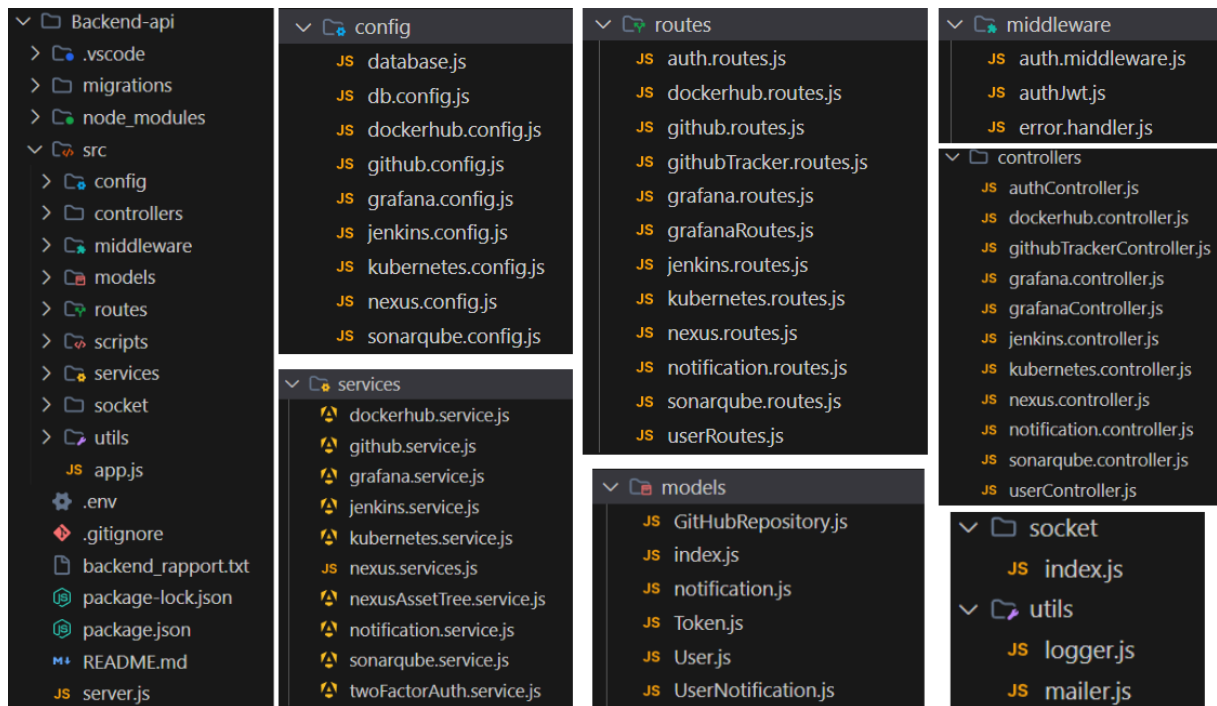


Figure 74 : Structure technique du backend

Le fichier **server.js** constitue le point d'entrée de l'application. Il initialise le serveur HTTP et WebSocket. Toute la logique applicative est centralisée dans le dossier **src/**, organisé en sous-dossiers selon des responsabilités bien définies :

- **app.js** : configure les middlewares globaux, le parsing des requêtes, la gestion des erreurs et le chargement des routes.
- **routes/** : contient les définitions des endpoints REST, chacun redirigeant vers un contrôleur correspondant.
- **controllers/** : reçoit les requêtes du frontend et délègue la logique métier aux services.
- **services/** : regroupe la logique métier, y compris les appels aux API externes et le traitement des données.
- **models/** : contient les schémas de données Sequelize liés à MariaDB (par ex. : User, Notification, GitHubRepository).
- **middleware/** : gère l'authentification JWT, les validations ou la gestion personnalisée des erreurs.
- **socket/** : implémente la logique Socket.io, assurant la diffusion de notifications en temps réel.
- **config/** : centralise les clés API, accès BDD, secrets JWT, et autres paramètres de configuration issus du fichier **.env**

- **utils/** : propose des fonctions utilitaires réutilisables
- **scripts/** : inclut des scripts d'automatisation, comme la création du compte administrateur à la première exécution.

Ce découpage modulaire garantit une organisation claire, facilite le travail en équipe, et permet l'ajout de nouvelles fonctionnalités sans compromettre l'architecture existante.

III. Gestion des API : Flux des Données

Le backend est conçu pour interagir avec deux types d'API : les API externes (comme celles de Jenkins, SonarQube, GitHub, etc.) et les API internes de la plateforme qui gèrent les données propres à notre application (utilisateurs, projets, notifications).

1) Flux des API Externes

Pour les API externes, le flux de données suit une séquence logique pour garantir la sécurité des informations sensibles et la modularité du code :

- **.env** : contient les variables sensibles (clés API, URLs) propres à chaque environnement.
- **config/** : centralise les variables sensibles (clés API, URLs), chargées depuis **.env** et exposées de manière sécurisée.
- **services/** : contiennent la logique métier liée aux API externes (ex. : déclenchement d'un build Jenkins), ainsi que la gestion des erreurs.
- **controllers/** : reçoivent les requêtes du frontend et délèguent les appels aux services concernés.
- **routes/** : définissent les endpoints REST et dirigent les requêtes vers les bons contrôleurs.

Ce flux garantit que les interactions avec les services externes sont centralisées, sécurisées et facilement gérables, tout en maintenant une séparation claire des préoccupations.

A) Exemple de Parcours d'une API Externe

Cet exemple illustre l'appel à une API externe (déclenchement d'un build Jenkins)



Figure 75 : Parcours d'une API Externe (déclenchement d'un build Jenkins)

2) Flux des API Internes

Les API internes gèrent les données spécifiques à notre plateforme, telles que les informations sur les utilisateurs, les projets gérés, et les notifications générées. Le flux pour ces API est optimisé pour l'interaction avec la base de données :

- **.env** : contient les informations de connexion à la base (URL, identifiants), sécurisées et propres à chaque environnement.
- **config/** : charge ces variables et les rend accessibles aux modules, y compris l'ORM Sequelize.
- **models/** : définissent les schémas de données (ex. : User) et gèrent les opérations CRUD.
- **controllers/** : reçoivent les requêtes du frontend et interagissent avec les modèles pour manipuler les données.
- **routes/** : exposent les endpoints REST et redirigent les appels vers les bons contrôleurs.

Ce flux assure une gestion efficace et structurée des données de la plateforme, en tirant parti des capacités des modèles pour interagir avec la base de données.

Exemple de Parcours d'une API Interne

Cet exemple montre un parcours classique d'API interne (création d'un utilisateur)



Figure 76 : Parcours d'une API Interne (création d'un utilisateur)

III. Système de Notification en Temps Réel

Le système de notification est une composante essentielle de la plateforme, permettant d'informer les utilisateurs en temps réel des événements critiques (succès/échec de pipelines, alertes de monitoring, etc.).

1) Architecture des Notifications

Nous utilisons Socket.io, une bibliothèque populaire pour la communication bidirectionnelle en temps réel basée sur les WebSockets.

socket/: Ce dossier contient la logique de Socket.io. Il gère les connexions WebSocket, l'émission d'événements aux clients connectés et la gestion des salles (rooms) pour des notifications ciblées (par exemple, notifications spécifiques à un projet ou à un utilisateur).

Intégration avec les Services: Les services backend (dans services/) sont chargés de déclencher les notifications. Par exemple, après qu'un pipeline Jenkins ait terminé son exécution (succès ou échec), le service responsable de la gestion des pipelines informera le module socket/ pour émettre une notification en temps réel aux utilisateurs concernés.

Persistance des Notifications: En plus des notifications en temps réel, toutes les alertes et événements importants sont persistés dans la base de données via le modèle Notification (situé dans models/). Cela permet aux utilisateurs de consulter un historique de leurs notifications, même s'ils n'étaient pas connectés au moment de l'événement.

2. Canaux de Notification

La plateforme prend en charge plusieurs canaux de notification pour s'adapter aux préférences des utilisateurs :

Notification par Email : Pour les alertes critiques ou les résumés quotidiens, le backend utilise un service d'envoi d'emails (via SMTP configuré dans config/). Le module services/ contient la logique pour formater et envoyer ces emails.

Notifications In-App (via WebSockets) : Les notifications en temps réel sont affichées directement dans l'interface utilisateur de la plateforme via les WebSockets. Cela assure une réactivité immédiate aux événements.

Extensions Futures : L'architecture modulaire permet d'ajouter facilement d'autres canaux de notification à l'avenir, tels que les SMS, les intégrations Slack ou Microsoft Teams, en ajoutant simplement de nouveaux services et en les intégrant au système de déclenchement des notifications.

Exemple de de Parcours Notification Temps Réel

Voici un exemple de déclenchement d'une notification suite à l'ajout d'un projet GitHub :

notification.service.js :

```
Backend-api > src > services > notification.service.js > NotificationService > create > Explain Refactor Add Docstring
10 }
11 class NotificationService {
12   static async create({ title, message, type = 'other', role_target = 'admin' }) {
13     if (!models) {
14       throw new Error('Models not initialized. Call setSequelize() first.');

```

githubTrackerController.js :

```
Backend-api > src > controllers > githubTrackerController.js > ...
1  const { GitHubRepository } = require('../models');
2  const githubService = require('../services/github.service');
3  const { NotificationService } = require('../services/notification.service');
4  const { Op } = require('sequelize');
5
6  exports.addRepository = async (req, res) => {
7    try {
8      const { name, link, status } = req.body;
9      const repository = await GitHubRepository.create({ name, link, status });
10
11      // Send notification
12      await NotificationService.create({
13        title: 'New Repository Added',
14        message: `Repository "${name}" has been added to tracking`,
15        type: 'github',
16        role_target: 'developpeur'
17      });
18
19      res.status(201).json(repository);
20    } catch (err) {
21      console.error('Error adding repository:', err);
22      res.status(500).json({ error: 'Failed to add repository' });
23    }
24  };
25 }
```

Figure 77 : Parcours Notification Temps Réel (ajout d'un projet GitHub)emple

IV. Sécurité et Gestion des Accès

L'architecture backend de cette plateforme s'appuie sur des pratiques adoptées à grande échelle par des entreprises telles que **Netflix**, **Airbnb**, **Uber**, **LinkedIn**, **PayPal** ou **Trello**, qui utilisent également des architectures **modulaires Node.js**, des **API REST**, des **ORM comme Sequelize**, et des communications temps réel via **Socket.io**. Ce type d'architecture permet de répondre à des exigences de performance, de scalabilité et de sécurité tout en facilitant la maintenance du code.

1) Authentification et Autorisation (JWT + RBAC)

La sécurisation des accès repose sur un système d'authentification via **JSON Web Tokens (JWT)**. Lorsqu'un utilisateur se connecte, un token signé est généré et renvoyé au client, qui l'utilise ensuite dans chaque requête HTTP pour s'authentifier. Ce jeton contient les informations d'identité et de rôle, et est vérifié à chaque appel grâce aux **middlewares d'authentification personnalisés** définis dans `middleware/`.

La logique de **contrôle d'accès basé sur les rôles (RBAC)** permet de restreindre certaines routes à des utilisateurs spécifiques. Par exemple, les endpoints d'administration ou de monitoring sont uniquement accessibles aux utilisateurs de rôle admin ou superviseur. Les secrets JWT sont centralisés dans le module `config/`, et chargés depuis les variables d'environnement.

2) Authentification à Deux Facteurs (2FA)

Pour les comptes disposant d'accès sensibles, une **authentification à deux facteurs (2FA)** vient renforcer la sécurité. Seule la méthode **OTP envoyé par email** est actuellement mise en place pour l'authentification à deux facteurs. Cette solution, simple à déployer et efficace, permet de vérifier l'identité de l'utilisateur en envoyant un code à usage unique sur son adresse email enregistrée.

Ce second facteur est exigé à chaque connexion tant qu'il est activé dans les paramètres de l'utilisateur. Le code à usage unique envoyé par email est valable pendant 10 minutes. L'utilisateur peut désactiver cette option depuis l'interface de gestion de son compte s'il le souhaite.

3) Gestion des Comptes Administrateurs

Un script présent dans `scripts/` permet la **création automatique d'un compte administrateur** à la première exécution du backend. Cela garantit une prise en main rapide, sécurisée et prévisible lors de déploiements sur de nouveaux environnements.

4) Journalisation et Suivi des Activités

Un mécanisme de **journalisation structurée** permet de tracer tous les événements importants :

- Connexions, déconnexions,
- Erreurs API,
- Accès non autorisés,
- Modifications de données sensibles,
- Échecs ou réussites de pipelines DevOps.

Les logs sont générés au **format JSON** avec des niveaux (DEBUG, INFO, ERROR, CRITICAL). Les fonctions de log sont regroupées dans **utils/** avec possibilité de rotation automatique et d'anonymisation des données sensibles.

Ce dispositif assure une traçabilité complète, un audit facilité, et une détection rapide d'anomalies ou de comportements suspects.

V. Diagramme de Séquence – Parcours complet du backend

Le diagramme de séquence ci-dessous illustre un parcours typique au sein du backend de la plateforme, depuis l'interface utilisateur jusqu'aux différents composants internes et services externes.

Il couvre plusieurs scénarios clés :

- **L'authentification utilisateur**, où le frontend envoie une requête POST `/api/auth/login`. Cette requête passe successivement par la couche route, le contrôleur, puis le service d'authentification qui vérifie les identifiants via le modèle User, génère un JWT, enregistre l'événement via le module de log, et retourne un token au client.
- **La création d'un utilisateur** via un appel API interne POST `/api/users`, qui suit le même chemin technique jusqu'à l'écriture en base via Sequelize, avec enregistrement de l'action.
- **Un appel API externe vers Jenkins** déclenché depuis POST `/api/jenkins/build`, où le service envoie une requête HTTP vers l'API Jenkins, journalise l'action, et envoie une notification temps réel via Socket.io.

- Enfin, le diagramme montre comment une **notification en temps réel** est persistée en base et transmise à l'interface utilisateur via un canal WebSocket.

Ce diagramme met en évidence la cohérence de l'architecture modulaire Node.js, la séparation des responsabilités, ainsi que l'intégration fluide des services externes et du système de notification. Il reflète également la structuration technique adoptée dans des projets industriels modernes.

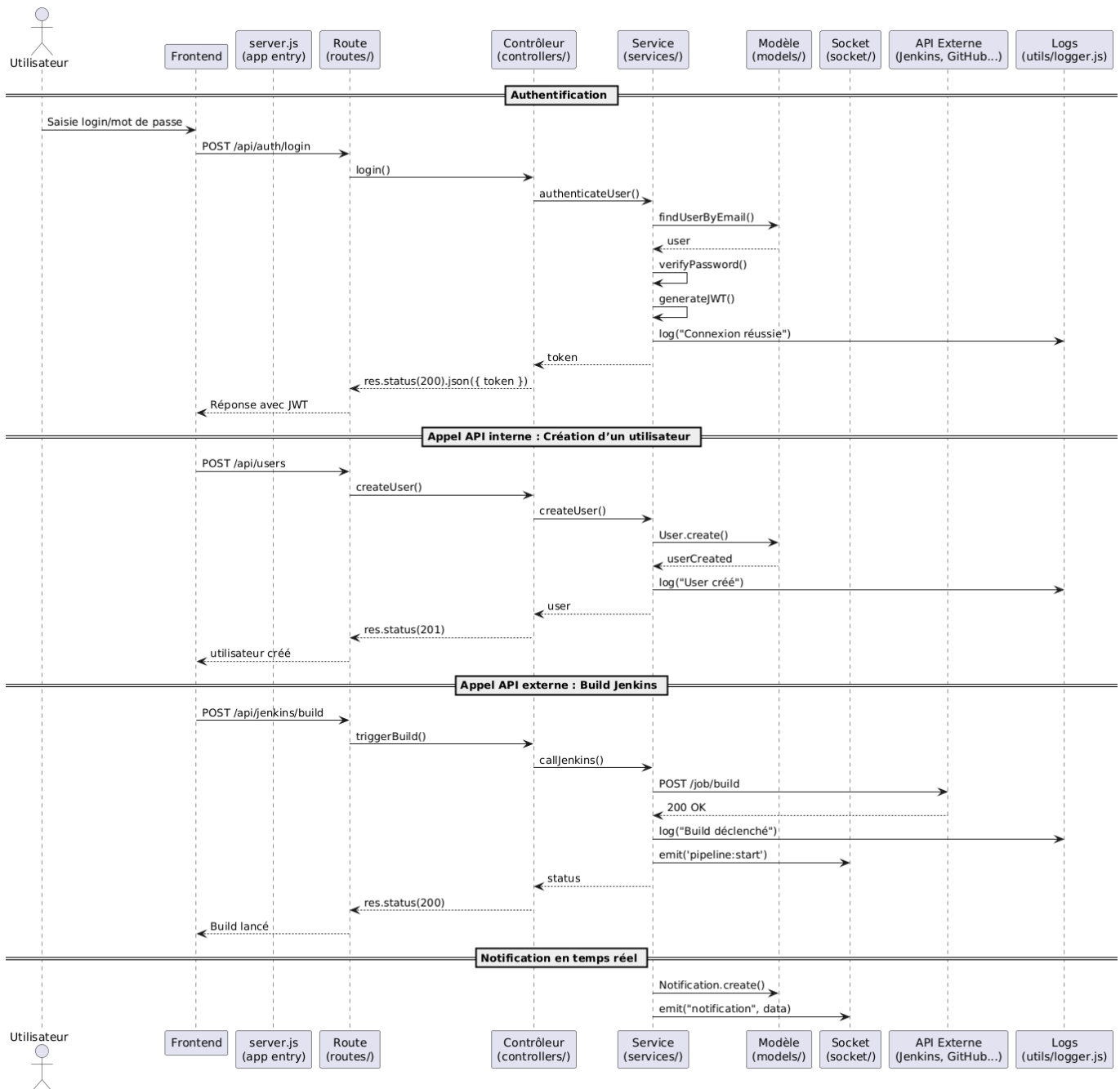


Figure 78 : Diagramme de Séquence – Parcours complet du backend

Conclusion

Ce chapitre a détaillé la mise en place du backend de notre plateforme, en soulignant son architecture modulaire basée sur Node.js. Nous avons explicité les flux de données pour les interactions avec les API externes (Jenkins, SonarQube, GitHub, etc.) et les API internes, en insistant sur la séparation des responsabilités et la sécurité des informations. Le système de notification en temps réel, essentiel pour la réactivité de l'équipe 3S, a également été présenté. Cette conception robuste et évolutive garantit une gestion centralisée et sécurisée des opérations DevOps, posant les bases solides pour le développement du frontend et du tableau de bord centralisé dans le prochain sprint.

Chapitre 7 : Sprint 4 - Développement Frontend et Dashboard centralisé

Introduction

Ce chapitre présente le développement front-end de la plateforme ainsi que le dashboard centralisé permettant de gérer et superviser l'ensemble des services. L'objectif était de concevoir des interfaces ergonomiques, adaptées aux différents profils utilisateurs (administrateur, développeur, DevOps et superviseur), tout en garantissant une centralisation des informations critiques. Grâce aux technologies web modernes, chaque utilisateur dispose d'un espace dédié, intégré dans un tableau de bord unifié facilitant la gestion des projets, le suivi des pipelines CI/CD, la visualisation des métriques et la supervision des déploiements.

I. Caractéristiques de l'Interface Utilisateur

L'interface front-end de la plateforme constitue l'un des piliers essentiels du projet, offrant aux utilisateurs une expérience à la fois fluide, intuitive et visuellement agréable. Pensée pour s'adapter aux besoins de tous les profils utilisateurs (administrateur, développeur, DevOps, superviseur), elle valorise à la fois la dimension technique et l'identité de la société 3S à travers un design raffiné, fonctionnel et technologique.

1) Design Moderne et Mode Sombre

L'interface propose un **mode sombre (dark mode)**, activé par défaut pour offrir un confort visuel optimal, notamment en environnement professionnel. Ce mode réduit la fatigue oculaire, améliore le contraste des composants visuels et confère un aspect moderne et épuré à l'ensemble de l'application. Le thème a été personnalisé aux couleurs de l'entreprise 3S, afin de refléter son **identité visuelle** et sa **culture d'excellence**.

2) Responsive et Accessible sur Tous les Écrans

L'un des points forts du front-end est sa **parfaite adaptabilité**. Grâce à une conception responsive, l'interface s'ajuste automatiquement à tous types d'écrans : ordinateurs de bureau, tablettes et smartphones. Les menus, les tableaux de bord, les graphiques et les sections critiques sont réorganisés dynamiquement pour garantir une lisibilité et une navigabilité sans faille, quelle que soit la résolution.

3) Fluidité et Expérience Utilisateur

La navigation sur la plateforme est rendue **extrêmement fluide** grâce à l'utilisation d'**animations douces**, de transitions naturelles et d'une organisation logique des composants. Chaque action de l'utilisateur est accompagnée d'un retour visuel agréable qui facilite la

compréhension des fonctionnalités. Cette attention portée à l'expérience utilisateur permet une prise en main rapide, même pour les profils non techniques.

4) Graphiques Dynamiques et Données en Temps Réel

Pour assurer une **lecture claire et immédiate** des indicateurs de performance, des pipelines CI/CD, ou encore de l'activité du système, l'interface intègre plusieurs **graphes et tableaux de bord interactifs**. Ces éléments visuels permettent de **suivre en temps réel** l'évolution de l'activité et de prendre des décisions rapides. Des animations accompagnent la mise à jour des données pour garantir un affichage vivant et dynamique.

5) Système de Notifications Avancé

Les notifications occupent une place importante dans l'interface. Les utilisateurs peuvent recevoir des **alertes sonores** accompagnées de messages visuels concernant les événements importants. Il est également possible d'**activer ou désactiver les notifications** selon ses préférences. Ce système a été pensé pour **améliorer la réactivité des utilisateurs** tout en leur laissant un contrôle total sur les flux d'informations.

6) Sécurité et Fiabilité Visuelle

Même si la logique de sécurité est gérée côté back-end, l'interface respecte les meilleures pratiques UX/UI pour la **confidentialité et l'intégrité des interactions**. Les menus, les options sensibles et les éléments critiques sont clairement différenciés, évitant les erreurs de manipulation. L'architecture visuelle garantit une expérience utilisateur fiable et maîtrisée.

7) Une Interface à l'image de 3S

Enfin, cette interface front-end met véritablement en avant l'image de **qualité et d'innovation** portée par la société 3S. Elle combine esthétique, performance et clarté dans une plateforme professionnelle, conçue pour répondre aux standards actuels de l'ingénierie logicielle moderne.

I. Interfaces Générales Partagées par Tous les Utilisateurs

Certaines interfaces de la plateforme sont communes à tous les profils, quel que soit leur rôle (administrateur, développeur, DevOps ou superviseur). Elles couvrent les fonctionnalités essentielles liées à l'accès sécurisé et à la gestion de compte utilisateur.

L'écran d'accueil présente une page de connexion simple, suivie d'un second écran de vérification si l'utilisateur a activé l'authentification à deux facteurs (2FA). Ce second facteur

repose sur un **code unique envoyé par email**, valable pendant **10 minutes**. Ce mécanisme renforce la sécurité sans compromettre la fluidité d'accès.

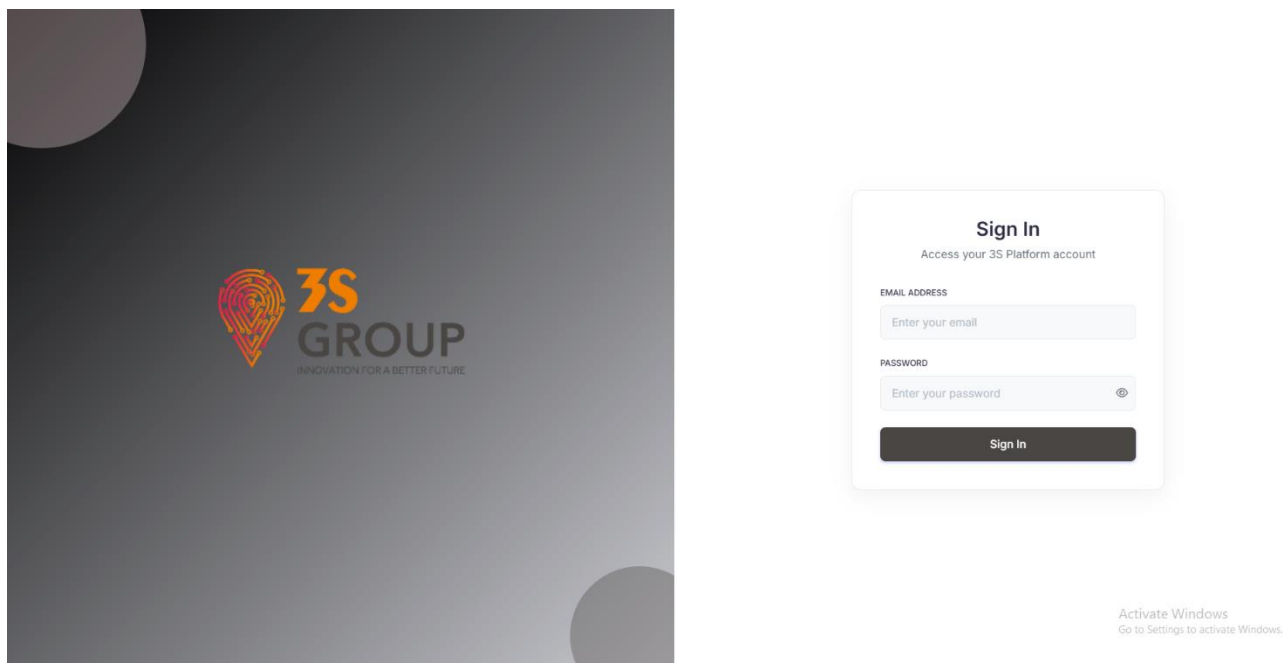


Figure 79 : Page de connexion (login)

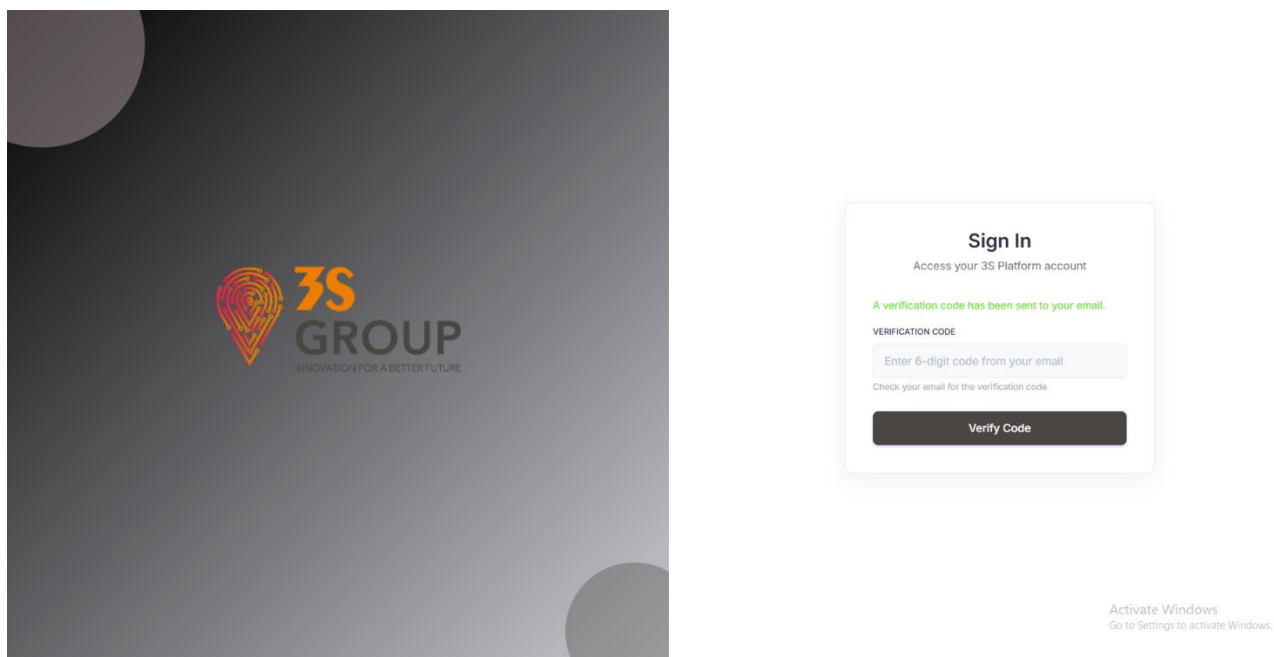


Figure 80 : Vérification du code 2FA

L'onglet **Settings** permet à l'utilisateur de modifier son mot de passe ou d'activer/désactiver la 2FA via un interrupteur. Lors de l'activation, la vérification par email est obligatoire avant que la 2FA soit effectivement activée.

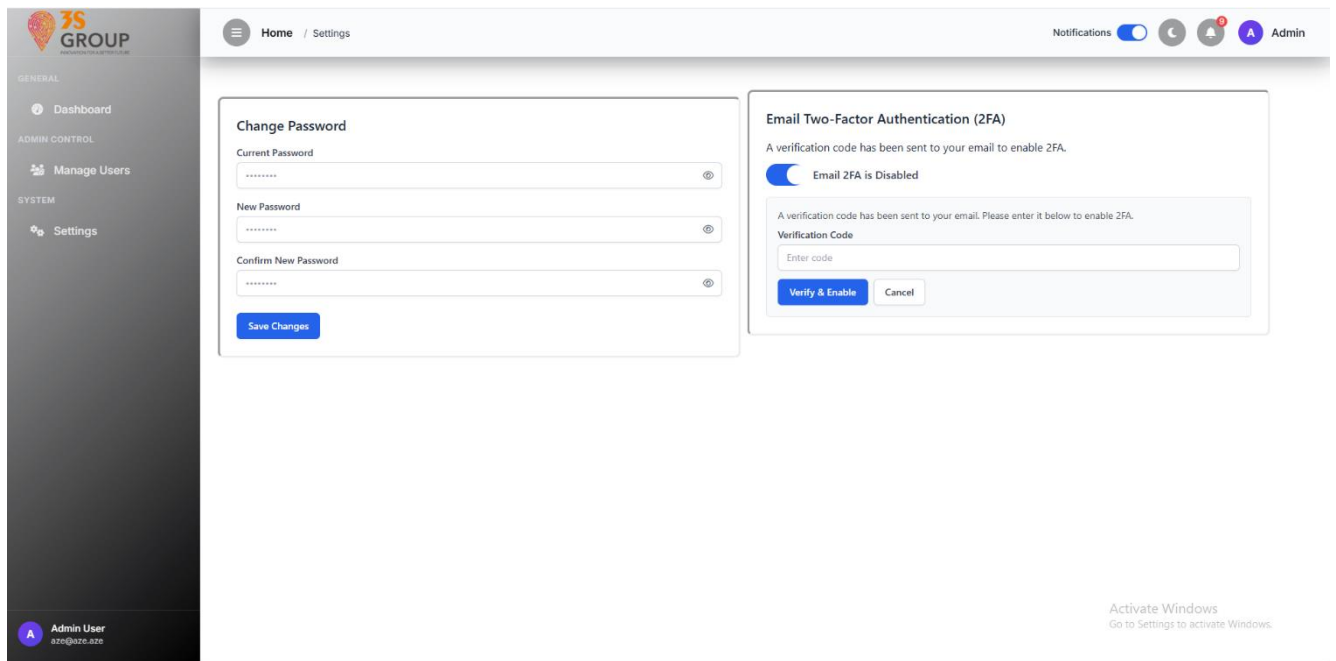


Figure 81 : Paramètres utilisateur (Settings)

Ces interfaces transversales garantissent une expérience utilisateur cohérente, sécurisée et adaptée à tous les rôles.

II. L'interface Administrateur

L'administrateur dispose d'une interface complète lui permettant de **superviser l'activité des utilisateurs**, de **visualiser les statistiques globales** et de **gérer les comptes** via une section dédiée. Il peut notamment :

- Consulter le nombre total d'utilisateurs inscrits et d'utilisateurs actifs en temps réel.
- Analyser la répartition des rôles à travers des graphiques interactifs.
- Accéder à un module de **gestion des comptes** permettant la création, la modification la suppression d'utilisateurs, ainsi que l'import/export de données au format Excel.

L'ensemble de l'interface est disponible en **mode clair et sombre**, et s'adapte dynamiquement aux préférences de l'utilisateur.

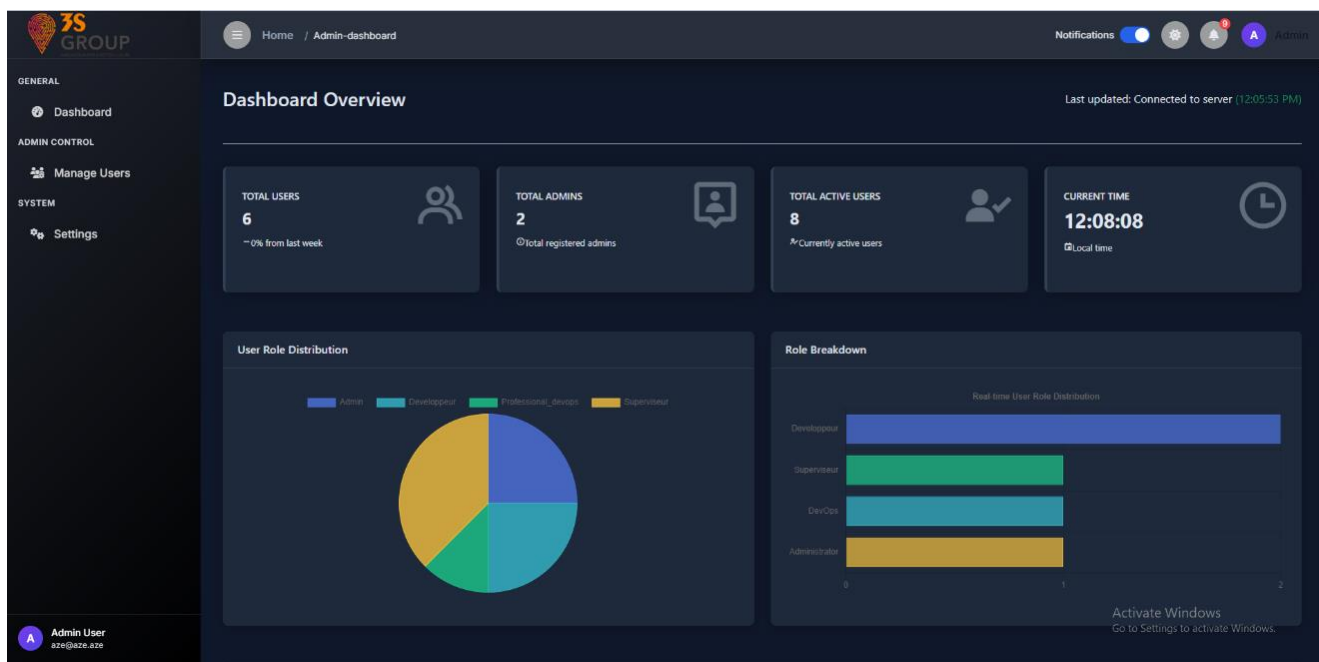


Figure 82 : Vue principale du dashboard admin partie 1 (mode sombre)

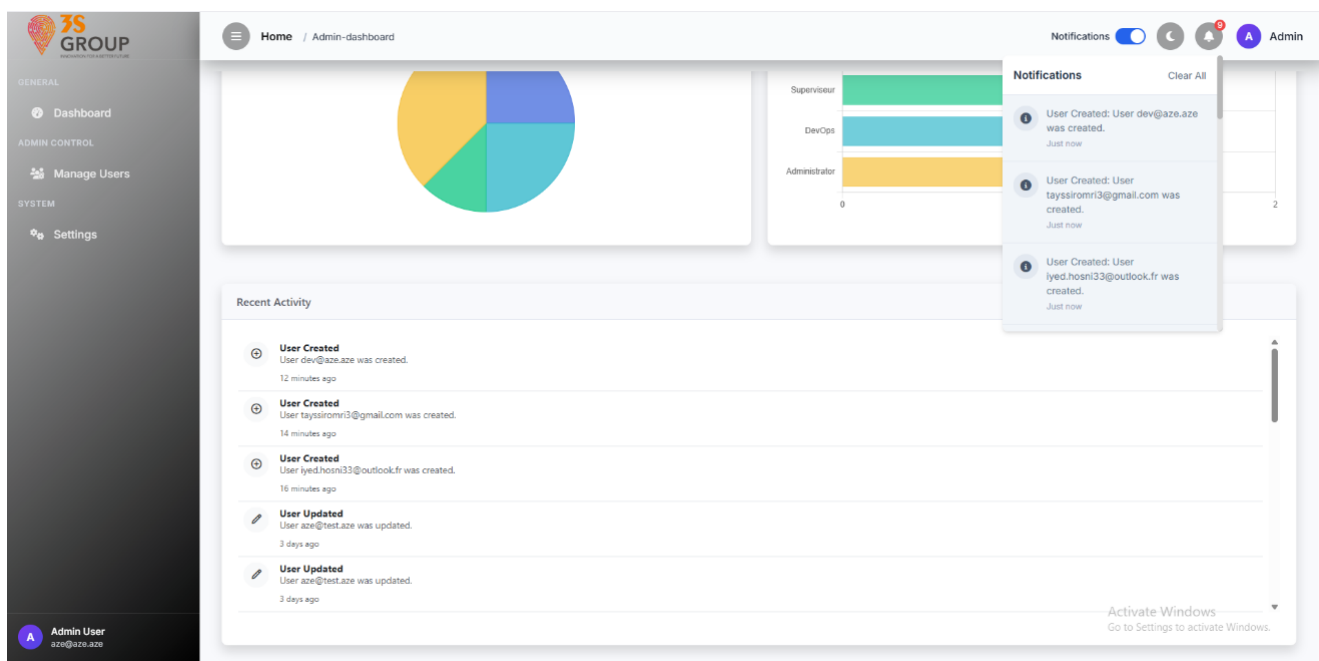


Figure 83 : Vue principale du dashboard admin partie 2 (mode clair)

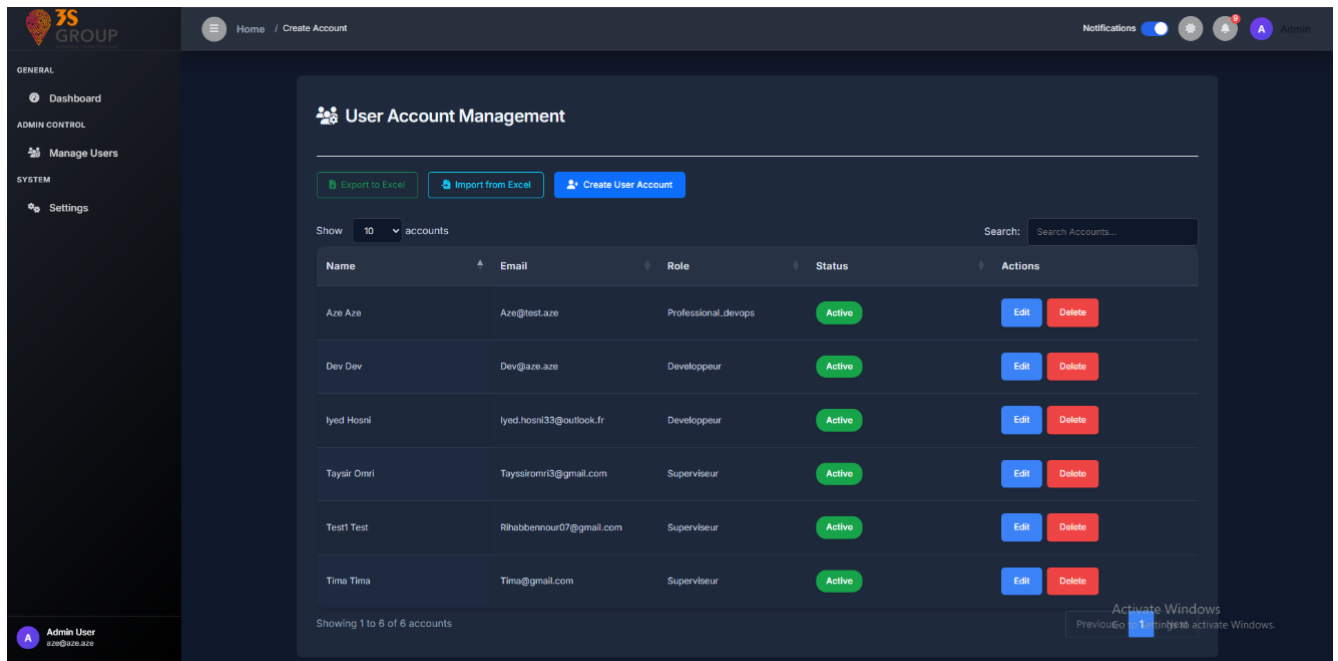


Figure 84 : Gestion des utilisateurs (mode sombre)

III. L'interface professionnel DevOps

Le rôle DevOps bénéficie d'une interface centralisée qui regroupe les informations critiques issues des outils d'intégration et de déploiement utilisés dans le cycle de vie logiciel : **Jenkins**, **SonarQube**, **Nexus** et **Docker Hub**. Cette interface permet une **surveillance en temps réel** de l'état des jobs CI/CD, des analyses de qualité de code, de la gestion des images Docker et des artefacts.

Chaque outil est intégré dans une section dédiée du menu latéral, permettant un accès rapide à ses données principales. Les **tableaux de bord** affichent les indicateurs essentiels (taux de succès, ressources utilisées, états des projets), accompagnés de **graphiques mis à jour dynamiquement**.

Les actions comme le **déclenchement d'un pipeline**, l'**analyse d'un projet** ou la **suppression d'un repository** sont accessibles via des boutons interactifs, souvent accompagnés de **fenêtres modales** (pop-ups) pour afficher les détails complémentaires.

Pour des raisons de clarté, seules les vues globales sont présentées ici.

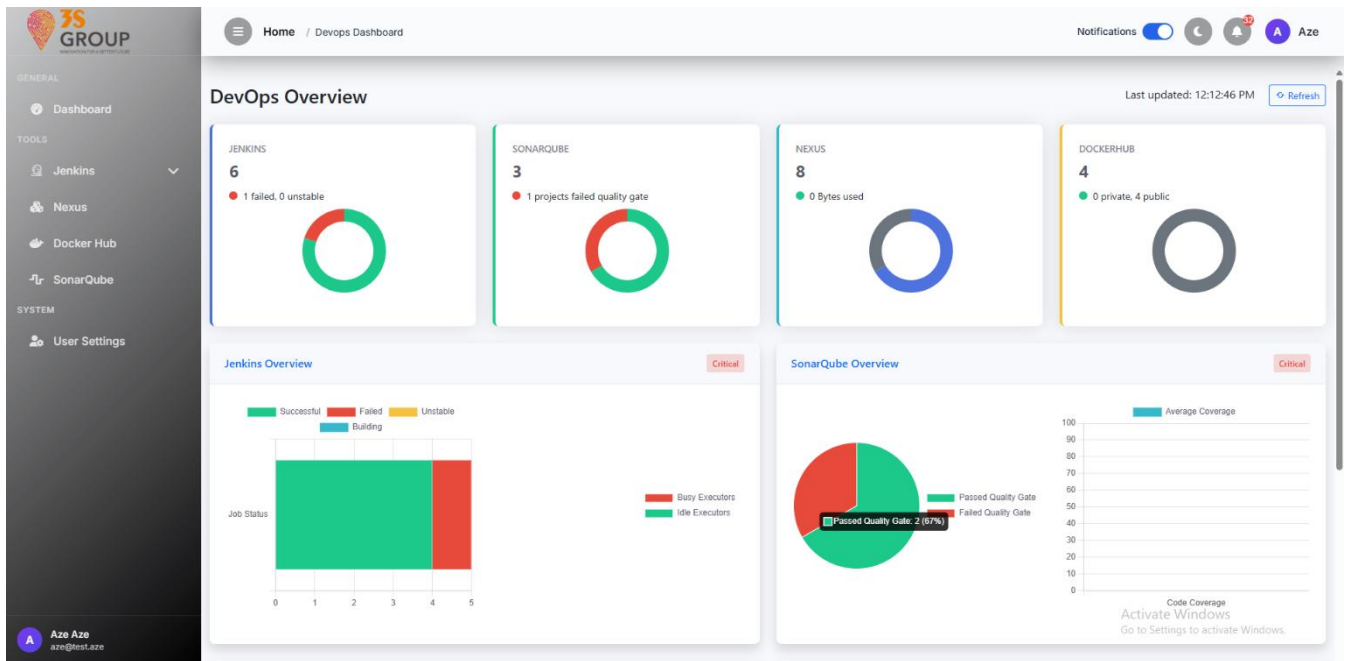


Figure 85 : vue globale du dashboard professionnel devOps partie 1 (mode clair)

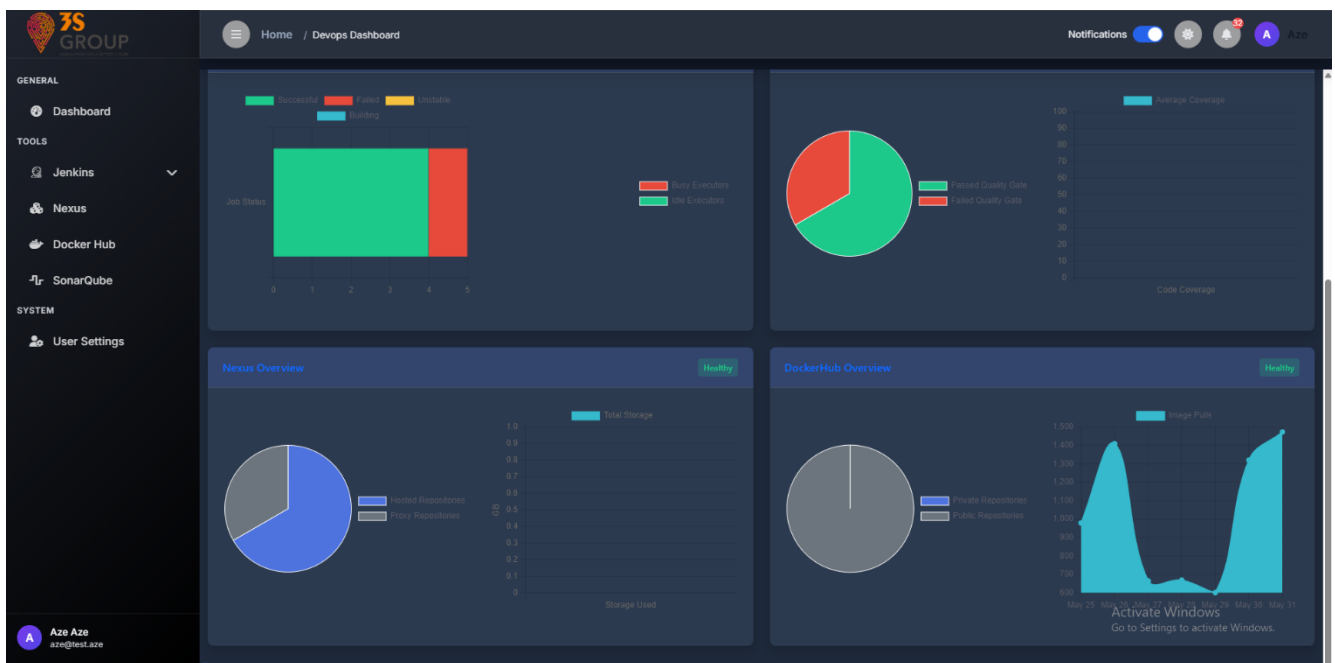


Figure 86 : vue globale du dashboard professionnel devOps partie 2 (mode sombre)

La création de pipelines Jenkins s'effectue via un **formulaire simplifié** centré sur l'essentiel : **nom du job**, **description**, **script de pipeline**, et une **option d'activation du déclencheur webhook GitHub**. Ce choix de minimalisme permet une création rapide tout en garantissant la compatibilité avec les besoins standards d'automatisation.

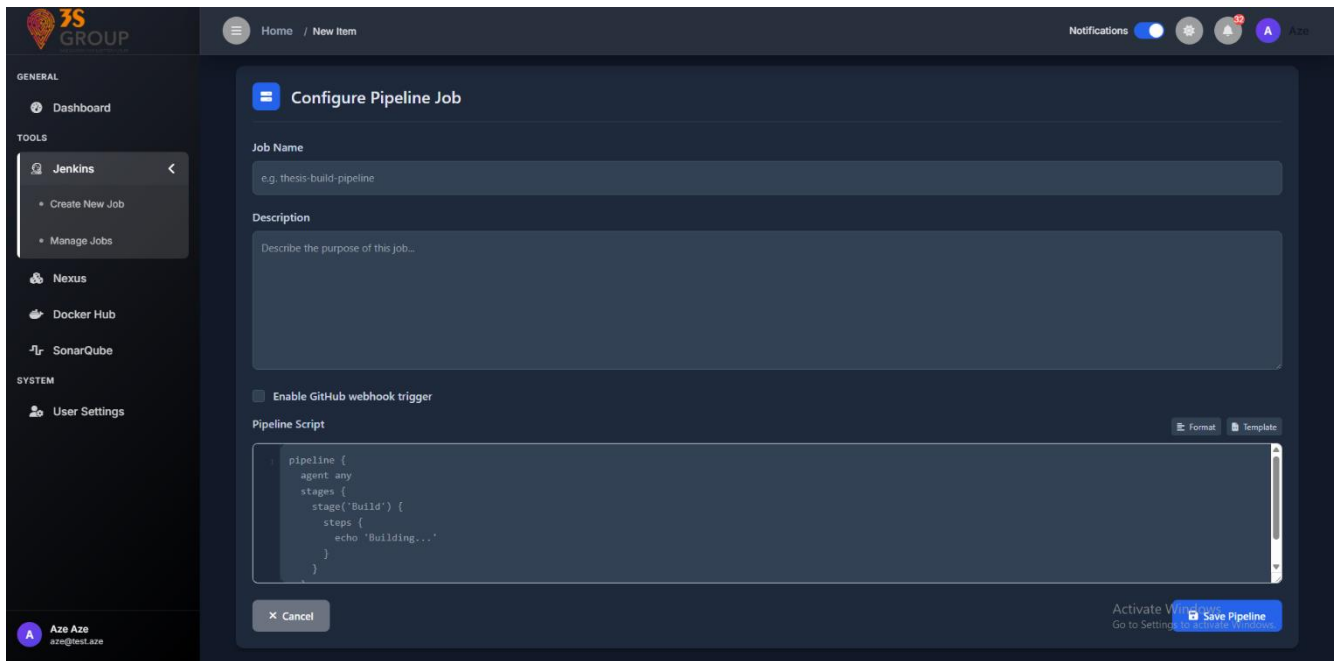


Figure 87 : Configuration d'un pipeline Jenkins (mode sombre)

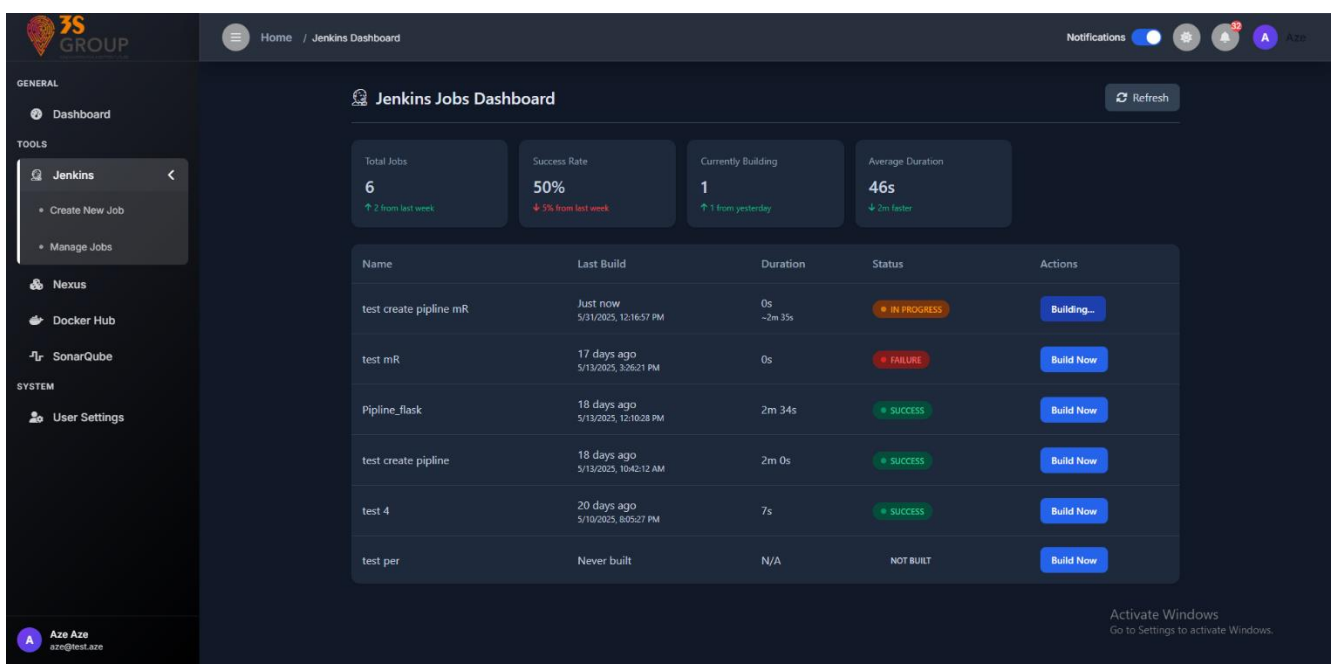


Figure 88 : Tableau de bord des jobs Jenkins (mode sombre)

En cliquant sur un job depuis le tableau de bord Jenkins, l'utilisateur accède à une vue détaillée du pipeline, affichant l'historique complet des builds. Chaque entrée de l'historique peut être cliquée pour afficher les logs du build dans une fenêtre modale (popup). Des options telles que modifier, supprimer, ou relancer un build sont également disponibles, chacune étant accompagnée d'un popup dédié pour une interaction rapide et sans rechargement de page.

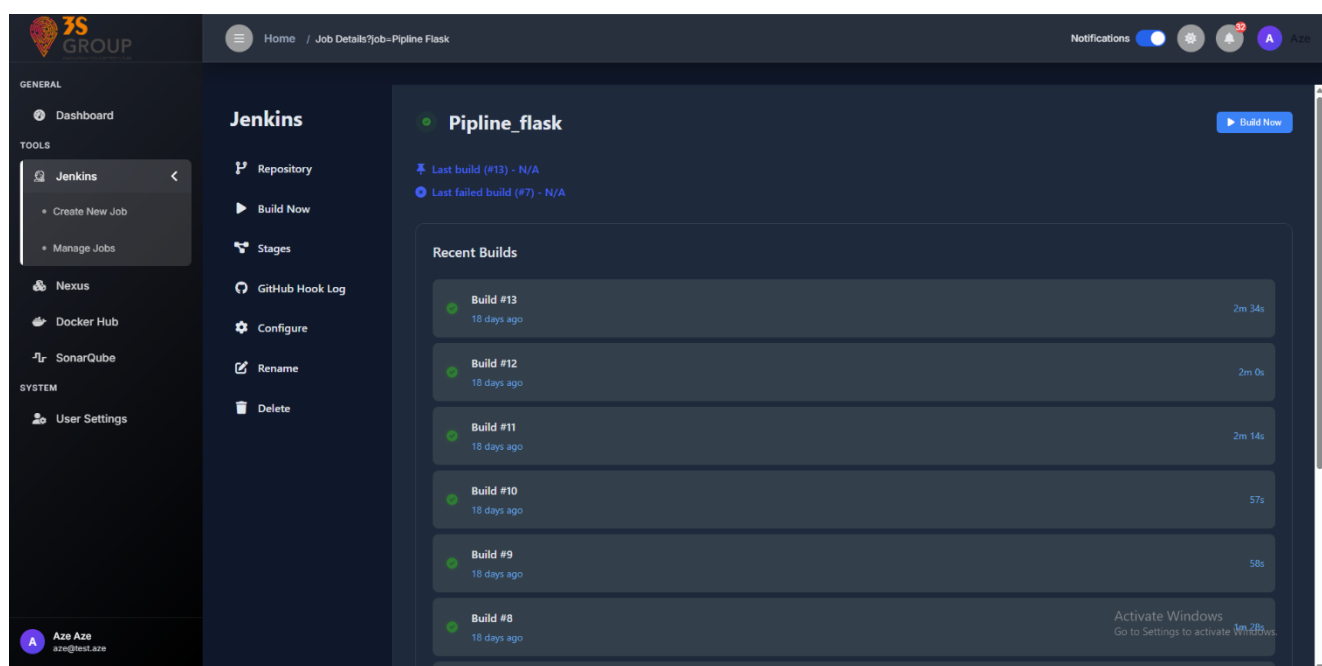


Figure 89 : vue détaillée du pipeline (mode sombre)

Concernant Docker Hub, **un clic sur un repository permet d'accéder à ses détails** : nom, description, visibilité (publique ou privée), commande de pull Docker, date de dernière mise à jour et liste des tags disponibles. Une section permet également de modifier ses informations via un formulaire simplifié.

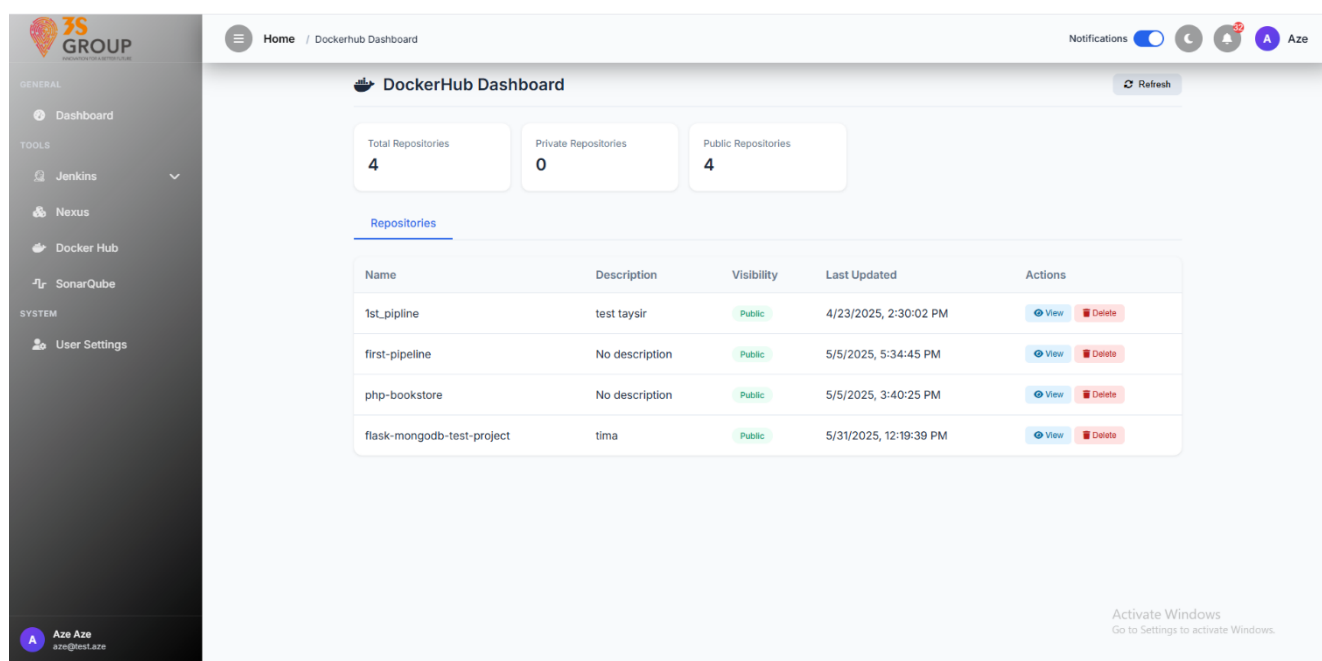


Figure 90 : Tableau de bord de Docker Hub (mode clair)

L'interface dédiée à Nexus permet de visualiser l'ensemble des dépôts configurés, qu'ils soient hébergés ou proxy. Les utilisateurs peuvent consulter le nom du repository, son type, son format (maven, docker, nuget...), son statut, ainsi que l'espace utilisé.

Un clic sur un dépôt ouvre une fenêtre modale contenant ses détails techniques : arborescence des fichiers, métadonnées, et actions possibles comme la suppression ou le lancement d'une tâche. Dans le cas des dépôts Docker, le popup affiche également la structure des manifests, les tags disponibles, et les identifiants SHA des images, permettant une analyse fine du contenu.

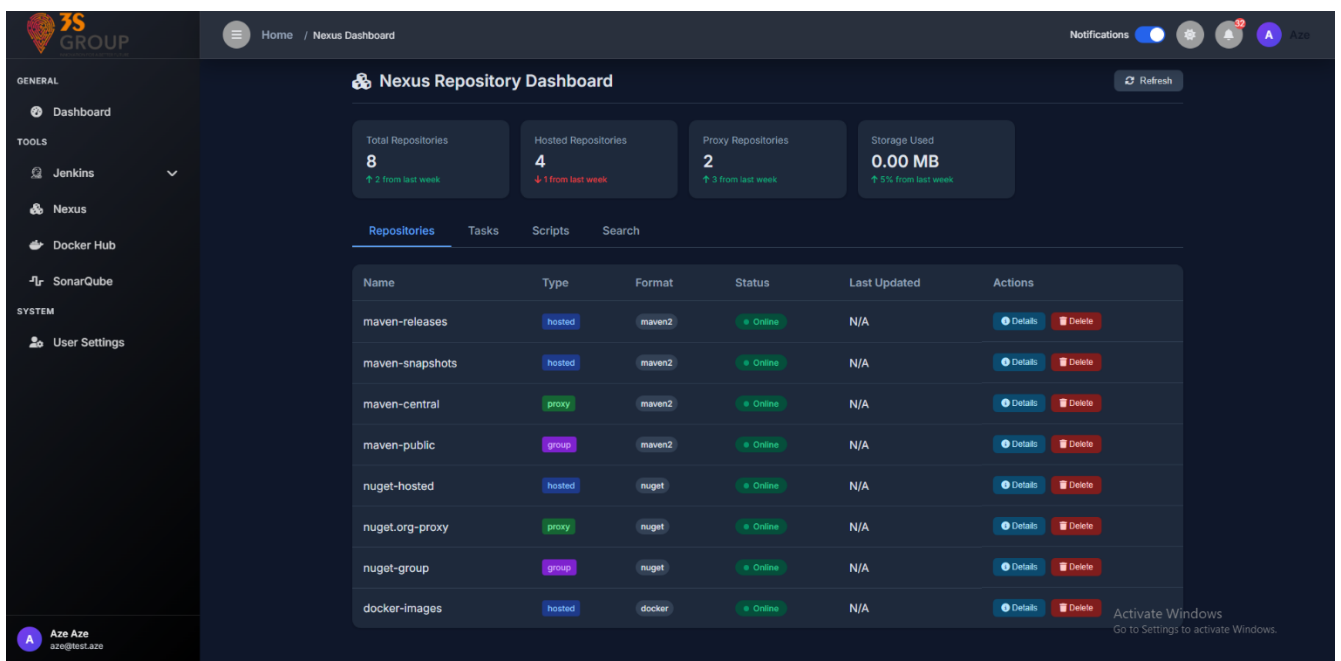


Figure 91 : Tableau de bord de nexus (mode sombre)

La section **SonarQube** permet aux utilisateurs DevOps de suivre l'état de santé des projets logiciels en analysant des métriques clés telles que les bugs, les vulnérabilités, le taux de couverture, les duplications ou encore le respect des **quality gates**. Chaque ligne du tableau correspond à un projet analysé, avec des indicateurs visuels facilitant la lecture.

En cliquant sur un projet, l'utilisateur accède à ses **détails techniques complets**. L'historique des analyses récentes est également affiché sous forme de liste, avec la possibilité de **consulter chaque rapport d'analyse** individuellement pour explorer les résultats en profondeur.

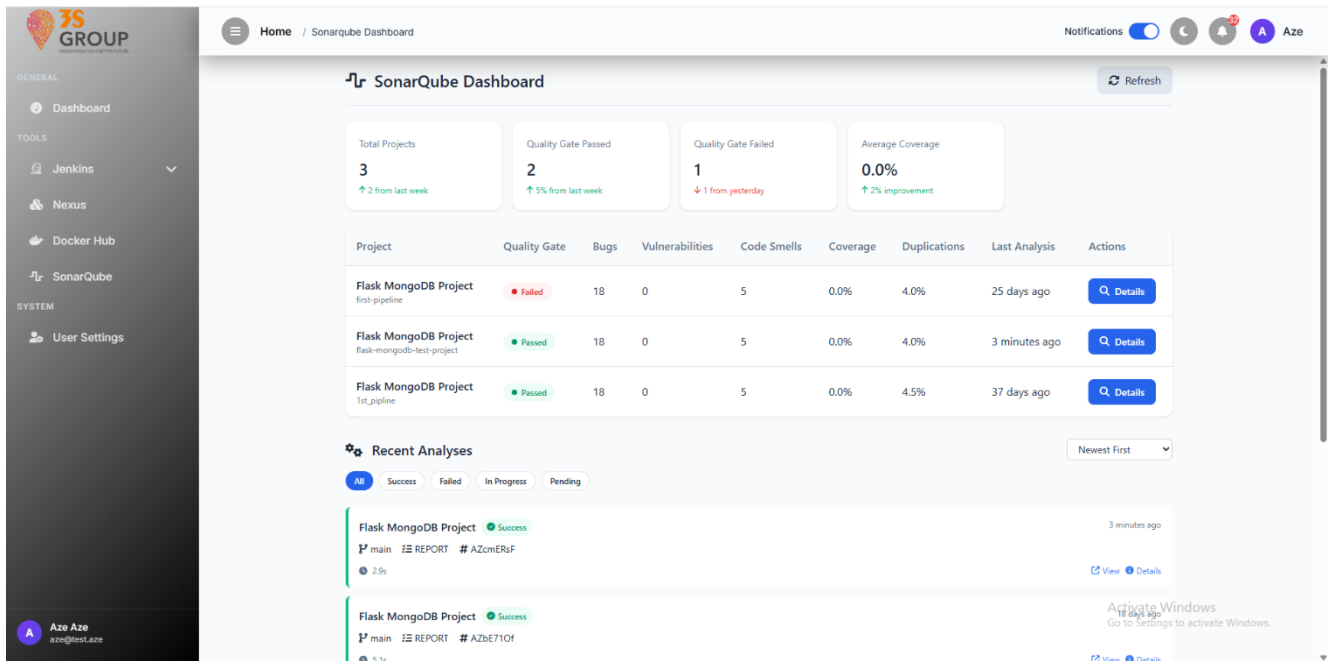


Figure 92 : Tableau de bord de SonarQube (mode claire)

IV. L'interface Développeur

L'interface dédiée aux développeurs a été pensée pour offrir une vue d'ensemble sur l'avancement des projets de l'équipe tout en facilitant l'interaction avec GitHub. Tous les développeurs disposent de la même interface et accèdent aux mêmes données, assurant ainsi une vision unifiée et collaborative sur les dépôts suivis, l'activité récente, ainsi que les priorités de développement.

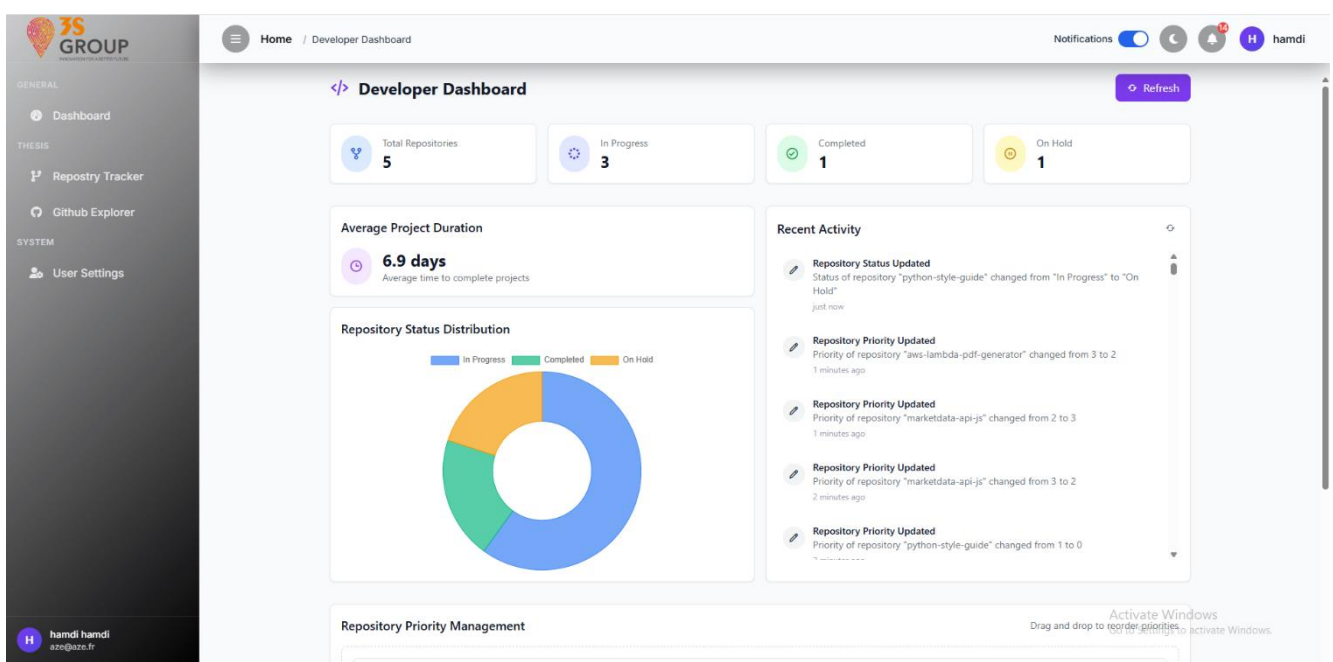


Figure 93 : vue globale du dashboard developpeur partie 1 (mode clair)

Le **dashboard développeur** présente des indicateurs clés comme le nombre total de dépôts suivis, leur répartition par statut (*In Progress*, *Completed*, *On Hold*), la durée moyenne des projets, ainsi qu'un historique des actions récentes. Les développeurs peuvent également ajuster **l'ordre de priorité** des dépôts à travers un système de glisser-déposer intuitif.

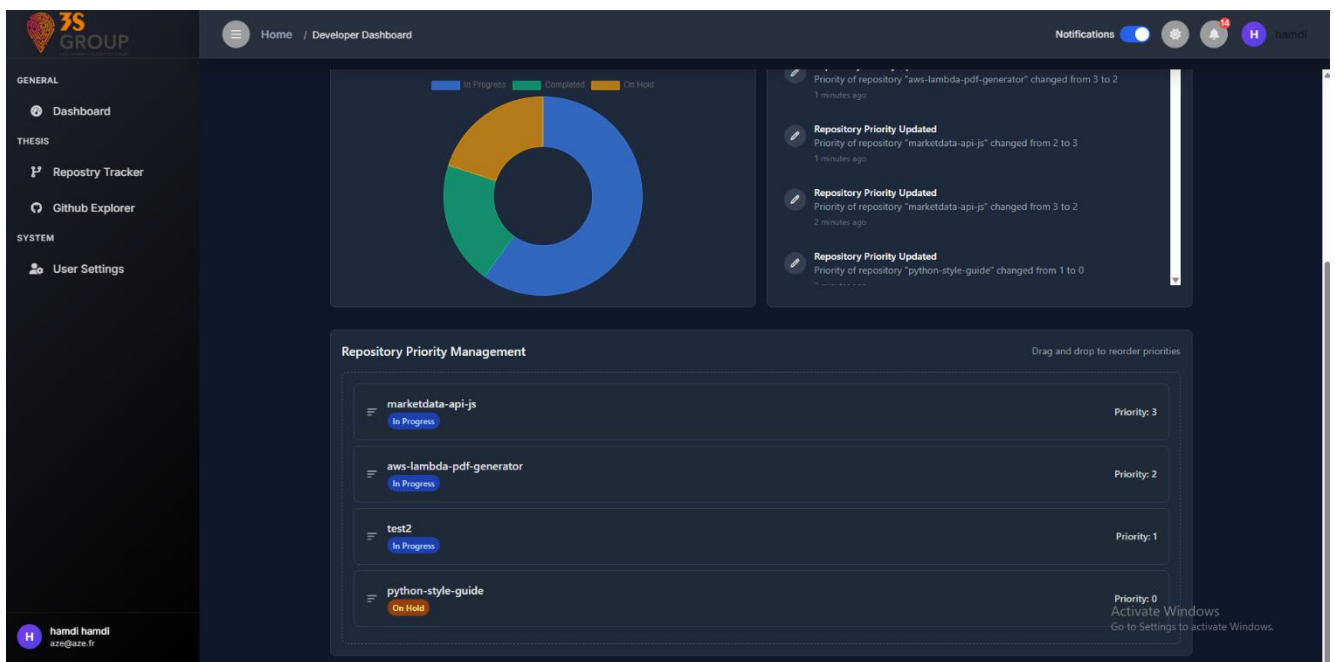


Figure 94 : vue globale du dashboard developpeur partie 2 (mode sombre)

Le module **GitHub Tracker** permet aux développeurs de suivre précisément l'avancement de chaque dépôt. Ils peuvent **ajouter un nouveau dépôt** via un lien GitHub, **modifier son statut** (*Completed*, *In Progress*, *On Hold*), ou **le supprimer** si nécessaire. Toutes les informations clés comme la description, le langage utilisé, les contributeurs, ou encore les fichiers du projet sont récupérées dynamiquement grâce à l'**API GitHub**.

Un **pop-up interactif** permet, en un clic sur un dépôt, d'accéder à une vue enrichie qui reprend tout ce que GitHub offre : liste des **commits**, résumé des **analyses** (langages utilisés), ainsi que la possibilité **d'explorer les fichiers**, les **ouvrir**, les **copier** ou même les **télécharger**.

Chaque dépôt ajouté est enregistré en base de données, et les développeurs peuvent intégrer leur **token GitHub personnel** pour débloquer l'accès à leurs **dépôts privés**.

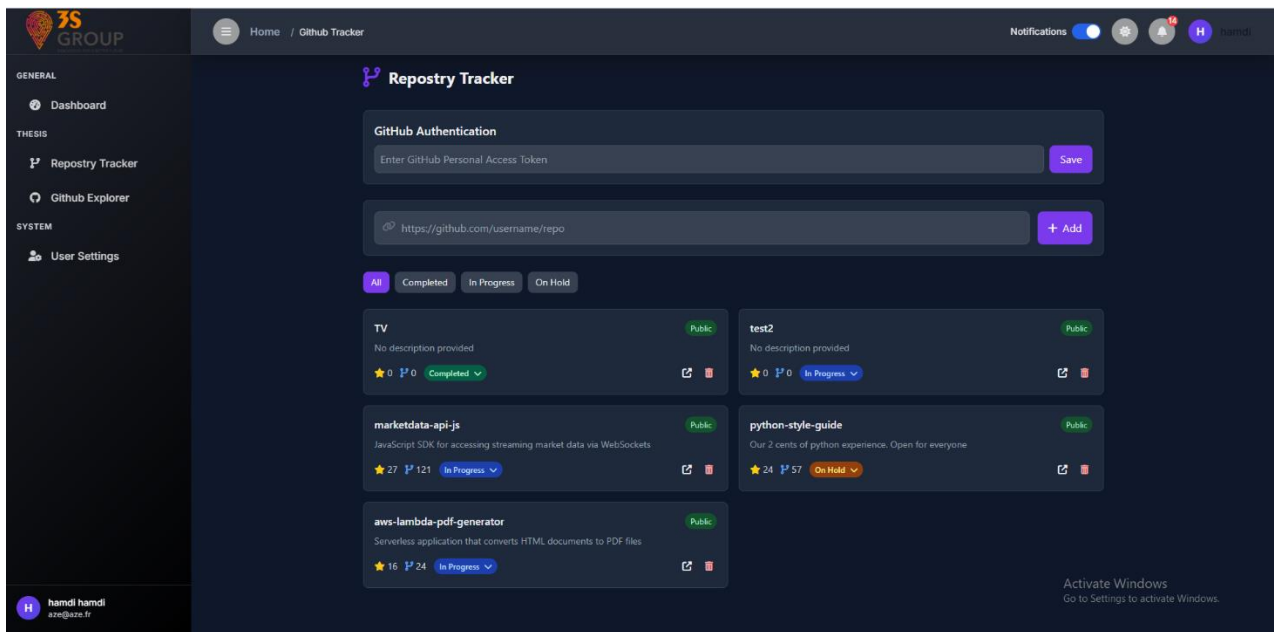


Figure 95 : Interface GitHub Tracker – Vue globale (Mode Sombre)

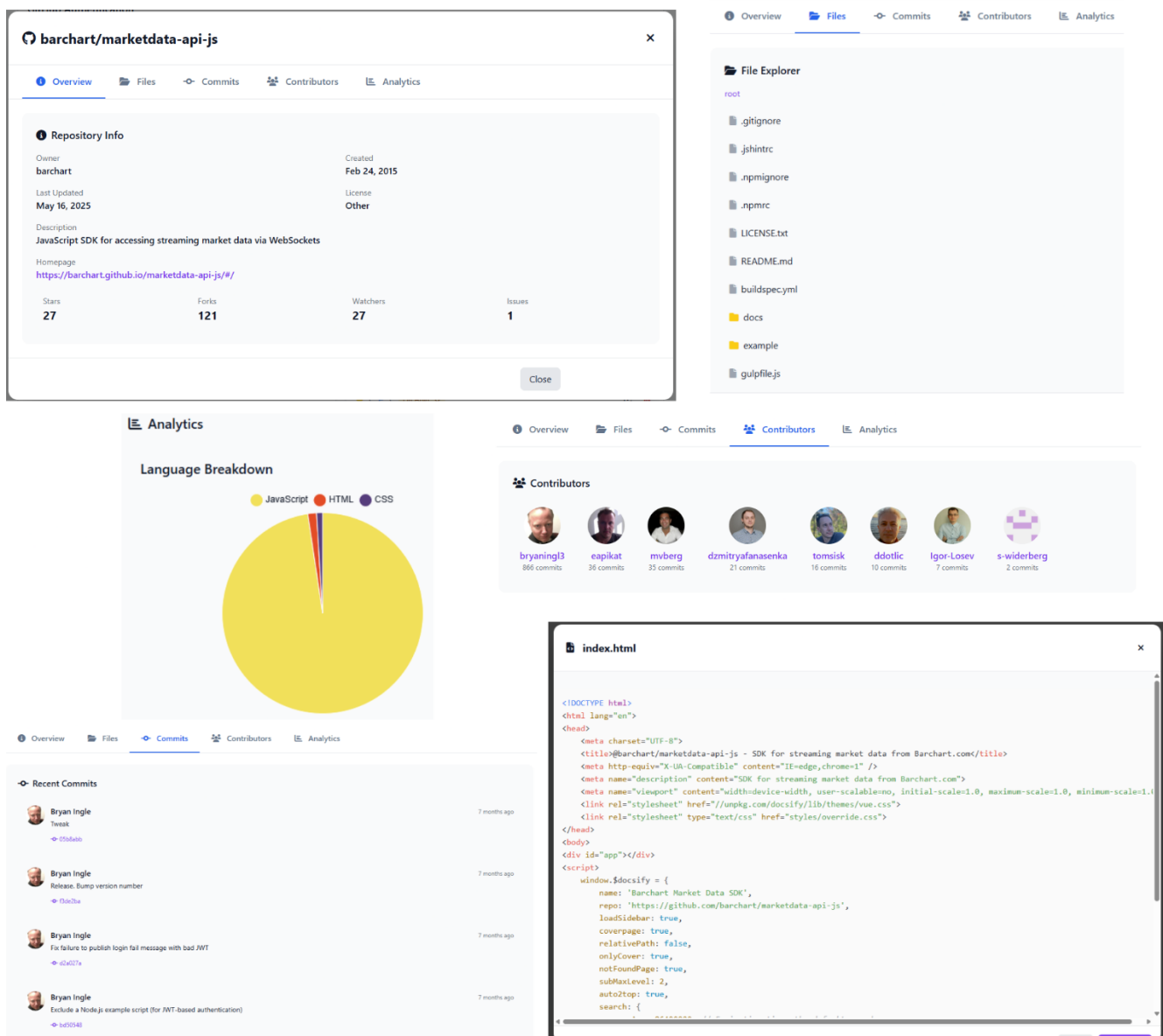


Figure 96 : Fenêtre Détail d'un Dépôt GitHub – Informations complètes (Mode Clair)

La section **GitHub Explorer** permet de rechercher des **profils publics GitHub** en renseignant simplement un nom d'utilisateur. Une fois le profil trouvé, l'interface affiche l'ensemble des **repositories associés**, ainsi que les détails essentiels de chaque projet. L'utilisateur peut également explorer un **repository public en particulier** en collant directement son lien GitHub. Dans les deux cas (recherche par nom ou ajout direct via lien), un clic sur un dépôt ouvre la même **fenêtre détaillée** présentée dans le *GitHub Tracker* (voir Figure 100 : Fenêtre détail d'un dépôt GitHub – informations complètes en mode clair).

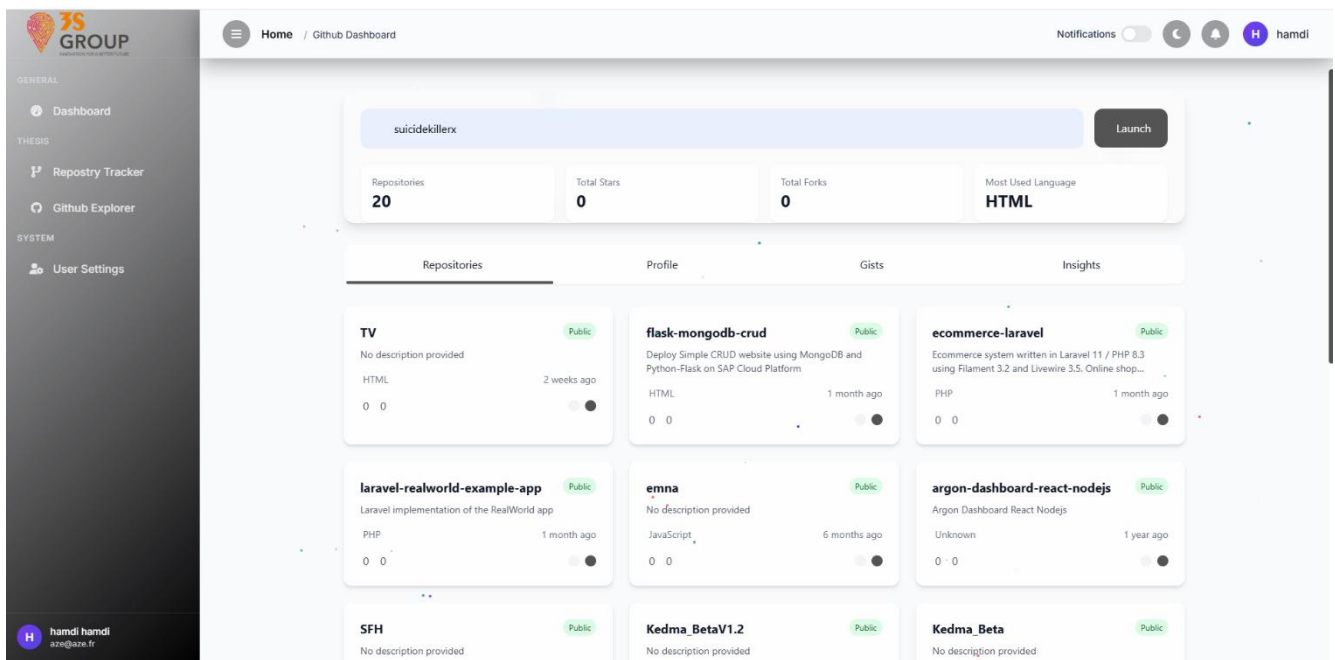


Figure 97 : Interface GitHub Explorer – Vue globale (Mode Claire)

Ce découpage permet de mettre en valeur les différents usages offerts à un développeur, tout en conservant une interface cohérente, moderne et centrée sur la productivité.

V. L'interface Superviseur

L'interface superviseur a été conçue pour fournir un **aperçu technique exhaustif** de l'état de l'infrastructure, en exploitant des **tableaux de bord Grafana intégrés** via des iframes. Chaque graphique est généré dynamiquement à partir du backend (via une API) et **s'adapte automatiquement au thème clair ou sombre** de l'interface globale, assurant une cohérence visuelle parfaite. Cette architecture permet également de centraliser la visualisation de multiples sources de données en un point unique de supervision.

```

<iframe
  src="${grafanaUrl}/d-solo/34b24cc6-ba91-4466-b9d2-5595c60c96fb/prometheus-
overview?orgId=1&timezone=utc&var-datasource=default&var-cluster=${__all}&var-
job=${__all}&var-
instance=${__all}&theme=${initialThemeForNewUrl}&panelId=14&__feature.dashboardSceneSolo"
  loading="lazy">
</iframe>

```

Le tableau de bord (dashboard) global de superviseur offre une **vision instantanée de la santé du système** : taux d'utilisation CPU, engagement de ressources (CPU/mémoire), occupation disque et évolution temporelle des courbes de charge. Il permet au superviseur de réagir rapidement à toute anomalie de performance.

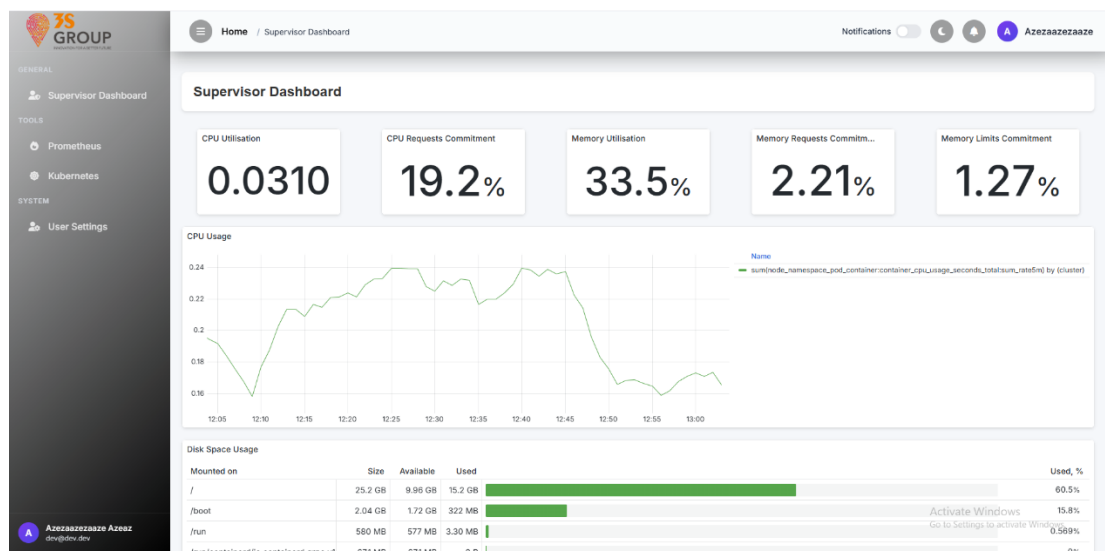


Figure 98 : vue globale du dashboard superviseur partie 1 (mode clair)

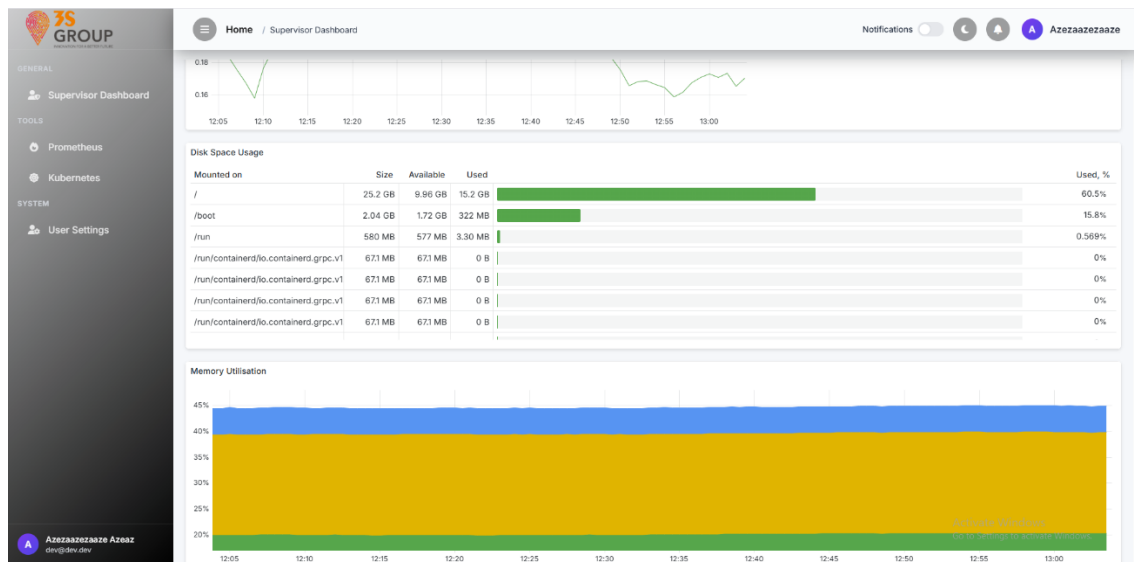


Figure 99 : vue globale du dashboard superviseur partie 2 (mode clair)

Le dashboard Prometheus expose des métriques internes à la plateforme de monitoring :

- durées moyennes de scrapping,
- temps de traitement des requêtes,
- volumes de mémoire tampon utilisés,
- taux de collecte des données.

Cette visualisation facilite la détection de lenteurs dans la collecte des indicateurs.

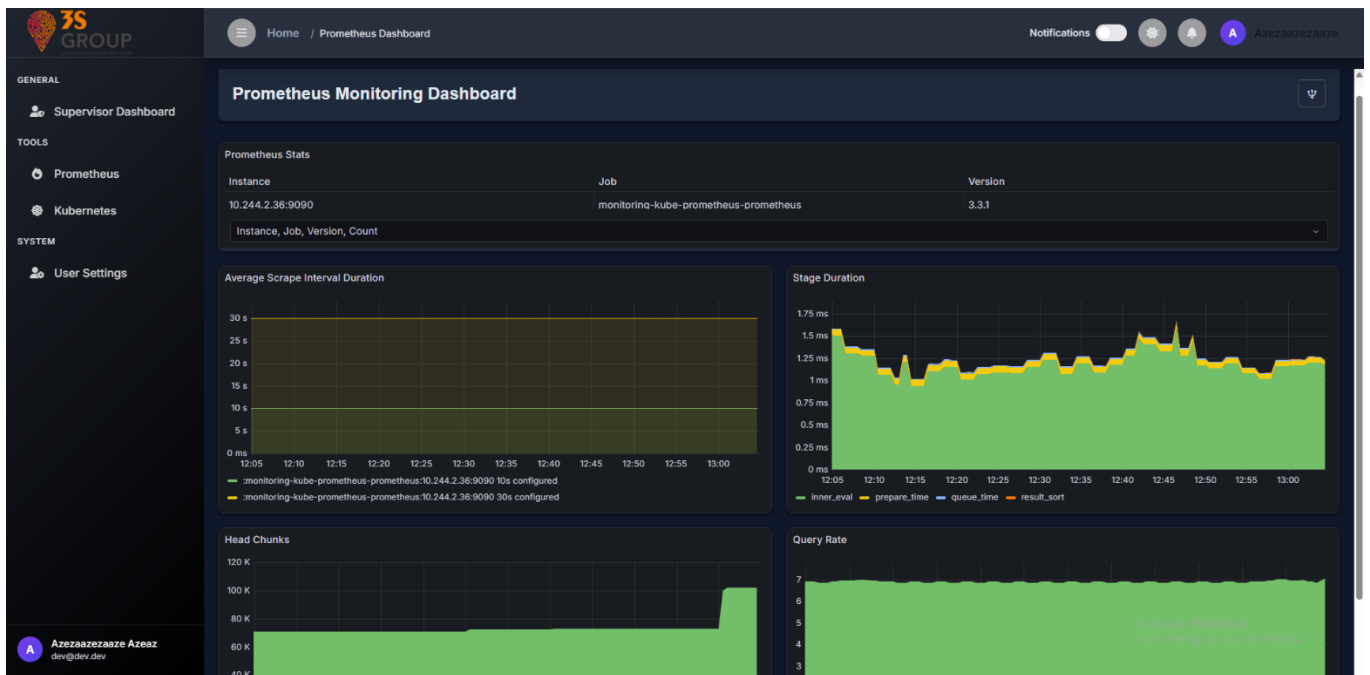


Figure 100 : Le dashboard Prometheus (mode sombre)

Le dashboard Kubernetes est dédié à la surveillance des **pods actifs** dans le cluster Kubernetes. Il affiche :

- l'état de chaque pod (running/pending/error),
- leur consommation de CPU/mémoire,
- les débits réseau en entrée/sortie.

Un clic sur un pod permet d'accéder à des **graphes détaillés** sur l'usage des ressources par namespace et l'application des quotas configurés.

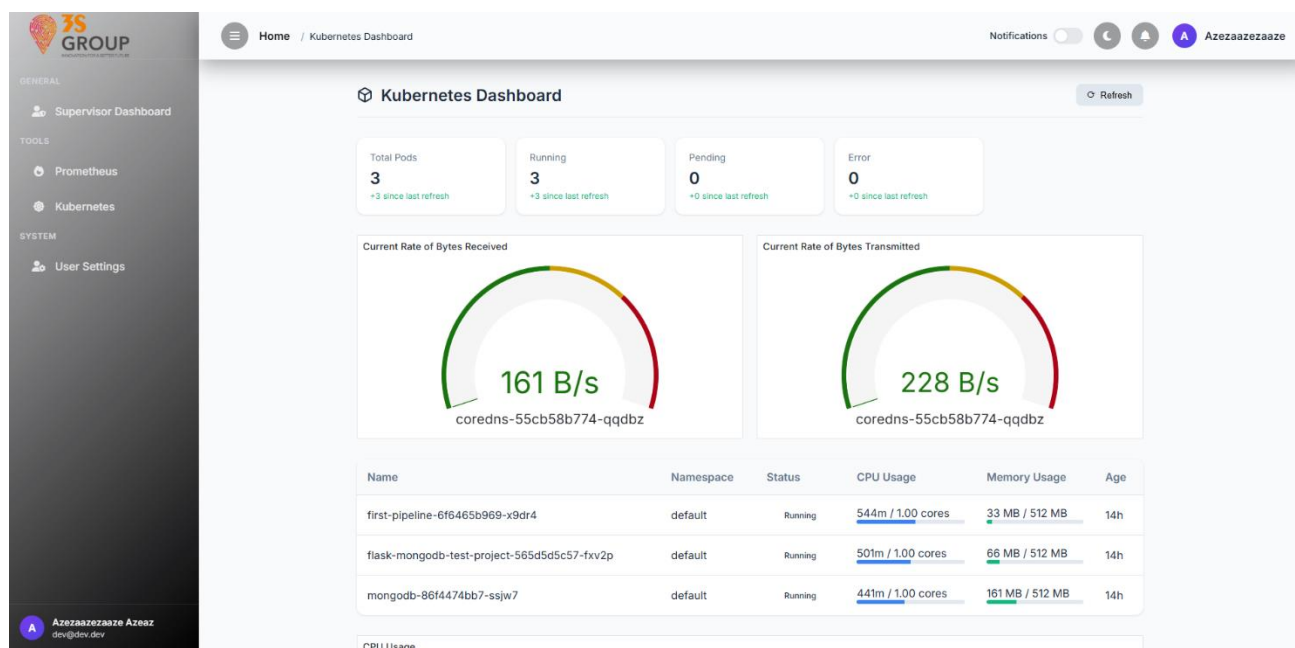


Figure 101 : Le dashboard Kubernetes partie 1 (mode clair)

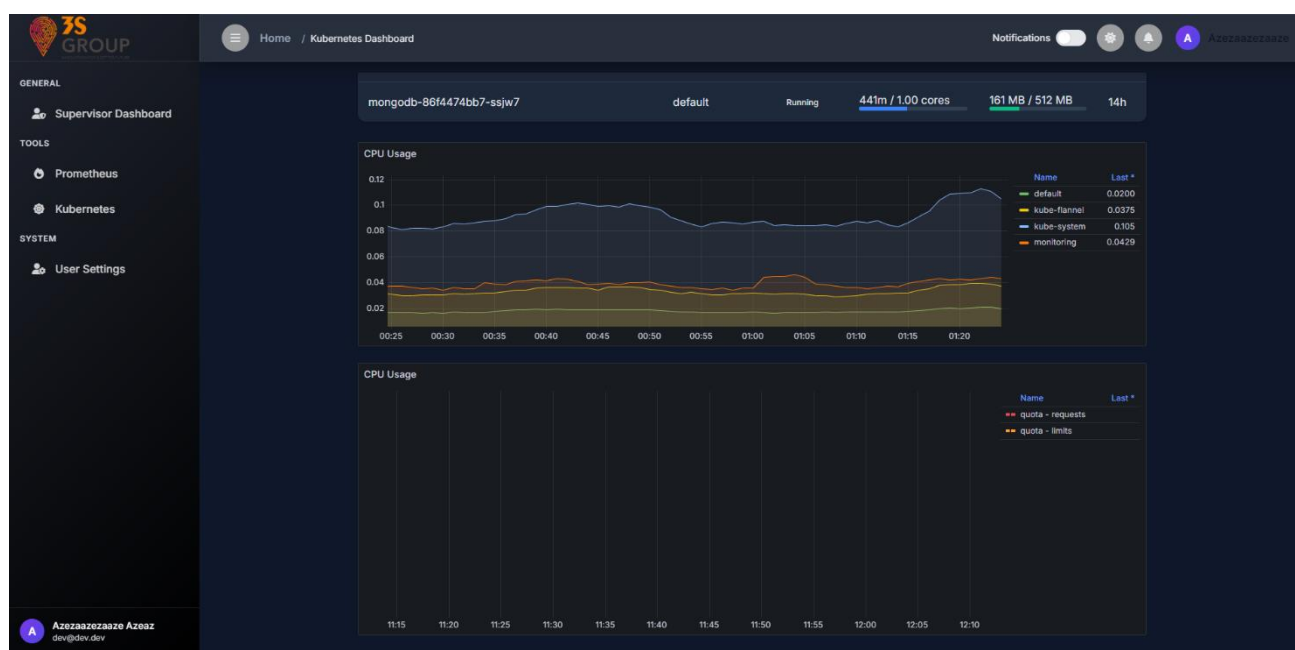


Figure 102 : Le dashboard Kubernetes partie 2 (mode sombre)

L'interface superviseur joue un rôle essentiel pour le **pilotage technique et préventif** de la plateforme, en intégrant de manière fluide les données Grafana dans une UI moderne et unifiée.

Conclusion

Ce sprint a marqué une étape déterminante dans l'expérience utilisateur et l'observabilité globale de la plateforme. En mettant en place des interfaces adaptées à chaque profil (administrateur, DevOps, développeur, superviseur), nous avons réussi à unifier la gestion, le suivi et l'analyse en temps réel au sein d'une architecture frontend cohérente et moderne.

L'intégration des données issues de multiples sources (API internes, outils DevOps, tableaux de bord Grafana) s'est traduite par des visualisations interactives, personnalisables et responsives. Chaque interface joue un rôle opérationnel clé, permettant aux utilisateurs de prendre des décisions informées plus rapidement et efficacement.

En somme, ce sprint a consolidé le volet visuel et fonctionnel du projet, en rendant l'ensemble de la plateforme **intelligente, accessible et pilotable en temps réel**.

Conclusion Générale

Ce projet de fin d'études a constitué un véritable défi, tant par son ambition technique que par sa portée fonctionnelle. Dans le cadre d'une licence en Génie Logiciel et Systèmes d'Information, où le domaine du DevOps reste encore relativement nouveau, nous avons relevé le pari audacieux de concevoir, développer et déployer une plateforme complète et opérationnelle d'automatisation, d'intégration continue, de supervision et de gestion multi-profils. Il s'agit d'un projet complexe qui regroupe à lui seul les difficultés techniques, fonctionnelles et organisationnelles de plusieurs sous-projets rassemblés en un.

L'ensemble de cette réalisation a été accompli en seulement quatre mois, un délai très court au vu de l'ampleur et de la profondeur du travail fourni. Chaque semaine, nous avons avancé sur des chantiers parallèles allant de la mise en place de l'infrastructure jusqu'à la conception UI/UX, tout en assurant un niveau de qualité professionnel.

En plus du développement applicatif, nous avons également pris en charge l'installation, la configuration et la gestion complète de l'infrastructure DevOps, en créant trois machines virtuelles Ubuntu pour y déployer les outils (Jenkins, SonarQube, Nexus, Docker, Kubernetes, Prometheus et Grafana). Ces composants ont été intégrés harmonieusement dans une architecture CI/CD robuste, sécurisée et automatisée, constituant un environnement réel de production.

Le backend, développé en Node.js selon une architecture modulaire, assure des communications sécurisées avec ces outils via des API dédiées. Le frontend, quant à lui, propose une interface adaptée à plusieurs rôles métiers (administrateur, DevOps, développeur, superviseur), avec un design moderne, responsive, et synchronisé avec les thèmes sombres/clairs de l'utilisateur. L'intégration de Grafana via iframe dynamique permet une visualisation en temps réel de l'état de l'infrastructure.

Nous avons également mis en œuvre des mécanismes de sécurité avancés, tels que le JWT, le RBAC (contrôle d'accès par rôle), la journalisation structurée, et l'authentification à deux facteurs (2FA) avec envoi de code unique par e-mail.

Actuellement, la plateforme est conçue pour être utilisée par une seule équipe projet. Tous les utilisateurs ayant un rôle donné partagent la même interface et accèdent aux mêmes données. Une évolution à prévoir dans le futur serait l'ouverture à la gestion multi-équipes : les utilisateurs ayant le même rôle mais appartenant à des équipes différentes accéderaient

uniquement aux données de leur propre espace, garantissant ainsi un cloisonnement logique, une meilleure sécurité des données et une adaptabilité à grande échelle.

En conclusion, ce projet a été pour nous une expérience technique, humaine et professionnelle extrêmement riche, nous permettant de valider nos compétences dans des domaines modernes, complexes et critiques de l'ingénierie logicielle. Il nous a préparé de manière concrète aux exigences du monde professionnel et constitue un socle solide pour démarrer nos carrières dans le monde du DevOps et de l'automatisation des systèmes.

Bibliographie

- [1] Kim Gene, D. P. (2016). *The DevOps Handbook: How to Create World-Class Agility*. United States of America: IT Revolution Press. Consulté le Avril 10, 2025
- [2] 3S. (2023). *groupe 3s tunisie*. Consulté le 20, 2025, sur 3s.com.tn: <https://www.3s.com.tn/groupe-3s-tunisie/>
- [3] wikipedia. (2024, avril 20). *Modèle en cascade*. Consulté le mars 26, 2025, sur [wikipedia.org](https://fr.wikipedia.org/wiki/Mod%C3%A8le_en_cascade): https://fr.wikipedia.org/wiki/Mod%C3%A8le_en_cascade
- [4] wikipedia. (2025, mars 5). *Scrum (développement)*. Consulté le mars 26, 2025, sur [wikipedia.org](https://fr.wikipedia.org/wiki/Scrum_(d%C3%A9veloppement)): [https://fr.wikipedia.org/wiki/Scrum_\(d%C3%A9veloppement\)](https://fr.wikipedia.org/wiki/Scrum_(d%C3%A9veloppement))
- [5] Hodicq, V. (2024, février 26). *L'importance d'une culture DevOps au sein d'une entreprise*. Consulté le mars 15, 2025, sur [ibm.com](https://www.ibm.com/blogs/ibm-france/2024/02/26/limportance-dune-culture-devops-au-sein-dune-entreprise/): <https://www.ibm.com/blogs/ibm-france/2024/02/26/limportance-dune-culture-devops-au-sein-dune-entreprise/>
- [6] Arua, U. F. (2024, Janvier 08). *blog*. Consulté le Mars 5, 2025 sur [civo.com](https://www.civo.com/blog/the-role-of-the-ci-cd-pipeline-in-cloud-computing): <https://www.civo.com/blog/the-role-of-the-ci-cd-pipeline-in-cloud-computing>
- [7] wikipedia. (2025, mars 11). *Jenkins (software)*. Consulté le mars 23, 2025, sur [wikipedia.org](https://en.wikipedia.org/wiki/Jenkins_(software)): [https://en.wikipedia.org/wiki/Jenkins_\(software\)](https://en.wikipedia.org/wiki/Jenkins_(software))
- [8] SonarSource SA. (2025). *sonarqube-server*. Consulté le mars 23, 2025, sur [docs.sonarsource.com](https://docs.sonarsource.com/sonarqube-server/10.8/): <https://docs.sonarsource.com/sonarqube-server/10.8/>
- [9] sonatype. (2008). *sonatype-nexus-repository*. Consulté le mars 20, 2025, sur <https://www.sonatype.com/>: <https://www.sonatype.com/products/sonatype-nexus-repository>
- [10] wikipedia. (2025, mars 5). *Ubuntu (système d'exploitation)*. Consulté le mars 20, 2025, sur [wikipedia.org](https://fr.wikipedia.org/wiki/Ubuntu_(syst%C3%A8me_d%27exploitation)): [https://fr.wikipedia.org/wiki/Ubuntu_\(syst%C3%A8me_d%27exploitation\)](https://fr.wikipedia.org/wiki/Ubuntu_(syst%C3%A8me_d%27exploitation))
- [11] mobatek. (2025). *MobaXterm features*. Consulté le Mars 10, 2025, sur [mobatek.net](https://mobaxterm.mobatek.net/features.html): <https://mobaxterm.mobatek.net/features.html>
- [12] Kubernetes. (2024, September 11). *overview*. Consulté le avril 5, 2025, sur [kubernetes.io](https://kubernetes.io/docs/concepts/overview/): <https://kubernetes.io/docs/concepts/overview/>

- [13] Docker. (2025). *quickstart*. Consulté le avril 08, 2025, sur docs.docker.com:
<https://docs.docker.com/docker-hub/quickstart>
- [14] Labs, G. (2025). *introduction*. Consulté le avril 8, 2025, sur grafana.com:
<https://grafana.com/docs/grafana/latest/introduction/>
- [15] Prometheus. (2025). *overview*. Consulté le avril 8, 2025, sur prometheus.io:
<https://prometheus.io/docs/introduction/overview/>

Résumé

Ce projet de fin d'études s'inscrit dans le cadre d'une licence en Génie Logiciel et Systèmes d'Information, et porte sur la conception et le développement d'une plateforme centralisée dédiée à l'automatisation, la supervision et la gestion des opérations DevOps au sein d'une équipe technique.

Développée selon une architecture modulaire en Node.js, cette solution intègre un backend sécurisé, un frontend moderne et responsive, ainsi qu'un système d'interactions avec des outils essentiels tels que Jenkins, SonarQube, Nexus, GitHub, Grafana et Kubernetes. La plateforme offre une gestion fine des rôles (administrateur, DevOps, développeur, superviseur), avec des interfaces personnalisées, des dashboards temps réel et un système de notifications avancé.

Au-delà du développement logiciel, le projet inclut la mise en place de toute l'infrastructure DevOps sur trois machines virtuelles Ubuntu, avec l'installation et la configuration complète de l'environnement CI/CD et de monitoring.

Réalisé en seulement quatre mois, ce projet ambitieux combine plusieurs sous-projets en une solution unifiée, opérationnelle et extensible. Il constitue une preuve concrète de notre maîtrise des technologies modernes, de la sécurité, de l'intégration continue, et de la gestion multi-profils dans un contexte DevOps. Des évolutions futures sont prévues, notamment pour adapter la plateforme à la gestion multi-équipes.